

LAPORAN TUGAS PROYEK PENGENALAN POLA

JUDUL PROYEK: Klasifikasi Daun Herbal Menggunakan *Hybrid Feature Extraction* (GLCM, LBP, Fourier Descriptor, dan Deskriptor Bentuk Geometris) dengan Metode SVM, KNN, dan Naive Bayes

MATA KULIAH: Pengenalan Pola

DOSEN PENGAMPU: Prof. I Gede Pasek Suta Wijaya

Disusun Oleh (Kelompok 2):

1. Dodi Wijaya — F1D02310047
2. Muhammad Arafa Yahfazhka — F1D02310126
3. Azizah Nurul Indah Amriani — F1D02310004

PROGRAM STUDI: Teknik Informatika, Fakultas Teknik, Universitas Mataram (FT-UNRAM)

TAHUN: 2026

Summary

Daun herbal seperti jambu biji, kunyit, singkong, sirih, dan paku telah lama dimanfaatkan dalam pengobatan tradisional di Indonesia, namun identifikasi jenisnya secara visual masih banyak dilakukan secara manual berdasarkan pengetahuan empiris masyarakat, yang rentan terhadap kesalahan terutama bagi orang yang tidak terlatih. Proyek ini mengembangkan sistem klasifikasi otomatis untuk lima jenis daun herbal (Jambu, Kunyit, Paku, Singkong, dan Sirih) menggunakan pendekatan *hybrid feature extraction* yang menggabungkan fitur tekstur (*Gray Level Co-occurrence Matrix*/GLCM dan *Local Binary Pattern*/LBP) dengan dua representasi fitur bentuk yang berbeda (*Fourier Descriptor* dan deskriptor bentuk geometris klasik), sebagai respons atas *research gap* pada penelitian terdahulu yang umumnya hanya menggunakan satu jenis fitur saja atau bergantung pada *deep learning* yang membutuhkan dataset besar dan komputasi tinggi. Dataset dikumpulkan secara mandiri menggunakan kamera *smartphone*, melalui tahap pra-proses berupa penghapusan latar belakang berbasis segmentasi objek salien (*rembg/U²-Net*), augmentasi data (*flip* dan rotasi) khusus pada data latih, konversi *grayscale*, *thresholding*, dan deteksi kontur. Sepuluh skenario kombinasi fitur diuji secara sistematis dan dibandingkan menggunakan tiga algoritma *machine learning* yang ringan — *Support Vector Machine* (SVM) berkernel linear, *K-Nearest Neighbor* (KNN), dan *Naive Bayes* — untuk menentukan kombinasi fitur dan model dengan performa klasifikasi terbaik. [Lengkapi paragraf ini dengan hasil akhir: skenario dan model terbaik beserta akurasi yang dicapai, setelah seluruh skenario dijalankan pada bagian Hasil dan Pembahasan.]

Kata Kunci: klasifikasi daun herbal, *hybrid feature extraction*, GLCM, *Fourier Descriptor*, *Support Vector Machine*

1. PENDAHULUAN

1.1 Latar Belakang

Indonesia memiliki keanekaragaman tanaman herbal yang sangat tinggi dan telah dimanfaatkan secara turun-temurun sebagai obat tradisional. Daun jambu biji (*Psidium guajava*) digunakan untuk mengatasi diare dan memiliki aktivitas antibakteri; daun kunyit (*Curcuma longa*) bersifat antiinflamasi dan digunakan dalam jamu; daun sirih (*Piper betle*) bersifat antiseptik dan membantu penyembuhan luka; daun singkong (*Manihot utilissima*) kaya vitamin dan mineral untuk daya tahan tubuh; serta berbagai jenis daun paku (*Pteridophyta*) mengandung antioksidan alami. Pengetahuan untuk membedakan jenis-jenis daun ini secara visual masih banyak bersifat empiris dan diwariskan secara lisan, sehingga rentan terhadap kesalahan identifikasi, terutama bagi generasi muda atau praktisi kesehatan yang tidak terlatih secara botani. Sistem klasifikasi citra daun otomatis memiliki urgensi praktis untuk mendukung dokumentasi pengetahuan etnobotani, edukasi masyarakat, serta potensi aplikasi identifikasi tanaman obat berbasis *mobile* atau web.

Kajian Literatur Terkini (State of the Art):

Peneliti	Judul	Dataset	Fitur	Metode	Hasil	Kelemahan
Furqan dkk. (2020)	Klasifikasi Daun Bugenvil Menggunakan GLCM dan KNN	4 jenis daun bugenvil	GLCM (Contrast, Correlation, Energy, Homogeneity)	KNN	Akurasi 87%	Hanya fitur tekstur, bentuk daun tidak digunakan
Herdiansah dkk. (2022)	Klasifikasi Citra Daun Herbal dengan <i>Backpropagation Neural Network</i> Berdasarkan Ekstraksi Ciri Bentuk	Beberapa jenis daun herbal	Operasi morfologi, <i>Metric, Eccentricity</i>	BNN	Akurasi 88,75%	Hanya fitur bentuk, kurang optimal untuk citra <i>close-up</i> tekstur
Prayoga dkk. (2023)	Klasifikasi Daun Herbal Menggunakan KNN dan SVM dengan Fitur <i>Fourier Descriptor</i>	480 citra daun katuk & kelor (80:20)	Grayscale, binerisasi, <i>Fourier Descriptor</i>	KNN, SVM	KNN 92–94%, SVM 96%	Fokus representasi bentuk, belum mengombinasikan fitur tekstur secara eksplisit
Azzahra	Klasifikasi	480 citra	GLCM	CNN,	CNN 98%,	Dataset

Peneliti	Judul	Dataset	Fitur	Metode	Hasil	Kelemahan
dkk. (2023)	Daun Herbal Menggunakan CNN dan <i>Naive Bayes</i> dengan Fitur GLCM	daun katuk & kelor	(Contrast, Correlation, Homogeneity, Dissimilarity, ASM)	Naive Bayes	NB Multinomial 90%, Gaussian 77%, Bernoulli 46%	hanya 2 kelas, CNN butuh komputasi tinggi
Nurhikam dkk. (2024)	Klasifikasi Tanaman Obat Berdasarkan Citra Daun Menggunakan BNN	40 citra, 4 kelas	Segmentasi, morfologi, <i>Metric</i> , <i>Eccentricity</i>	BNN	Akurasi 93,75%	Dataset kecil, dominan fitur bentuk
Wardhani dkk. (2026)	Klasifikasi Daun Herbal Bertulang Menyirip Menggunakan KNN	1.200 citra, 2 jenis	<i>Deep Feature Extraction</i> (ResNet-50)	KNN	Akurasi 98,33%	Hanya 2 kelas, bergantung pada fitur <i>deep learning</i>
Abdiel dkk. (2026)	Klasifikasi 10 Jenis Daun Herbal Indonesia Menggunakan MobileNet V2 Berbasis Web	Indonesian Herb Leaf Dataset, 3.500 citra (10 jenis)	Resize, normalisasi, <i>feature learning</i> CNN	MobileNet V2	Akurasi 92%	Butuh dataset besar dan sumber daya komputasi tinggi

Analisis SOTA: Pola yang konsisten terlihat pada Tabel di atas: penelitian yang hanya mengandalkan fitur tekstur (Furqan dkk., 2020; Azzahra dkk., 2023) tidak dapat menangkap perbedaan bentuk daun yang seringkali menjadi pembeda paling intuitif antar spesies, sementara penelitian yang hanya mengandalkan fitur bentuk (Herdiansah dkk., 2022; Nurhikam dkk., 2024; Prayoga dkk., 2023) kurang optimal ketika citra diambil dari jarak dekat (*close-up*) sehingga bentuk daun secara utuh tidak terlihat dan permukaan/tekstur daun menjadi informasi yang lebih dominan. Di sisi lain, pendekatan berbasis *deep learning* (Azzahra dkk., 2023; Wardhani dkk., 2026; Abdiel dkk., 2026) memang mampu mencapai akurasi tinggi (92–98%), tetapi membutuhkan dataset berskala besar (ribuan citra) dan sumber daya komputasi (GPU) yang tidak selalu tersedia, terutama dalam konteks tugas akademik atau aplikasi skala kecil.

Research Gap: Berdasarkan kajian di atas, penelitian-penelitian terdahulu masih berfokus pada penggunaan **satu jenis fitur saja** — baik hanya fitur tekstur berbasis GLCM, atau hanya fitur bentuk melalui *Fourier Descriptor*/deskriptor morfologi — tanpa mengombinasikan keduanya secara eksplisit dan sistematis. Penelitian berbasis *deep learning* di sisi lain menghasilkan akurasi tinggi namun dengan biaya komputasi dan kebutuhan data yang besar. Proyek ini mengisi kekosongan tersebut dengan mengembangkan metode klasifikasi daun herbal yang **mengombinasikan fitur tekstur dan bentuk** (*hybrid feature extraction*) menggunakan algoritma *machine learning* yang lebih ringan seperti SVM, KNN, dan *Naive Bayes*, untuk memperoleh performa klasifikasi yang baik tanpa kebutuhan komputasi sebesar *deep learning*.

1.2 Rumusan Masalah

1. Bagaimana cara mengekstraksi dan mengombinasikan fitur tekstur (GLCM, LBP) dan fitur bentuk (*Fourier Descriptor*, deskriptor geometris) yang tepat dari citra daun herbal, mengingat variasi kondisi pengambilan citra (latar belakang, pencahayaan, jarak) yang memengaruhi kualitas ekstraksi fitur?
2. Bagaimana performa algoritma SVM, KNN, dan *Naive Bayes* dalam mengenali pola lima jenis daun herbal (Jambu, Kunyit, Paku, Singkong, Sirih) berdasarkan kombinasi fitur *hybrid* tersebut?
3. Kombinasi fitur dan algoritma klasifikasi manakah yang memberikan performa terbaik, dan apakah penambahan fitur bentuk benar-benar meningkatkan akurasi dibandingkan fitur tekstur saja (sesuai klaim *hybrid feature extraction* yang diajukan)?

1.3 Tujuan Proyek

1. Menerapkan metode *hybrid feature extraction* (GLCM, LBP, *Fourier Descriptor*, dan deskriptor bentuk geometris) untuk mengenali dan mengklasifikasikan lima jenis daun herbal.
2. Menganalisis tingkat akurasi dan performa model SVM, KNN, dan *Naive Bayes* berdasarkan sepuluh skenario kombinasi fitur yang diuji secara sistematis.
3. Mengidentifikasi kombinasi fitur tekstur dan bentuk yang paling berkontribusi terhadap performa klasifikasi, sebagai bukti empiris atas manfaat pendekatan *hybrid* dibandingkan penggunaan satu jenis fitur saja.

2. DATASET DAN PRA-PROSES DATA (PREPROCESSING)

2.1 Deskripsi Dataset

- **Sumber Data:** Data primer, dikumpulkan secara mandiri menggunakan kamera *smartphone*. Pengambilan citra dilakukan langsung dari lingkungan sekitar dengan variasi jarak pengambilan (*close-up*, *medium*, *wide*), sudut *top view*, kondisi pencahayaan (*indoor* dan *outdoor*), latar belakang (kertas polos dan lantai), serta posisi daun (bagian depan dan belakang) — variasi ini sengaja dilakukan untuk mengurangi bias dataset dan meningkatkan kemampuan generalisasi model terhadap kondisi identifikasi di dunia nyata.
- **Jumlah Data:** Total data, jumlah data latih, dan data uji ditampilkan pada output kode di bawah (lihat cetakan **Total Data** dan **Jumlah Kelas**). Data dibagi dengan rasio 80% data latih : 20% data uji menggunakan `train_test_split` dengan parameter `stratify` (lihat pembahasan rasio ini lebih lanjut pada Bagian 4.2), dilakukan **sebelum** tahap augmentasi maupun ekstraksi fitur — ini penting agar citra hasil augmentasi dari

data latih tidak pernah "bocor" ke dalam data uji, yang dapat membuat estimasi akurasi menjadi terlalu optimis (*data leakage*; Shorten & Khoshgoftaar, 2019).

- **Fitur/Atribut:** Citra RGB, diresize ke resolusi **128×128 piksel** agar konsisten dan efisien secara komputasi untuk seluruh tahap ekstraksi fitur tekstur dan bentuk.
- **Target/Kelas (Label):** Lima jenis daun herbal — **Jambu** (*Psidium guajava*), **Kunyit** (*Curcuma longa*), **Paku** (Pteridophyta), **Singkong** (*Manihot utilissima*), dan **Sirih** (*Piper betle*).

Kode pemuatan dataset, *encoding* label, pembagian data latih/uji, dan visualisasi sampel ditampilkan pada cell-cell di bawah ini.

```
import os
import cv2
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from skimage.measure import regionprops

from tqdm import tqdm
import joblib
from rembg import remove

from skimage.feature import graycomatrix, graycoprops,
local_binary_pattern

from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix, ConfusionMatrixDisplay

c:\Users\tuf51\AppData\Local\Programs\Python\Python314\Lib\site-
packages\tqdm\auto.py:21: TqdmWarning: IProgress not found. Please
update jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
from .autonotebook import tqdm as notebook_tqdm
```

Tahap awal implementasi dilakukan dengan mengimpor seluruh *library* yang dibutuhkan untuk proses pengolahan citra, ekstraksi fitur, pelatihan model, hingga evaluasi hasil. Pemilihan setiap *library* didasarkan pada kemampuannya dalam mendukung tahapan penelitian secara efisien serta telah banyak digunakan pada penelitian di bidang *computer vision* dan *machine learning*.

- **OpenCV (cv2)** digunakan sebagai pustaka utama untuk pengolahan citra digital, seperti membaca citra, mengubah ruang warna, *resize*, *thresholding*, operasi morfologi, dan deteksi kontur. OpenCV dipilih karena menyediakan implementasi

algoritma pengolahan citra yang efisien dan telah menjadi standar dalam berbagai penelitian *computer vision* (Bradski & Kaehler, 2008).

- **NumPy (numpy)** digunakan untuk merepresentasikan citra sebagai array multidimensi serta melakukan operasi numerik yang efisien. Hampir seluruh algoritma pada penelitian ini bekerja menggunakan struktur data array dari NumPy sehingga proses komputasi menjadi lebih cepat (Harris et al., 2020).
- **Pandas (pandas)** digunakan untuk menyusun, mengelola, dan menampilkan hasil eksperimen dalam bentuk tabel sehingga mempermudah analisis performa setiap skenario pengujian (McKinney, 2010).
- **Matplotlib** dan **Seaborn** digunakan untuk visualisasi data, termasuk menampilkan sampel citra, grafik, serta *confusion matrix*. Seaborn dipilih karena menyediakan visualisasi statistik yang lebih informatif di atas Matplotlib (Waskom, 2021).
- **scikit-image (regionprops, graycomatrix, graycoprops, local_binary_pattern)** digunakan karena menyediakan implementasi resmi berbagai metode ekstraksi fitur citra klasik seperti GLCM, LBP, dan analisis bentuk objek (*region properties*). Library ini banyak digunakan dalam penelitian pengenalan pola karena implementasinya telah tervalidasi secara ilmiah (van der Walt et al., 2014).
- **rembg** digunakan untuk menghapus latar belakang citra secara otomatis. Library ini memanfaatkan model **U²-Net**, yaitu jaringan *deep learning* yang dirancang untuk *salient object detection*. Pendekatan ini dipilih karena lebih robust dibanding segmentasi berbasis warna terhadap variasi latar belakang dan pencahayaan (Qin et al., 2020).
- **scikit-learn** digunakan sebagai pustaka utama pada tahap *machine learning*. Library ini menyediakan implementasi berbagai algoritma klasifikasi seperti Support Vector Machine (SVM), K-Nearest Neighbor (KNN), dan Gaussian Naive Bayes, serta utilitas penting seperti *train-test split*, *label encoding*, *standardization*, dan metrik evaluasi. Scikit-learn dipilih karena merupakan salah satu pustaka *machine learning* yang paling banyak digunakan dalam penelitian akademik (Pedregosa et al., 2011).
- **joblib** digunakan untuk menyimpan model, *scaler*, dan *label encoder* yang telah dilatih sehingga model dapat digunakan kembali tanpa proses pelatihan ulang. Joblib lebih efisien dibandingkan modul *pickle* untuk menyimpan objek NumPy berukuran besar (Pedregosa et al., 2011).
- **tqdm** digunakan untuk menampilkan *progress bar* selama proses yang membutuhkan waktu cukup lama sehingga memudahkan pemantauan proses eksekusi program.

```
DATASET_PATH = "DATASET"  
IMG_SIZE = (128,128)
```

Pada tahap ini ditentukan lokasi dataset serta ukuran citra yang akan digunakan selama proses penelitian.

```
DATASET_PATH = "DATASET"  
IMG_SIZE = (128,128)
```

- **DATASET_PATH** digunakan untuk menentukan direktori utama tempat seluruh citra daun herbal disimpan. Dataset disusun berdasarkan folder kelas sehingga setiap folder merepresentasikan satu jenis daun herbal.
- **IMG_SIZE = (128,128)** digunakan untuk menyeragamkan ukuran seluruh citra sebelum dilakukan proses ekstraksi fitur. Proses *resize* diperlukan karena citra hasil pengambilan menggunakan kamera memiliki resolusi yang berbeda-beda. Keseragaman ukuran akan memudahkan proses pengolahan citra dan memastikan setiap citra memiliki dimensi yang sama.

Ukuran **128 × 128 piksel** dipilih karena mampu mempertahankan informasi tekstur dan bentuk daun yang dibutuhkan pada ekstraksi fitur GLCM, LBP, Fourier Descriptor, dan Shape Descriptor, sekaligus mengurangi beban komputasi dibandingkan menggunakan resolusi yang lebih besar seperti 224×224 atau ukuran asli. Pada pendekatan berbasis *handcrafted feature extraction*, ukuran citra yang terlalu besar tidak selalu meningkatkan performa, namun justru dapat meningkatkan waktu komputasi dan kompleksitas proses ekstraksi fitur (Gonzalez & Woods, 2018).

Selain itu, seluruh citra pada penelitian ini diproses menggunakan ukuran yang sama sehingga fitur yang dihasilkan memiliki representasi yang konsisten pada setiap sampel.

```
def load_dataset(dataset_path):  
    images = []  
    labels = []  
  
    for label in sorted(os.listdir(dataset_path)):  
        folder = os.path.join(dataset_path, label)  
  
        if not os.path.isdir(folder):  
            continue  
  
        for file in os.listdir(folder):  
            if file.lower().endswith(('.jpg', '.jpeg', '.png')):  
                path = os.path.join(folder, file)
```

```

        img = cv2.imread(path)

        if img is None:
            continue

        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        img = cv2.resize(img, IMG_SIZE)

        images.append(img)
        labels.append(label)

    return np.array(images), np.array(labels)

```

Memuat Dataset

Tahap ini bertujuan untuk membaca seluruh citra daun herbal dari direktori dataset dan mengubahnya menjadi data yang siap diproses pada tahap *preprocessing* dan ekstraksi fitur.

```
def load_dataset(dataset_path):
```

Fungsi `load_dataset()` melakukan beberapa tahapan sebagai berikut.

1. Membaca Struktur Dataset

Dataset disusun berdasarkan folder, di mana setiap folder merepresentasikan satu kelas daun herbal. Nama folder digunakan sebagai **label kelas**, sedangkan seluruh file gambar di dalamnya menjadi sampel dari kelas tersebut.

2. Membaca Citra

Setiap file dengan ekstensi **JPG**, **JPEG**, atau **PNG** dibaca menggunakan fungsi `cv2.imread()`. File yang gagal dibaca (**None**) akan dilewati sehingga tidak mengganggu proses selanjutnya.

3. Konversi Ruang Warna

OpenCV secara bawaan membaca citra dalam format **BGR (Blue-Green-Red)**, sedangkan sebagian besar pustaka pengolahan citra seperti Matplotlib dan scikit-image menggunakan format **RGB (Red-Green-Blue)**. Oleh karena itu dilakukan konversi menggunakan:

```
cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

Konversi ini dilakukan agar representasi warna tetap konsisten selama proses visualisasi maupun pengolahan citra (Bradski & Kaehler, 2008).

1. Penyeragaman Ukuran Citra

Seluruh citra kemudian diubah menjadi ukuran **128 × 128 piksel** menggunakan:

```
cv2.resize(img, IMG_SIZE)
```

Proses *resize* diperlukan agar seluruh citra memiliki dimensi yang sama sehingga proses ekstraksi fitur dapat dilakukan secara konsisten. Ukuran 128×128 dipilih karena masih mampu mempertahankan karakteristik tekstur dan bentuk daun, namun lebih efisien secara komputasi dibandingkan resolusi yang lebih besar (Gonzalez & Woods, 2018).

1. Penyimpanan Data

Seluruh citra disimpan ke dalam variabel `images`, sedangkan nama folder disimpan sebagai `labels`. Pada akhir proses, keduanya dikonversi menjadi **NumPy Array** agar dapat diproses secara efisien oleh pustaka *machine learning*.

Output fungsi ini berupa:

- **images** → kumpulan citra daun herbal berukuran 128 × 128 piksel.
- **labels** → label kelas setiap citra berdasarkan nama folder.

```
X_raw, y = load_dataset(DATASET_PATH)
print("Total Data :", len(X_raw))
print("Jumlah Kelas :", len(np.unique(y)))
```

```
Total Data : 279
Jumlah Kelas : 5
```

Setelah fungsi `load_dataset()` didefinisikan, langkah berikutnya adalah memuat seluruh citra ke dalam memori serta melakukan verifikasi awal terhadap dataset.

```
X_raw, y = load_dataset(DATASET_PATH)
```

Variabel yang dihasilkan terdiri dari:

- **X_raw** berisi seluruh citra daun herbal dalam bentuk array NumPy dengan ukuran **128 × 128 piksel**.
- **y** berisi label kelas yang diperoleh dari nama folder dataset.

Selanjutnya dilakukan pengecekan jumlah data dan jumlah kelas menggunakan:

```
print("Total Data :", len(X_raw))
print("Jumlah Kelas :", len(np.unique(y)))
```

Tahap ini bertujuan untuk memastikan bahwa seluruh citra berhasil dimuat dengan benar serta seluruh kelas telah terdeteksi sebelum proses *preprocessing*, pembagian data, maupun pelatihan model dilakukan.

Fungsi `np.unique()` digunakan untuk memperoleh seluruh label unik pada dataset sehingga jumlah kelas dapat diketahui secara otomatis tanpa perlu ditentukan secara manual. Pendekatan

ini mengurangi risiko kesalahan apabila terdapat perubahan jumlah kelas pada dataset di masa mendatang.

Verifikasi awal dataset merupakan langkah penting dalam proses *machine learning*, karena dapat membantu mendeteksi kesalahan seperti citra yang gagal dimuat, kelas yang hilang, maupun ketidaksesuaian struktur dataset sebelum memasuki tahap analisis yang lebih kompleks (Pedregosa et al., 2011).

Output dari kode ini menampilkan:

- **Total Data** → jumlah seluruh citra yang berhasil dimuat.
- **Jumlah Kelas** → banyaknya kategori daun herbal yang akan diklasifikasikan.

Hasil pada penelitian ini menunjukkan bahwa dataset terdiri atas **279 citra** yang terbagi ke dalam **5 kelas**, yaitu Jambu, Kunyit, Paku, Singkong, dan Sirih.

```
encoder = LabelEncoder()  
y_encoded = encoder.fit_transform(y)
```

Encoding Label Kelas

Tahap berikutnya adalah mengubah label kelas yang masih berbentuk teks (*string*) menjadi nilai numerik menggunakan **LabelEncoder** dari pustaka *scikit-learn*.

```
encoder = LabelEncoder()  
y_encoded = encoder.fit_transform(y)
```

Pada dataset, setiap citra masih memiliki label berupa nama kelas, misalnya:

- Jambu
- Kunyit
- Paku
- Singkong
- Sirih

Sebagian besar algoritma *machine learning*, termasuk **Support Vector Machine (SVM)**, **K-Nearest Neighbor (KNN)**, dan **Gaussian Naive Bayes**, tidak dapat memproses label bertipe teks secara langsung. Oleh karena itu, setiap kelas perlu direpresentasikan dalam bentuk bilangan bulat (*integer encoding*).

Melalui proses **Label Encoding**, setiap kelas akan diberikan indeks numerik yang unik, misalnya:

Label Asli	Label Hasil Encoding
Jambu	0
Kunyit	1
Paku	2
Singkong	3
Sirih	4

(Urutan indeks mengikuti hasil pengurutan alfabet oleh `LabelEncoder`.)

Perlu diperhatikan bahwa proses ini **tidak memberikan makna urutan (ordinal)** pada setiap kelas, melainkan hanya mengubah representasi label agar dapat diproses oleh algoritma klasifikasi. Informasi mengenai hubungan antar kelas tidak berubah karena algoritma hanya menggunakan nilai tersebut sebagai identitas kelas.

Selain digunakan pada proses pelatihan model, objek `encoder` juga disimpan sehingga hasil prediksi yang masih berupa angka dapat dikonversi kembali menjadi nama kelas menggunakan fungsi `inverse_transform()`. Hal ini memudahkan interpretasi hasil klasifikasi pada tahap pengujian maupun implementasi sistem.

```
X_train_raw, X_test_raw, y_train, y_test = train_test_split(
    X_raw,
    y_encoded,
    test_size=0.2,
    stratify=y_encoded,
    random_state=42
)
```

Sebelum proses *preprocessing* lanjutan, ekstraksi fitur, dan pelatihan model dilakukan, dataset terlebih dahulu dibagi menjadi **data latih (*training set*)** dan **data uji (*testing set*)** menggunakan fungsi `train_test_split()` dari pustaka *scikit-learn*.

```
X_train_raw, X_test_raw, y_train, y_test = train_test_split(
    X_raw,
    y_encoded,
    test_size=0.2,
    stratify=y_encoded,
    random_state=42
)
```

Parameter yang digunakan memiliki fungsi sebagai berikut.

1. `test_size = 0.2`

Parameter ini menunjukkan bahwa **20% data digunakan sebagai data uji**, sedangkan **80% sisanya digunakan sebagai data latih**.

Rasio **80:20** dipilih karena merupakan salah satu pembagian data yang paling umum digunakan pada penelitian klasifikasi, terutama ketika jumlah dataset relatif kecil hingga menengah. Rasio ini memberikan jumlah data latih yang cukup besar agar model mampu mempelajari pola dengan baik, sekaligus tetap menyediakan data uji yang representatif untuk mengevaluasi kemampuan generalisasi model terhadap data yang belum pernah dilihat sebelumnya (Géron, 2022; Pedregosa et al., 2011).

2. `stratify = y_encoded`

Parameter `stratify` digunakan untuk mempertahankan **proporsi setiap kelas** pada data latih maupun data uji.

Sebagai contoh, apabila kelas **Daun Paku** memiliki proporsi sekitar 22% dari seluruh dataset, maka proporsi tersebut akan tetap dipertahankan pada data latih maupun data uji.

Pendekatan ini sangat penting pada permasalahan klasifikasi multikelas karena dapat mengurangi kemungkinan terjadinya distribusi kelas yang tidak seimbang (*class imbalance*) setelah proses pembagian data. Dengan demikian, hasil evaluasi model menjadi lebih representatif terhadap distribusi data asli (Kuhn & Johnson, 2013).

3. `random_state = 42`

Parameter `random_state` digunakan untuk mengatur nilai *seed* pada proses pembagian data sehingga hasil pembagian dataset selalu konsisten setiap kali program dijalankan.

Penggunaan nilai tetap (42) bertujuan meningkatkan **reproducibility**, yaitu kemampuan penelitian untuk menghasilkan pembagian data yang sama ketika kode dijalankan kembali oleh peneliti lain. Nilai **42** sendiri tidak memiliki makna khusus secara matematis dan dipilih karena telah menjadi konvensi yang banyak digunakan dalam berbagai contoh implementasi *machine learning*.

Hasil Pembagian Dataset

Melalui konfigurasi tersebut, dataset dibagi menjadi:

- **80%** sebagai data latih (*training set*), yang digunakan untuk membangun model klasifikasi.
- **20%** sebagai data uji (*testing set*), yang digunakan untuk mengevaluasi performa model pada data yang belum pernah digunakan selama proses pelatihan.

Pembagian data dilakukan **sebelum proses augmentasi**, sehingga seluruh citra hasil augmentasi hanya berasal dari data latih. Pendekatan ini bertujuan mencegah terjadinya **data leakage**, yaitu kondisi ketika citra yang memiliki kemiripan tinggi dengan data latih ikut masuk ke dalam data uji sehingga dapat menghasilkan estimasi akurasi yang terlalu optimistis (Shorten & Khoshgoftaar, 2019).

```
kelas_unik = sorted(np.unique(y))

fig, axes = plt.subplots(
    1,
    len(kelas_unik),
    figsize=(15,4)
)

for ax, label in zip(axes, kelas_unik):
    idx = np.where(y == label)[0][0]

    ax.imshow(X_raw[idx])
    ax.set_title(label)
    ax.axis("off")

plt.show()
```



2.2 Pra-proses Data (Data Preprocessing)

Pipeline pra-proses yang digunakan terdiri dari lima tahap berurutan, masing-masing dengan justifikasi sebagai berikut:

a. Penghapusan Latar Belakang (*Background Removal*) Latar belakang citra (kertas/lantai) dihilangkan menggunakan *library rembg*, yang secara internal memanfaatkan arsitektur *deep learning U²-Net* (Qin et al., 2020) — sebuah *nested U-structure* yang dirancang khusus untuk *salient object detection* (deteksi objek visual yang menonjol). Pendekatan ini dipilih dibandingkan *thresholding* warna manual (mis. *masking* HSV) karena lebih robust terhadap variasi warna latar belakang (kertas vs lantai) dan variasi pencahayaan (*indoor/outdoor*) yang ada pada dataset ini, sehingga daun tetap terdeteksi sebagai objek salien terlepas dari warna latar di belakangnya. Pixel latar belakang yang terdeteksi (nilai alpha = 0) diputihkan murni (255,255,255) agar konsisten untuk tahap *thresholding* selanjutnya.

b. Augmentasi Data (*Data Augmentation*) — khusus data latih Karena ukuran dataset primer relatif terbatas, setiap citra latih diperbanyak melalui transformasi geometris: *flip horizontal*, rotasi +15°, dan rotasi -15° (total 4× jumlah data asli, termasuk citra original). Augmentasi geometris berupa *flip* dan rotasi dipilih karena tidak mengubah identitas/kelas objek (daun yang dibalik atau dirotasi tetap merupakan jenis daun yang sama), sesuai dengan kategori *geometric transformation* pada survei augmentasi citra oleh Shorten & Khoshgoftaar (2019). **Penting:** augmentasi hanya diterapkan pada data latih (X_train_aug), bukan data uji (X_test_remove/gray_test), agar evaluasi performa model tetap mencerminkan kemampuan generalisasi pada citra yang benar-benar belum pernah "dilihat" model dalam bentuk apa pun.

c. Konversi *Grayscale* Citra RGB dikonversi ke skala keabuan karena ekstraksi fitur tekstur (GLCM, LBP) bekerja pada intensitas piksel tunggal, bukan informasi warna — ini merupakan praktik standar dalam analisis tekstur citra (Haralick, Shanmugam, & Dinstein, 1973; Gonzalez & Woods, 2018).

d. *Thresholding* Citra *grayscale* dibinerisasi menggunakan *threshold* tetap pada nilai 240 dengan cv2.THRESH_BINARY_INV. Nilai ambang 240 (mendekati nilai piksel maksimum 255) dipilih karena latar belakang pada tahap (a) sudah dipaksa menjadi putih murni (255) — sehingga piksel apa pun yang nilainya di bawah 240 (termasuk tepi daun yang mengalami *anti-aliasing*) dapat diasumsikan sebagai bagian daun, bukan latar belakang. Mode **INV** (*inverse*) digunakan agar daun (lebih gelap dari latar belakang putih) menjadi piksel bernilai 255 (foreground putih) pada citra biner, sehingga kontur yang terdeteksi pada tahap berikutnya benar-benar mengikuti bentuk daun, bukan bentuk latar belakang yang menyentuh tepi *frame*.

e. Deteksi Kontur (*Contour Detection*) Kontur eksternal terbesar diekstraksi dari citra biner menggunakan cv2.findContours dengan mode RETR_EXTERNAL (hanya kontur terluar,

mengabaikan lubang/kontur internal) dan `CHAIN_APPROX_NONE` (menyimpan seluruh titik kontur tanpa penyederhanaan, untuk menjaga presisi bentuk sebelum dipakai pada ekstraksi *Fourier Descriptor*). Kontur inilah yang menjadi dasar bagi seluruh fitur bentuk pada Bagian 3.1.

Kode untuk kelima tahap di atas, beserta visualisasi hasil pada tiap tahap, ditampilkan pada cell-cell berikut.

```
def remove_bg(img):  
    _, buffer = cv2.imencode('.png', cv2.cvtColor(img,  
cv2.COLOR_RGB2BGR))  
  
    output_bytes = remove(buffer.tobytes())  
  
    result = cv2.imdecode(np.frombuffer(output_bytes, np.uint8),  
cv2.IMREAD_UNCHANGED)  
  
    if result.shape[2] == 4:  
        alpha = result[:, :, 3]  
  
        rgb = result[:, :, :3]  
  
        rgb[alpha == 0] = [255, 255, 255]  
  
        return cv2.cvtColor(rgb, cv2.COLOR_BGR2RGB)  
  
    return img  
X_train_remove = []  
for img_rgb in X_train_raw:  
    img_no_bg = remove_bg(  
        img_rgb  
    )  
  
    X_train_remove.append(  
        img_no_bg  
    )  
X_test_remove = []  
for img_rgb in X_test_raw:  
    img_no_bg = remove_bg(  
        img_rgb  
    )  
  
    X_test_remove.append(  
        img_no_bg  
    )
```

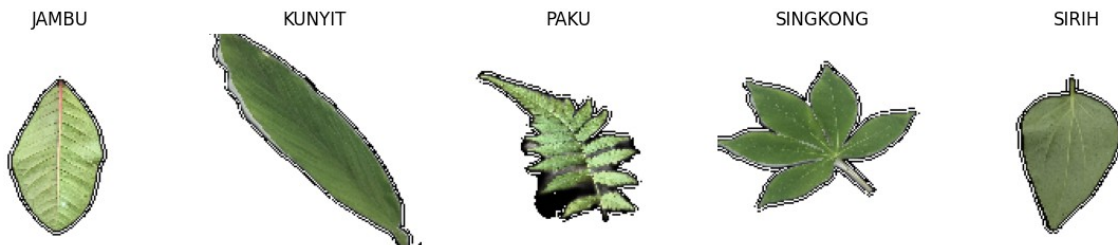
```

fig, axes = plt.subplots(
    1,
    len(kelas_unik),
    figsize=(15,4)
)

for ax, label in zip(axes, kelas_unik):
    idx = np.where(encoder.inverse_transform(y_train) == label)[0][0]
    ax.imshow(X_train_remove[idx])
    ax.set_title(label)
    ax.axis("off")

plt.show()

```



```

def augment_image(img):
    augmented = []
    augmented.append(img)

    # Flip horizontal
    augmented.append(
        cv2.flip(img,1)
    )

    # Rotasi +15
    M = cv2.getRotationMatrix2D(
        (IMG_SIZE[0]//2, IMG_SIZE[1]//2),
        15,
        1
    )

    augmented.append(
        cv2.warpAffine(
            img,
            M,
            IMG_SIZE,
            borderValue=(255,255,255)
        )
    )

```

```

    )
)

# Rotasi -15
M = cv2.getRotationMatrix2D(
    (IMG_SIZE[0]//2, IMG_SIZE[1]//2),
    -15,
    1
)

augmented.append(
    cv2.warpAffine(
        img,
        M,
        IMG_SIZE,
        borderValue=(255,255,255)
    )
)

return augmented

X_train_aug = []
y_train_aug = []

for img, label in zip(X_train_remove, y_train):
    augmented = augment_image(img)

    for aug in augmented:
        X_train_aug.append(aug)
        y_train_aug.append(label)

X_train_aug = np.array(X_train_aug)
y_train_aug = np.array(y_train_aug)

def grayscale(img):
    gray = cv2.cvtColor(
        img,
        cv2.COLOR_RGB2GRAY
    )

    return gray

gray_images = []

for img_bg in X_train_aug:
    gray = grayscale(
        img_bg

```



```

    )
    gray_images.append(
        gray
    )
gray_images = np.array(
    gray_images
)
gray_test = []
for img_bg in X_test_remove:
    gray = grayscale(
        img_bg
    )
    gray_test.append(
        gray
    )
gray_test = np.array(
    gray_test
)
fig, axes = plt.subplots(
    1,
    len(kelas_unik),
    figsize=(15,4)
)
for ax, label in zip(axes, kelas_unik):
    idx = np.where(encoder.inverse_transform(y_train_aug) == label)[0]
    ax.imshow(
        gray_images[idx],
        cmap='gray'
    )
    ax.set_title(label)
    ax.axis("off")
plt.show()

```

JAMBU



KUNYIT



PAKU



SINGKONG



SIRIH



```
def otsu_threshold(gray):
    _, thresh = cv2.threshold(gray, 240, 255, cv2.THRESH_BINARY_INV)
    return thresh

thresh_images = []
for gray in gray_images:
    thresh = otsu_threshold(
        gray
    )
    thresh_images.append(
        thresh
    )
thresh_images = np.array(
    thresh_images
)

thresh_test = []
for gray in gray_test:
    thresh = otsu_threshold(
        gray
    )
    thresh_test.append(
        thresh
    )

thresh_test = np.array(
    thresh_test
)

fig, axes = plt.subplots(
    1,
    len(kelas_unik),
    figsize=(15,4)
)

for ax, label in zip(axes, kelas_unik):
```

```

idx = np.where(encoder.inverse_transform(y_train_aug) == label)[0]
[0]

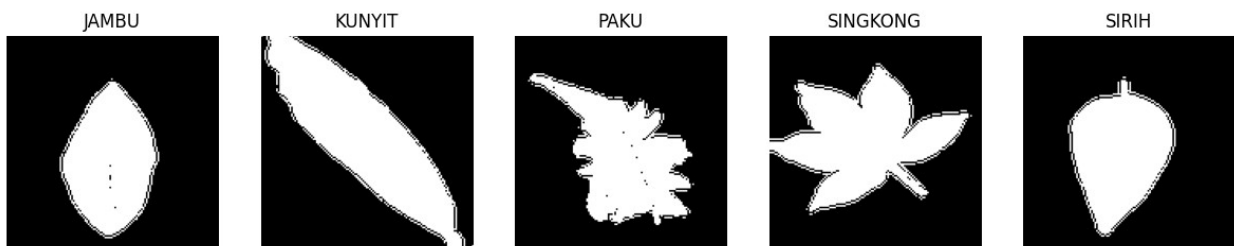
ax.imshow(
    thresh_images[idx],
    cmap='gray'
)

ax.set_title(label)

ax.axis('off')

plt.show()

```



```

def contour_detection(thresh):
    contours, _ = cv2.findContours(
        thresh,
        cv2.RETR_EXTERNAL,
        cv2.CHAIN_APPROX_NONE
    )

    if len(contours) == 0:
        return None

    largest_contour = max(
        contours,
        key=cv2.contourArea
    )

    return largest_contour

```

Setelah citra diubah menjadi citra biner melalui proses *thresholding*, tahap berikutnya adalah mendeteksi kontur objek daun. Kontur digunakan sebagai dasar untuk mengekstraksi fitur bentuk, yaitu **Fourier Descriptor** dan **Deskriptor Bentuk Geometris**.

```

def contour_detection(thresh):

```

Proses deteksi kontur dilakukan menggunakan fungsi `cv2.findContours()` dari OpenCV dengan parameter sebagai berikut.

```
contours, _ = cv2.findContours(  
    thresh,  
    cv2.RETR_EXTERNAL,  
    cv2.CHAIN_APPROX_NONE  
)
```

1. `cv2.RETR_EXTERNAL`

Mode `RETR_EXTERNAL` digunakan untuk mengambil **hanya kontur terluar** dari objek.

Pada penelitian ini, objek yang ingin dikenali adalah **bentuk keseluruhan daun**, sehingga kontur internal seperti lubang kecil, urat daun, atau noise tidak diperlukan. Dengan hanya mengambil kontur terluar, bentuk daun dapat direpresentasikan secara lebih sederhana dan stabil.

Pendekatan ini juga membantu mengurangi kompleksitas komputasi serta mencegah fitur bentuk dipengaruhi oleh detail internal yang tidak relevan terhadap proses klasifikasi (Bradski & Kaehler, 2008).

2. `cv2.CHAIN_APPROX_NONE`

Mode `CHAIN_APPROX_NONE` digunakan agar **seluruh titik kontur disimpan tanpa proses penyederhanaan**.

Parameter ini dipilih karena penelitian ini menggunakan **Fourier Descriptor**, yang memerlukan representasi kontur yang lengkap. Apabila digunakan metode seperti `CHAIN_APPROX_SIMPLE`, sebagian titik kontur akan dihilangkan sehingga bentuk kontur dapat berubah dan memengaruhi hasil transformasi Fourier.

Dengan menyimpan seluruh titik kontur, bentuk daun dapat direpresentasikan secara lebih akurat sehingga informasi geometris yang diekstraksi tetap terjaga (Zahn & Roskies, 1972).

Pemilihan Kontur Terbesar

Setelah seluruh kontur diperoleh, dipilih kontur dengan luas terbesar menggunakan

```
largest_contour = max(  
    contours,  
    key=cv2.contourArea  
)
```

Pemilihan kontur terbesar dilakukan dengan asumsi bahwa objek utama pada citra adalah daun, sedangkan kontur lain umumnya berasal dari noise, bayangan, atau artefak hasil segmentasi.

Luas kontur dihitung menggunakan fungsi `cv2.contourArea()`, kemudian fungsi `max()` digunakan untuk memperoleh kontur dengan area terbesar. Pendekatan ini banyak digunakan pada aplikasi segmentasi objek karena mampu mengurangi pengaruh objek-objek kecil yang tidak relevan (Gonzalez & Woods, 2018).

Apabila tidak ditemukan kontur pada citra hasil *thresholding*, fungsi akan mengembalikan nilai **None**. Penanganan ini dilakukan agar proses ekstraksi fitur berikutnya tidak menghasilkan kesalahan (*error*) ketika menerima citra tanpa objek yang terdeteksi.

Output dari fungsi ini berupa **kontur terbesar** yang selanjutnya digunakan sebagai masukan pada proses ekstraksi **Fourier Descriptor** dan **Deskriptor Bentuk Geometris**.

```
contour_images = []
for thresh in thresh_images:
    contour = contour_detection(thresh)
    contour_images.append(
        contour
    )
contour_images = np.array(
    contour_images,
    dtype=object
)
contour_test = []
for thresh in thresh_test:
    contour = contour_detection(
        thresh
    )
    contour_test.append(
        contour
    )
contour_test = np.array(
    contour_test,
    dtype=object
)
kelas_unik = encoder.classes_
labels = encoder.inverse_transform(y_train_aug)
fig, axes = plt.subplots(
    1,
    len(kelas_unik),
    figsize=(15,4)
)
for ax, label in zip(axes, kelas_unik):
    idx = np.where(labels == label)[0][0]
```

```

# Canvas kosong
contour_canvas = np.zeros(
    (IMG_SIZE[1], IMG_SIZE[0]),
    dtype=np.uint8
)

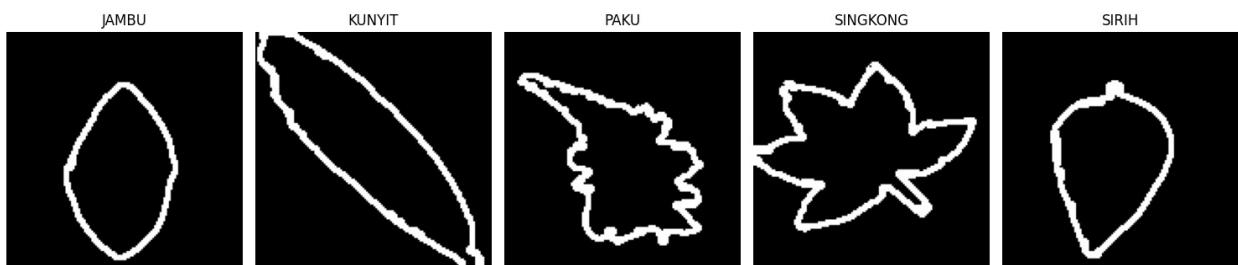
# Gambar contour jika ada
if contour_images[idx] is not None:
    cv2.drawContours(
        contour_canvas,
        [contour_images[idx]],
        -1,
        255,
        2
    )

ax.imshow(
    contour_canvas,
    cmap="gray"
)

ax.set_title(label)
ax.axis("off")

plt.tight_layout()
plt.show()

```



3. METODOLOGI DAN ARSITEKTUR MODEL

3.1 Ekstraksi Fitur (*Feature Extraction*)

Sesuai dengan *research gap* yang diidentifikasi pada Bagian 1.1, proyek ini menggunakan pendekatan **konvensional/handcrafted feature extraction** (bukan *deep learning*) yang secara eksplisit menggabungkan dua kategori fitur:

Kategori Fitur Tekstur:

- **GLCM (*Gray Level Co-occurrence Matrix*)** — diperkenalkan oleh Haralick, Shanmugam, & Dinstein (1973), GLCM menangkap distribusi statistik pasangan piksel bertetangga pada jarak dan sudut tertentu untuk mengukur properti tekstur seperti *contrast* dan *correlation*. Pada implementasi ini, GLCM dihitung pada **tiga jarak piksel (1, 3, 5)** dan dua

sudut (0° dan 90°) — penggunaan multi-jarak bertujuan menangkap pola tekstur permukaan daun (mis. kerapatan urat daun) pada skala spasial yang berbeda (halus hingga kasar), bukan hanya piksel yang langsung bertetangga. Dari seluruh kombinasi jarak/sudut/properti GLCM yang mungkin, hanya properti **Contrast** (pada seluruh kombinasi jarak×sudut) dan **Correlation** (khusus sudut 90°) yang dipertahankan sebagai fitur akhir — properti lain (*Dissimilarity, Homogeneity, Energy, ASM*) terbukti secara empiris hampir seragam nilainya di seluruh kelas pada analisis *heatmap* profil fitur (nilai rata-rata per kelas), sehingga tidak memberikan daya pembeda yang berarti dan dibuang untuk mengurangi dimensi fitur sekaligus *noise*.

- **LBP (Local Binary Pattern)** — diperkenalkan oleh Ojala, Pietikäinen, & Mäenpää (2002), LBP menangkap pola tekstur lokal yang invariant terhadap rotasi dan skala keabuan dengan membandingkan setiap piksel terhadap piksel-piksel di sekitarnya dalam radius tertentu. Implementasi ini menggunakan **LBP multi-resolusi** pada radius 1 (8 titik) dan radius 2 (16 titik) dengan metode *uniform*, sesuai prinsip multi-resolusi yang diajukan Ojala et al. (2002) untuk menangkap tekstur pada skala berbeda. Dari histogram LBP yang dihasilkan, hanya *bin* tertentu (*R1_Bin_6, R2_Bin_10, R2_Bin_12, R2_Bin_14*) yang dipertahankan, berdasarkan analisis *heatmap* yang menunjukkan *bin-bin* tersebut paling konsisten membedakan profil rata-rata antar kelas, terutama untuk kelas-kelas yang secara visual mirip (mis. Kunyit vs Singkong).

Kategori Fitur Bentuk (dua representasi berbeda diuji secara terpisah untuk mengisi *research gap* — penelitian terdahulu umumnya hanya menggunakan satu representasi bentuk saja):

- **Fourier Descriptor (FD)** — diperkenalkan oleh Zahn & Roskies (1972) dan telah diterapkan khusus pada pengenalan bentuk daun oleh studi-studi seperti Prayoga dkk. (2023) dan Linsangan dkk. (2023). FD merepresentasikan kontur daun sebagai sinyal kompleks (koordinat $x+iy$) yang ditransformasi ke domain frekuensi menggunakan *Fast Fourier Transform* (FFT). Sebanyak **40 koefisien frekuensi terendah** (selain komponen DC, yang dibuang karena hanya merepresentasikan translasi/posisi) dipertahankan sebagai deskriptor, dinormalisasi terhadap nilai magnitudo maksimum agar invariant terhadap skala/ukuran daun. FD bersifat global — menangkap bentuk keseluruhan daun (oval, lonjong, berlobus) dalam domain frekuensi.
- **Deskriptor Bentuk Geometris** — sembilan fitur geometris klasik yang diadaptasi dari Wu, Bao, Xu, Wang, Chang, & Xiang (2007) — penelitian rujukan utama dalam pengenalan daun berbasis fitur morfologi digital — yaitu: *area*, *perimeter*, *aspect ratio* (rasio lebar-tinggi *bounding box*), *extent* (rasio luas kontur terhadap luas *bounding box*), *solidity* (rasio luas kontur terhadap luas *convex hull*, sangat sensitif terhadap daun berlekuk/berlobus seperti Paku), *circularity* (kebulatan bentuk), *convexity* (rasio keliling *convex hull* terhadap keliling kontur asli), *eccentricity* (kelonjongan bentuk, dihitung melalui *regionprops* dari *scikit-image*), dan *rectangularity*. Berbeda dengan FD yang bersifat global dalam domain frekuensi, deskriptor geometris ini bersifat eksplisit dan dapat diinterpretasikan secara langsung secara visual.

Kedua representasi bentuk (FD dan deskriptor geometris) diuji secara terpisah pada Bagian 3.2/4.2 untuk membandingkan representasi bentuk mana yang melengkapi fitur tekstur dengan paling efektif — sebuah pertanyaan yang belum dijawab secara eksplisit oleh penelitian-penelitian pada Tabel SOTA (Bagian 1.1), yang masing-masing hanya menggunakan satu representasi bentuk.

Kode keempat fungsi ekstraksi fitur (`extract_glcm`, `extract_lbp`, `extract_fd`, `extract_shape`) serta fungsi `extract_features_scenario` yang menggombinasikannya sesuai skenario ditampilkan pada cell-cell berikut.

```
def extract_glcm(gray):
    glcm = graycomatrix(gray, distances=[1, 3, 5], angles=[0,
np.pi/2],
                        levels=256, symmetric=True, normed=True)
    contrast = graycoprops(glcm, 'contrast')
    correlation = graycoprops(glcm, 'correlation')
    features = list(contrast.flatten())
    features += list(correlation[:, 1])
    return np.array(features)
```

Ekstraksi Fitur Tekstur Menggunakan Gray Level Co-occurrence Matrix (GLCM)

Tahap ini bertujuan untuk mengekstraksi karakteristik tekstur daun menggunakan metode **Gray Level Co-occurrence Matrix (GLCM)** yang diperkenalkan oleh Haralick, Shanmugam, dan Dinstein (1973). GLCM merupakan metode statistik orde kedua (*second-order statistics*) yang menggambarkan hubungan kemunculan pasangan piksel dengan tingkat keabuan tertentu pada jarak dan orientasi tertentu. Berbeda dengan histogram intensitas yang hanya memperhatikan distribusi nilai piksel, GLCM mampu menangkap pola spasial antar piksel sehingga efektif untuk merepresentasikan tekstur permukaan daun.

```
def extract_glcm(gray):
```

Implementasi GLCM pada penelitian ini menggunakan fungsi `graycomatrix()` dari pustaka **scikit-image** dengan parameter sebagai berikut.

```
glcm = graycomatrix(
    gray,
    distances=[1,3,5],
    angles=[0,np.pi/2],
    levels=256,
    symmetric=True,
    normed=True
)
```

1. `distances=[1,3,5]`

Parameter `distance` menunjukkan jarak antar pasangan piksel yang digunakan dalam pembentukan matriks ko-okurensi.

Penelitian ini menggunakan tiga jarak berbeda, yaitu:

- **1 piksel** → menangkap tekstur sangat halus (*fine texture*)
- **3 piksel** → menangkap tekstur menengah

- **5 piksel** → menangkap tekstur yang lebih kasar (*coarse texture*)

Penggunaan beberapa jarak dilakukan agar karakteristik tekstur daun dapat diamati pada berbagai skala spasial. Menurut Haralick et al. (1973), informasi tekstur dapat berubah sesuai jarak antar piksel, sehingga penggunaan lebih dari satu jarak mampu memberikan representasi tekstur yang lebih lengkap dibanding hanya menggunakan satu nilai jarak.

2. `angles=[0,np.pi/2]`

GLCM dihitung pada dua orientasi, yaitu

- **0° (horizontal)**
- **90° (vertikal)**

Pemilihan kedua sudut tersebut dilakukan karena pola urat daun umumnya berkembang mengikuti arah horizontal maupun vertikal sehingga kedua orientasi tersebut telah mampu merepresentasikan pola tekstur utama pada permukaan daun.

Penggunaan dua orientasi juga mengurangi dimensi fitur dibanding menggunakan empat arah (0°, 45°, 90°, 135°), namun tetap mempertahankan informasi tekstur yang penting sehingga proses komputasi menjadi lebih efisien (Haralick et al., 1973).

3. `levels=256`

Nilai `levels=256` menunjukkan bahwa citra grayscale menggunakan **8-bit**, sehingga memiliki rentang intensitas **0–255**.

Karena citra pada penelitian ini tidak mengalami proses kuantisasi tambahan (*gray level reduction*), seluruh tingkat keabuan dipertahankan agar informasi tekstur tidak hilang.

4. `symmetric=True`

Parameter ini membuat matriks GLCM menjadi **simetris**, sehingga hubungan antar pasangan piksel (i, j) dianggap sama dengan (j, i) .

Pendekatan ini menghasilkan representasi tekstur yang lebih stabil serta mengurangi sensitivitas terhadap arah pembacaan pasangan piksel (Hall-Beyer, 2017).

5. `normed=True`

GLCM kemudian dinormalisasi sehingga seluruh elemen matriks merepresentasikan probabilitas kemunculan pasangan piksel.

Normalisasi diperlukan agar nilai fitur tidak dipengaruhi oleh jumlah pasangan piksel pada citra sehingga hasil ekstraksi dapat dibandingkan secara adil antar citra dengan ukuran yang sama maupun berbeda (Haralick et al., 1973).

Pemilihan Properti GLCM

Setelah matriks GLCM dibentuk, dilakukan ekstraksi dua properti tekstur.

```
contrast = graycoprops(glcml, 'contrast')
correlation = graycoprops(glcml, 'correlation')
```

Penelitian ini hanya menggunakan:

Contrast

Contrast mengukur tingkat variasi intensitas antar piksel.

Semakin besar nilai contrast, semakin besar perbedaan tingkat keabuan pada permukaan daun sehingga teksturnya semakin kasar.

Correlation

Correlation mengukur tingkat hubungan linear antar pasangan piksel yang berdekatan.

Fitur ini mampu menggambarkan keteraturan pola tekstur pada permukaan daun.

Alasan hanya menggunakan Contrast dan Correlation

Meskipun GLCM menyediakan beberapa properti lain seperti:

- Energy
- Homogeneity
- Dissimilarity
- ASM

penelitian ini hanya mempertahankan **Contrast** dan **Correlation**.

Pemilihan tersebut dilakukan berdasarkan analisis awal terhadap profil fitur menggunakan **heatmap rata-rata setiap kelas**, yang menunjukkan bahwa kedua fitur tersebut memiliki kemampuan diskriminasi yang lebih baik dalam membedakan lima jenis daun herbal. Sebaliknya, properti seperti **Energy**, **Homogeneity**, **ASM**, dan **Dissimilarity** menunjukkan pola yang relatif seragam antar kelas sehingga kontribusinya terhadap proses klasifikasi lebih kecil. Oleh karena itu, fitur-fitur tersebut tidak digunakan untuk mengurangi dimensi data sekaligus meminimalkan redundansi fitur.

Output

Nilai **Contrast** pada seluruh kombinasi jarak dan sudut kemudian digabungkan dengan nilai **Correlation** pada orientasi 90° sehingga menghasilkan sebuah vektor fitur tekstur yang akan digunakan sebagai masukan pada proses klasifikasi.

```
def extract_lbp(gray):
    selected = []
    lbp_r1 = local_binary_pattern(gray, 8, 1, method='uniform')
    hist_r1, _ = np.histogram(lbp_r1.ravel(), bins=np.arange(0, 11),
range=(0, 10))
    hist_r1 = hist_r1.astype(float) / (hist_r1.sum() + 1e-7)
    selected.append(hist_r1[6])

    lbp_r2 = local_binary_pattern(gray, 16, 2, method='uniform')
    hist_r2, _ = np.histogram(lbp_r2.ravel(), bins=np.arange(0, 19),
range=(0, 18))
    hist_r2 = hist_r2.astype(float) / (hist_r2.sum() + 1e-7)
    selected.extend([hist_r2[10], hist_r2[12], hist_r2[14]])
    return np.array(selected)
```

Selain menggunakan Gray Level Co-occurrence Matrix (GLCM), penelitian ini juga memanfaatkan **Local Binary Pattern (LBP)** untuk merepresentasikan tekstur lokal permukaan daun. LBP diperkenalkan oleh Ojala et al. (2002) sebagai operator tekstur yang membandingkan intensitas piksel pusat dengan piksel-piksel di sekitarnya sehingga menghasilkan pola biner yang mampu menggambarkan karakteristik tekstur secara lokal.

Keunggulan LBP adalah memiliki kompleksitas komputasi yang rendah, mampu menangkap pola tekstur mikro, serta relatif tahan terhadap perubahan intensitas pencahayaan (*illumination invariant*). Karakteristik tersebut menjadikan LBP banyak digunakan pada berbagai penelitian klasifikasi citra berbasis tekstur.

```
def extract_lbp(gray):
```

Implementasi LBP pada penelitian ini menggunakan fungsi `local_binary_pattern()` dari pustaka **scikit-image**.

Radius 1

```
lbp_r1 = local_binary_pattern(
    gray,
    8,
    1,
    method='uniform'
)
```

Jumlah Tetangga = 8

Nilai **P = 8** menunjukkan bahwa setiap piksel dibandingkan dengan **8 piksel tetangga** yang berada pada lingkaran berjari-jari 1 piksel.

Konfigurasi ini merupakan konfigurasi standar yang diperkenalkan oleh Ojala et al. (2002) karena mampu merepresentasikan tekstur lokal dengan baik tanpa meningkatkan kompleksitas komputasi.

Radius = 1

Radius **R = 1** digunakan untuk menangkap pola tekstur yang sangat halus (*fine texture*) pada permukaan daun, seperti urat kecil maupun variasi permukaan daun.

Method = "uniform"

Penelitian ini menggunakan metode **Uniform LBP**.

Uniform LBP hanya mempertahankan pola biner yang memiliki maksimum dua kali perubahan bit (bitwise transition). Menurut Ojala et al. (2002), sebagian besar pola tekstur alami termasuk ke dalam kategori uniform sehingga metode ini mampu mengurangi jumlah pola secara signifikan tanpa kehilangan informasi tekstur yang penting.

Dibandingkan LBP biasa, Uniform LBP menghasilkan dimensi fitur yang lebih rendah sehingga lebih efisien untuk proses klasifikasi.

Radius 2

```
lbp_r2 = local_binary_pattern(  
    gray,  
    16,  
    2,  
    method='uniform'  
)
```

Jumlah Tetangga = 16

Pada radius kedua digunakan **16 piksel tetangga**.

Penambahan jumlah tetangga bertujuan agar pola tekstur dapat diamati pada area yang lebih luas sehingga informasi tekstur dengan skala menengah (*medium texture*) juga dapat direpresentasikan.

Radius = 2

Radius **2 piksel** dipilih agar fitur LBP tidak hanya menangkap tekstur lokal yang sangat kecil, tetapi juga pola permukaan daun yang lebih besar.

Penggunaan dua radius (R=1 dan R=2) memungkinkan representasi tekstur dilakukan pada dua skala berbeda (*multi-scale texture analysis*), yang diketahui mampu meningkatkan kemampuan diskriminasi fitur dibandingkan hanya menggunakan satu radius (Ojala et al., 2002).

Histogram LBP

Setelah citra LBP diperoleh, dilakukan pembentukan histogram frekuensi kemunculan setiap pola menggunakan

```
np.histogram(...)
```

Histogram kemudian dinormalisasi menggunakan

```
hist = hist.astype(float) / (hist.sum() + 1e-7)
```

Normalisasi dilakukan agar nilai histogram berubah menjadi distribusi probabilitas sehingga tidak dipengaruhi oleh jumlah piksel pada citra. Penambahan konstanta kecil **1e-7** bertujuan menghindari pembagian dengan nol (*division by zero*) apabila histogram memiliki jumlah yang sangat kecil.

Pemilihan Bin Histogram

Berbeda dengan implementasi LBP pada umumnya yang menggunakan seluruh histogram sebagai fitur, penelitian ini hanya menggunakan beberapa bin histogram, yaitu:

Radius 1

```
hist_r1[6]
```

Radius 2

```
hist_r2[10]
```

```
hist_r2[12]
```

```
hist_r2[14]
```

Pemilihan bin tersebut **tidak dilakukan secara acak**.

Berdasarkan analisis **heatmap rata-rata nilai histogram setiap kelas**, bin ke-6 pada radius 1 serta bin ke-10, ke-12, dan ke-14 pada radius 2 menunjukkan variasi yang paling jelas antar lima

kelas daun herbal. Sebaliknya, sebagian besar bin lainnya memiliki distribusi yang relatif seragam sehingga kontribusinya terhadap proses klasifikasi lebih kecil.

Dengan hanya mempertahankan bin yang paling informatif, dimensi fitur dapat dikurangi tanpa menghilangkan informasi penting, sehingga proses pelatihan model menjadi lebih efisien dan potensi redundansi fitur dapat diminimalkan.

Output

Fitur yang digunakan pada penelitian ini terdiri atas empat nilai histogram terpilih, yaitu:

- Radius 1 → Bin ke-6
- Radius 2 → Bin ke-10
- Radius 2 → Bin ke-12
- Radius 2 → Bin ke-14

Keempat nilai tersebut kemudian digabungkan menjadi satu vektor fitur tekstur yang selanjutnya digunakan sebagai masukan pada proses klasifikasi menggunakan SVM, KNN, dan Naive Bayes.

```
def extract_fd(contour, num_descriptors=40):  
    if contour is None: return np.zeros(num_descriptors)  
    contour = contour[:, 0, :]  
    complex_contour = contour[:, 0] + 1j * contour[:, 1]  
    fd = np.abs(np.fft.fft(complex_contour))  
    fd = fd[1:(num_descriptors + 1)]  
    if len(fd) > 0: fd = fd / (np.max(fd) + 1e-10)  
    if len(fd) < num_descriptors: fd = np.pad(fd, (0, num_descriptors  
- len(fd)))  
    return fd
```

Selain tekstur, penelitian ini juga memanfaatkan **Fourier Descriptor (FD)** untuk merepresentasikan karakteristik bentuk daun. Fourier Descriptor merupakan metode ekstraksi fitur yang mengubah koordinat kontur objek dari domain spasial ke domain frekuensi menggunakan **Fast Fourier Transform (FFT)**. Pendekatan ini memungkinkan bentuk objek direpresentasikan dalam bentuk koefisien frekuensi yang lebih ringkas serta lebih tahan terhadap variasi translasi, rotasi, dan skala (Zahn & Roskies, 1972).

```
def extract_fd(contour, num_descriptors=40):
```

Input fungsi ini berupa **kontur terluar daun** yang diperoleh dari proses *Contour Detection*.

Penanganan Kontur Kosong

```
if contour is None:  
    return np.zeros(num_descriptors)
```

Apabila proses segmentasi tidak menghasilkan kontur, fungsi mengembalikan vektor nol sebanyak **40 elemen**.

Pendekatan ini bertujuan agar jumlah fitur yang dihasilkan tetap konsisten pada setiap sampel sehingga proses pelatihan model tidak mengalami kesalahan akibat dimensi fitur yang berbeda.

Konversi Kontur ke Bilangan Kompleks

```
contour = contour[:,0,:]
complex_contour = contour[:,0] + 1j * contour[:,1]
```

Koordinat kontur terdiri atas pasangan titik

$[(x,y)]$

yang kemudian direpresentasikan sebagai bilangan kompleks

$[z=x+jy]$

dengan:

- bagian real mewakili koordinat **x**
- bagian imajiner mewakili koordinat **y**

Representasi kompleks ini merupakan bentuk standar pada Fourier Descriptor karena memudahkan transformasi Fourier terhadap bentuk kontur (Zahn & Roskies, 1972).

Fast Fourier Transform (FFT)

```
fd = np.abs(np.fft.fft(complex_contour))
```

Setelah kontur direpresentasikan sebagai bilangan kompleks, dilakukan **Fast Fourier Transform (FFT)** untuk memperoleh koefisien Fourier.

FFT dipilih karena merupakan algoritma yang jauh lebih efisien dibandingkan Discrete Fourier Transform (DFT), dengan kompleksitas

$[O(n \log n)]$

dibandingkan

$[O(n^2)]$

(Cooley & Tukey, 1965).

Pada penelitian ini hanya digunakan **magnitudo koefisien Fourier** (`np.abs`) sehingga representasi bentuk tidak dipengaruhi oleh perubahan fase.

Menghilangkan Koefisien Pertama

```
fd = fd[1:(num_descriptors+1)]
```

Koefisien pertama (DC Component) tidak digunakan.

Koefisien tersebut hanya merepresentasikan posisi pusat objek (*centroid*) dan tidak mengandung informasi mengenai bentuk daun.

Menghilangkan komponen DC membuat Fourier Descriptor menjadi **invariant terhadap translasi**, sehingga perpindahan posisi daun pada citra tidak memengaruhi fitur yang dihasilkan (Persoon & Fu, 1977).

Jumlah Descriptor = 40

```
num_descriptors = 40
```

Penelitian ini menggunakan **40 koefisien Fourier pertama**.

Pemilihan jumlah descriptor didasarkan pada karakteristik Fourier Descriptor, yaitu semakin rendah frekuensi maka semakin besar kontribusinya terhadap bentuk global objek.

Empat puluh koefisien pertama dipilih karena telah mampu merepresentasikan bentuk utama daun tanpa memasukkan komponen frekuensi tinggi yang umumnya berasal dari noise maupun ketidakteraturan kontur.

Selain itu, pembatasan jumlah descriptor juga bertujuan mengurangi dimensi fitur sehingga proses pelatihan model menjadi lebih efisien.

Pendekatan penggunaan sejumlah kecil koefisien Fourier juga telah banyak digunakan pada penelitian klasifikasi bentuk objek karena mampu menjaga keseimbangan antara akurasi representasi bentuk dan kompleksitas komputasi (Zahn & Roskies, 1972; Persoon & Fu, 1977).

Normalisasi

```
fd = fd / (np.max(fd)+1e-10)
```

Koefisien Fourier kemudian dinormalisasi menggunakan nilai maksimum.

Normalisasi dilakukan agar seluruh descriptor memiliki rentang nilai yang seragam sehingga tidak dipengaruhi oleh ukuran objek.

Dengan demikian, daun yang memiliki bentuk sama tetapi ukuran berbeda akan menghasilkan representasi fitur yang relatif konsisten.

Penambahan konstanta

```
1e-10
```


bertujuan mencegah pembagian dengan nol (*division by zero*).

Padding

```
np.pad(...)
```

Jumlah titik kontur setiap daun tidak selalu sama.

Oleh karena itu dilakukan **padding** apabila jumlah descriptor kurang dari 40.

Padding memastikan seluruh sampel memiliki dimensi fitur yang identik sehingga dapat digunakan sebagai masukan pada algoritma klasifikasi seperti SVM, KNN, dan Gaussian Naive Bayes.

Output

Output fungsi ini berupa **40 nilai Fourier Descriptor** yang merepresentasikan karakteristik bentuk global daun.

Vektor fitur tersebut kemudian dapat dikombinasikan dengan fitur tekstur (GLCM dan LBP) maupun fitur geometris (Shape Descriptor) dalam proses **Hybrid Feature Extraction**.

```
def extract_shape(contour):  
    if contour is None:  
        return np.zeros(9)  
  
    area = cv2.contourArea(contour)  
  
    perimeter = cv2.arcLength(  
        contour,  
        True  
    )  
  
    x, y, w, h = cv2.boundingRect(contour)  
  
    aspect_ratio = w / (h + 1e-10)  
  
    rect_area = w * h  
  
    extent = area / (rect_area + 1e-10)  
  
    hull = cv2.convexHull(contour)  
  
    hull_area = cv2.contourArea(hull)  
  
    hull_perimeter = cv2.arcLength(  
        hull,
```

```

        True
    )

    solidity = area / (hull_area + 1e-10)

    circularity = (
        4 * np.pi * area
    ) / ((perimeter ** 2) + 1e-10)

    convexity = hull_perimeter / (perimeter + 1e-10)

    mask = np.zeros(
        (IMG_SIZE[1], IMG_SIZE[0]),
        dtype=np.uint8
    )

    cv2.drawContours(
        mask,
        [contour],
        -1,
        255,
        -1
    )

    props = regionprops(mask)

    if len(props) > 0:
        eccentricity = props[0].eccentricity
    else:
        eccentricity = 0

    rectangularity = area / (rect_area + 1e-10)

    return np.array([
        area,
        perimeter,
        aspect_ratio,
        extent,
        solidity,
        circularity,
        convexity,
        eccentricity,
        rectangularity
    ])

```

Ekstraksi Fitur Bentuk Menggunakan Shape Descriptor

Selain Fourier Descriptor, penelitian ini juga menggunakan **Shape Descriptor** untuk merepresentasikan karakteristik geometris daun. Berbeda dengan Fourier Descriptor yang merepresentasikan bentuk melalui transformasi frekuensi, Shape Descriptor menghitung ukuran-ukuran geometris secara langsung dari kontur objek sehingga mampu menggambarkan bentuk daun secara kuantitatif.

Pendekatan ini dipilih karena beberapa jenis daun memiliki tekstur yang mirip, namun masih dapat dibedakan berdasarkan karakteristik bentuknya, seperti luas, tingkat kebulatan, maupun tingkat kecembungan daun. Dengan mengombinasikan Shape Descriptor dan Fourier Descriptor, representasi bentuk menjadi lebih lengkap.

```
def extract_shape(contour):
```

Apabila tidak ditemukan kontur, fungsi akan mengembalikan vektor nol sebanyak **9 fitur** agar dimensi fitur setiap sampel tetap konsisten.

1. Area

```
area = cv2.contourArea(contour)
```

Area merupakan luas objek daun yang dihitung berdasarkan jumlah piksel di dalam kontur.

Fitur ini digunakan karena ukuran permukaan daun berbeda pada setiap jenis tanaman sehingga dapat menjadi karakteristik pembeda.

2. Perimeter

```
perimeter = cv2.arcLength(contour, True)
```

Perimeter merupakan panjang keliling objek daun.

Fitur ini menggambarkan kompleksitas bentuk tepi daun. Daun dengan tepian bergerigi atau berlekuk umumnya memiliki perimeter yang lebih besar dibandingkan daun dengan bentuk sederhana.

3. Aspect Ratio

```
aspect_ratio = w / (h + 1e-10)
```

Aspect Ratio merupakan perbandingan antara lebar dan tinggi dari **bounding box** objek.

Fitur ini menunjukkan kecenderungan bentuk daun, apakah lebih memanjang atau lebih melebar.

Nilai ini relatif stabil terhadap perubahan ukuran sehingga sering digunakan sebagai fitur dasar pada klasifikasi bentuk objek (Gonzalez & Woods, 2018).

4. Extent

```
extent = area / (rect_area + 1e-10)
```

Extent merupakan rasio antara luas objek dengan luas **bounding box**.

Fitur ini menunjukkan seberapa besar objek memenuhi kotak pembatasnya.

Semakin tinggi nilai extent maka bentuk objek semakin memenuhi area persegi pembatas.

5. Solidity

```
solidity = area / (hull_area + 1e-10)
```

Solidity merupakan perbandingan antara luas objek dengan luas **convex hull**.

Fitur ini digunakan untuk mengukur tingkat kekonkavan bentuk daun.

Daun yang memiliki banyak lekukan akan menghasilkan nilai solidity yang lebih kecil dibandingkan daun dengan bentuk yang lebih utuh.

6. Circularity

```
circularity =  
(4π × area) / perimeter2
```

Circularity mengukur tingkat kemiripan bentuk objek terhadap lingkaran sempurna.

Nilai circularity mendekati 1 menunjukkan bentuk semakin bulat, sedangkan nilai yang lebih kecil menunjukkan bentuk semakin memanjang atau tidak beraturan.

7. Convexity

```
convexity =  
hull_perimeter / perimeter
```

Convexity menggambarkan tingkat kecembungan bentuk objek.

Fitur ini membandingkan panjang convex hull dengan panjang kontur asli sehingga mampu mendeteksi adanya lekukan pada tepi daun.

8. Eccentricity

```
props = regionprops(mask)
eccentricity = props[0].eccentricity
```

Eccentricity dihitung menggunakan fungsi `regionprops()` dari pustaka **scikit-image**.

Nilai ini menunjukkan tingkat kememanjangan objek berdasarkan elips yang memiliki momen inersia sama dengan objek.

- Nilai mendekati **0** menunjukkan bentuk hampir bulat.
- Nilai mendekati **1** menunjukkan bentuk semakin memanjang.

Fitur ini banyak digunakan pada klasifikasi objek biologis karena mampu membedakan bentuk daun yang lonjong dan membulat.

9. Rectangularity

```
rectangularity =
area / rect_area
```

Rectangularity menunjukkan tingkat kesesuaian objek terhadap bentuk persegi panjang pembatasnya.

Semakin besar nilainya maka bentuk objek semakin menyerupai persegi panjang.

Pada penelitian ini rectangularity dihitung menggunakan rasio luas objek terhadap luas bounding box sehingga memberikan informasi mengenai efisiensi pengisian area objek.

Penambahan Konstanta `1e-10`

Pada beberapa proses pembagian digunakan konstanta kecil

```
1e-10
```

Konstanta ini ditambahkan untuk menghindari terjadinya **division by zero**, misalnya apabila perimeter, tinggi, atau luas convex hull bernilai nol akibat proses segmentasi yang tidak sempurna.

Pendekatan ini merupakan praktik yang umum digunakan dalam komputasi numerik untuk menjaga stabilitas perhitungan.

Output

Fungsi ini menghasilkan **9 fitur geometris**, yaitu:

1. Area
2. Perimeter
3. Aspect Ratio
4. Extent
5. Solidity
6. Circularity
7. Convexity
8. Eccentricity
9. Rectangularity

Kesembilan fitur tersebut kemudian digabungkan dengan fitur tekstur (GLCM dan LBP) serta Fourier Descriptor untuk membentuk **Hybrid Feature Extraction** yang digunakan sebagai masukan pada algoritma SVM, KNN, dan Gaussian Naive Bayes.

```
def extract_features_scenario(gray, contour, scenario):  
  
    glcm = extract_glcm(gray)  
    lbp = extract_lbp(gray)  
    fd = extract_fd(contour)  
    shape = extract_shape(contour)  
  
    if scenario == 1:  
        return glcm  
  
    elif scenario == 2:  
        return lbp  
  
    elif scenario == 3:  
        return fd  
  
    elif scenario == 4:  
        return np.hstack([  
            glcm,  
            fd  
        ])  
  
    elif scenario == 5:  
        return np.hstack([  
            lbp,  
            fd  
        ])  
  
    elif scenario == 6:
```

```

        return np.hstack([
            glcm,
            lbp,
            fd
        ])

    elif scenario == 7:
        return shape

    elif scenario == 8:
        return np.hstack([
            glcm,
            shape
        ])

    elif scenario == 9:
        return np.hstack([
            lbp,
            shape
        ])

    elif scenario == 10:
        return np.hstack([
            glcm,
            lbp,
            shape
        ])

```

Perancangan Skenario Hybrid Feature Extraction

Tahap ini bertujuan untuk membangun beberapa kombinasi fitur (*feature scenarios*) yang akan digunakan sebagai masukan pada proses klasifikasi. Pendekatan ini dilakukan untuk menganalisis kontribusi masing-masing fitur, baik secara individu maupun kombinasi (*hybrid feature extraction*), terhadap performa klasifikasi daun herbal.

```
def extract_features_scenario(gray, contour, scenario):
```

Pada fungsi ini, terlebih dahulu dilakukan ekstraksi seluruh fitur yang telah dikembangkan pada penelitian, yaitu:

- **GLCM** → merepresentasikan tekstur global daun.
- **LBP** → merepresentasikan tekstur lokal daun.
- **Fourier Descriptor (FD)** → merepresentasikan bentuk global berdasarkan kontur.
- **Shape Descriptor** → merepresentasikan karakteristik geometris daun.

```

glcm = extract_glcm(gray)
lbp = extract_lbp(gray)
fd = extract_fd(contour)
shape = extract_shape(contour)

```

Setelah seluruh fitur diperoleh, dilakukan pembentukan **10 skenario pengujian**.

Alasan Penyusunan Skenario

Penelitian ini tidak hanya bertujuan memperoleh model dengan akurasi tertinggi, tetapi juga mengevaluasi pengaruh setiap jenis fitur terhadap performa klasifikasi.

Oleh karena itu, skenario disusun menjadi tiga kelompok utama:

1. **Fitur Tunggal (Single Feature)**

Digunakan untuk mengetahui kemampuan masing-masing metode ekstraksi fitur apabila digunakan secara individual.

2. **Hybrid Tekstur–Bentuk Menggunakan Fourier Descriptor**

Digunakan untuk mengetahui apakah kombinasi tekstur dan representasi bentuk global mampu meningkatkan performa dibandingkan fitur tunggal.

3. **Hybrid Tekstur–Bentuk Menggunakan Shape Descriptor**

Digunakan untuk mengevaluasi kontribusi fitur geometris apabila dikombinasikan dengan fitur tekstur.

Pendekatan **Hybrid Feature Extraction** banyak digunakan pada penelitian klasifikasi citra karena kombinasi beberapa jenis fitur mampu menghasilkan representasi objek yang lebih lengkap dibandingkan penggunaan satu fitur saja (Bishop, 2006; Gonzalez & Woods, 2018).

Skenario Pengujian

Skenario	Kombinasi Fitur	Tujuan
1	GLCM	Mengukur performa tekstur global.
2	LBP	Mengukur performa tekstur lokal.
3	Fourier Descriptor	Mengukur performa bentuk global.
4	GLCM + Fourier Descriptor	Menguji kombinasi tekstur global dan bentuk global.
5	LBP + Fourier Descriptor	Menguji kombinasi tekstur lokal dan

Skenario	Kombinasi Fitur	Tujuan
6	GLCM + LBP + Fourier Descriptor	bentuk global. Menguji kombinasi seluruh fitur tekstur dengan bentuk global.
7	Shape Descriptor	Mengukur performa fitur geometris.
8	GLCM + Shape Descriptor	Menguji kombinasi tekstur global dan bentuk geometris.
9	LBP + Shape Descriptor	Menguji kombinasi tekstur lokal dan bentuk geometris.
10	GLCM + LBP + Shape Descriptor	Menguji kombinasi seluruh fitur tekstur dengan bentuk geometris.

Mengapa Fourier Descriptor dan Shape Descriptor Tidak Digabungkan?

Pada penelitian ini **Fourier Descriptor** dan **Shape Descriptor** tidak digabungkan dalam satu skenario.

Hal ini dilakukan karena kedua metode sama-sama merepresentasikan **karakteristik bentuk daun**, namun menggunakan pendekatan yang berbeda.

- **Fourier Descriptor** menggambarkan bentuk berdasarkan representasi frekuensi dari kontur.
- **Shape Descriptor** menggambarkan bentuk menggunakan karakteristik geometris seperti area, perimeter, circularity, dan solidity.

Penelitian ini bertujuan membandingkan efektivitas kedua representasi bentuk tersebut ketika dikombinasikan dengan fitur tekstur. Oleh karena itu, kedua fitur bentuk diuji secara terpisah agar kontribusinya dapat dianalisis dengan lebih jelas.

Hybrid Feature Extraction

Kombinasi fitur dilakukan menggunakan fungsi

```
np.hstack()
```

Fungsi ini menggabungkan beberapa vektor fitur menjadi satu vektor fitur baru secara horizontal.

Sebagai contoh,

```
np.hstack([
    glcm,
    lbp,
    fd
])
```

akan menghasilkan satu vektor fitur yang terdiri atas:

- fitur tekstur global (GLCM),
- fitur tekstur lokal (LBP),
- fitur bentuk global (Fourier Descriptor).

Pendekatan ini memungkinkan algoritma klasifikasi memperoleh informasi tekstur dan bentuk secara bersamaan sehingga diharapkan mampu meningkatkan kemampuan diskriminasi antar kelas daun herbal.

Tujuan Evaluasi

Melalui sepuluh skenario tersebut, penelitian ini bertujuan untuk menjawab beberapa pertanyaan berikut.

- Seberapa baik performa setiap fitur apabila digunakan secara individual?
- Apakah kombinasi fitur tekstur dan bentuk mampu meningkatkan akurasi klasifikasi?
- Representasi bentuk mana yang memberikan hasil lebih baik, yaitu Fourier Descriptor atau Shape Descriptor?
- Kombinasi fitur mana yang menghasilkan performa terbaik ketika diklasifikasikan menggunakan SVM, KNN, dan Gaussian Naive Bayes?

Hasil dari seluruh skenario kemudian dibandingkan menggunakan metrik **Accuracy**, **Precision**, **Recall**, **F1-Score**, serta **Confusion Matrix** untuk menentukan kombinasi fitur terbaik.

3.2 Arsitektur Model Pengenalan Pola

Tiga algoritma klasifikasi konvensional (bukan *deep learning*) dipilih sesuai arah kontribusi proyek ini, yaitu mengombinasikan *hybrid feature extraction* dengan algoritma *machine learning* yang ringan secara komputasi:

- **Support Vector Machine (SVM)** dengan **kernel linear** dan parameter regulasi **C = 1**. Pemilihan kernel linear (bukan RBF/non-linear) mengacu pada panduan praktis Hsu, Chang, & Lin (2003): kernel RBF unggul ketika hubungan antara fitur dan label bersifat non-linear dan kompleks, namun memerlukan tuning dua hiperparameter (*C* dan γ) yang berisiko *overfitting* pada dataset berukuran kecil hingga menengah. Karena fitur yang digunakan pada proyek ini sudah melalui seleksi berbasis analisis *heatmap* (Bagian 3.1)

yang menunjukkan separasi kelas yang cukup jelas pada level fitur mentah, kernel linear yang lebih sederhana dan lebih sedikit hiperparameter menjadi pilihan yang lebih *robust* dan mengurangi risiko *overfitting*. Parameter `probability=True` diaktifkan agar SVM dapat menghasilkan estimasi probabilitas (`predict_proba`) untuk keperluan prediksi citra baru pada Lampiran.

- **K-Nearest Neighbor (KNN)** dengan **k = 5**. Nilai k=5 merupakan nilai *default* yang umum digunakan sebagai titik awal yang wajar dalam berbagai studi klasifikasi citra daun (termasuk Furqan dkk., 2020 dan Prayoga dkk., 2023 pada Tabel SOTA) — nilai k yang terlalu kecil (mis. k=1) rentan terhadap *noise/outlier*, sedangkan k yang terlalu besar berisiko mengaburkan batas antar kelas yang berdekatan.
- **Naive Bayes (GaussianNB)**, yang mengasumsikan distribusi Gaussian untuk setiap fitur per kelas dan independensi antar fitur. Model ini digunakan sebagai *baseline* probabilistik yang sangat ringan secara komputasi, sesuai pendekatan Azzahra dkk. (2023) yang juga menyertakan *Naive Bayes* sebagai pembanding terhadap CNN.

Catatan mengenai hiperparameter: Karena ketiga algoritma di atas merupakan metode *machine learning* klasik (bukan *deep learning* berbasis gradient descent), konsep *Learning Rate*, *Optimizer*, *Loss Function*, *Epoch*, dan *Batch Size* sebagaimana disebutkan pada template laporan **tidak relevan** untuk proyek ini — parameter yang relevan dan telah dijelaskan di atas adalah jenis kernel dan C (SVM), jumlah tetangga k (KNN), serta asumsi distribusi (*Naive Bayes*). Seluruh fitur sebelum dilatih distandardisasi menggunakan `StandardScaler` (zero mean, unit variance) agar fitur dengan rentang nilai besar (mis. `area` pada deskriptor bentuk geometris) tidak mendominasi fitur dengan rentang nilai kecil (mis. histogram LBP yang ternormalisasi 0–1) pada perhitungan jarak (KNN) maupun *margin* (SVM).

Implementasi model (`get_models`) dan fungsi pelatihan-evaluasi yang dipakai berulang untuk seluruh skenario (`train_evaluate_save`) — dirancang sebagai satu fungsi *reusable* agar tidak ada duplikasi kode pelatihan SVM/KNN/*Naive Bayes* di setiap skenario — ditampilkan pada cell-cell berikut.

```
SHORT_NAMES = {"SVM": "svm", "KNN": "knn", "Naive Bayes": "nb"}
```

```
def get_models():  
    return {  
        "SVM": SVC(kernel='linear', C=1, probability=True,  
random_state=42),  
        "KNN": KNeighborsClassifier(n_neighbors=5),  
        "Naive Bayes": GaussianNB()  
    }
```

Tahap berikutnya adalah mendefinisikan algoritma klasifikasi yang akan digunakan pada penelitian. Penelitian ini membandingkan tiga algoritma *supervised machine learning*, yaitu **Support Vector Machine (SVM)**, **K-Nearest Neighbor (KNN)**, dan **Gaussian Naive Bayes (GNB)**.

Ketiga algoritma dipilih karena memiliki karakteristik yang berbeda sehingga dapat memberikan gambaran mengenai performa berbagai pendekatan klasifikasi terhadap fitur hasil ekstraksi tekstur dan bentuk.

```
SHORT_NAMES = {  
    "SVM": "svm",  
    "KNN": "knn",  
    "Naive Bayes": "nb"  
}
```

Dictionary `SHORT_NAMES` digunakan sebagai singkatan nama model ketika menyimpan hasil evaluasi, model terlatih, maupun visualisasi sehingga penamaan file menjadi lebih ringkas dan konsisten.

Seluruh model kemudian diinisialisasi melalui fungsi berikut.

```
def get_models():
```

1. Support Vector Machine (SVM)

```
SVC(  
    kernel='linear',  
    C=1,  
    probability=True,  
    random_state=42  
)
```

Kernel = "linear"

Penelitian ini menggunakan **Linear Kernel** karena fitur yang digunakan merupakan hasil **Hybrid Feature Extraction** (GLCM, LBP, Fourier Descriptor, dan Shape Descriptor) yang memiliki dimensi relatif tinggi.

Linear SVM dipilih karena:

- lebih sederhana dibanding kernel non-linear,
- memiliki waktu pelatihan yang lebih cepat,
- mampu bekerja dengan baik pada data berdimensi tinggi,
- mengurangi risiko overfitting dibanding kernel RBF pada dataset yang relatif kecil.

Menurut Cortes dan Vapnik (1995), apabila data dapat dipisahkan secara cukup baik pada ruang fitur, penggunaan **Linear Kernel** sering kali menghasilkan performa yang kompetitif dengan kompleksitas komputasi yang lebih rendah.

C = 1

Parameter **C** merupakan parameter regularisasi yang mengontrol keseimbangan antara memperoleh margin yang lebar dan meminimalkan kesalahan klasifikasi.

Penelitian ini menggunakan

```
C = 1
```

karena merupakan **nilai default yang direkomendasikan pada scikit-learn** dan banyak digunakan sebagai titik awal (*baseline*) dalam penelitian klasifikasi.

Nilai ini memberikan keseimbangan yang baik antara kemampuan generalisasi model dan toleransi terhadap kesalahan pada data latih (Pedregosa et al., 2011).

```
probability = True
```

```
probability=True
```

Parameter ini mengaktifkan estimasi probabilitas kelas menggunakan **Platt Scaling**.

Walaupun penelitian ini menggunakan **akurasi, precision, recall, dan F1-score** sebagai metrik utama, probabilitas kelas tetap diaktifkan agar model dapat digunakan untuk analisis lanjutan maupun pengembangan sistem di masa mendatang.

```
random_state = 42
```

```
random_state=42
```

Digunakan untuk memastikan proses pelatihan model bersifat **reproducible**, sehingga hasil eksperimen tetap konsisten ketika kode dijalankan kembali.

2. K-Nearest Neighbor (KNN)

```
KNeighborsClassifier(  
    n_neighbors=5  
)
```

```
n_neighbors = 5
```

Parameter **k = 5** menunjukkan bahwa proses klasifikasi dilakukan berdasarkan **lima tetangga terdekat**.

Nilai ini dipilih karena merupakan salah satu nilai yang paling umum digunakan pada algoritma KNN dan mampu memberikan keseimbangan antara sensitivitas terhadap noise dan kemampuan generalisasi model.

Jika nilai **k terlalu kecil**, model menjadi sangat sensitif terhadap noise (*overfitting*).

Sebaliknya, jika nilai **k terlalu besar**, batas keputusan menjadi terlalu halus sehingga dapat menyebabkan *underfitting*.

Han, Kamber, dan Pei (2012) menyatakan bahwa nilai **k** pada rentang kecil seperti **3–7** umumnya memberikan performa yang baik pada berbagai kasus klasifikasi.

3. Gaussian Naive Bayes

GaussianNB()

Penelitian ini menggunakan **Gaussian Naive Bayes** karena seluruh fitur hasil ekstraksi berupa **data numerik kontinu**.

Gaussian Naive Bayes mengasumsikan bahwa distribusi setiap fitur mengikuti distribusi Gaussian (Normal Distribution).

Pendekatan ini sesuai untuk data hasil ekstraksi fitur citra seperti:

- Contrast
- Correlation
- Histogram LBP
- Fourier Descriptor
- Shape Descriptor

Selain memiliki kompleksitas komputasi yang rendah, Gaussian Naive Bayes juga sering dijadikan **baseline classifier** pada penelitian pengenalan pola karena proses pelatihannya sangat cepat dan tidak memerlukan proses optimasi parameter yang kompleks (Bishop, 2006).

Ringkasan Parameter Model

Algoritma	Parameter	Nilai	Alasan Pemilihan
SVM	Kernel	Linear	Cocok untuk fitur berdimensi tinggi dan lebih efisien dibanding kernel non-linear.
	C	1	Nilai default yang memberikan keseimbangan antara margin dan kesalahan klasifikasi.
	Probability	True	Menghasilkan probabilitas prediksi untuk analisis lanjutan.
	Random State	42	Menjamin reproduksibilitas eksperimen.
KNN	k	5	Nilai umum yang memberikan keseimbangan antara overfitting dan underfitting.
Gaussian NB	Distribusi	Gaussian	Sesuai untuk fitur numerik kontinu hasil ekstraksi citra.

```
def train_scenario(X_train, y_train, X_test, y_test, scenario_num,
encoder):

    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    models = get_models()
    for model in models.values():
        model.fit(X_train_scaled, y_train)

    for name, model in models.items():
        joblib.dump(model, f"{SHORT_NAMES[name]}_s{scenario_num}.pkl")
    joblib.dump(scaler, f"scaler_s{scenario_num}.pkl")
    joblib.dump(encoder, "label_encoder.pkl")

    return models, scaler, X_train_scaled, X_test_scaled
```

Pelatihan Model Klasifikasi

Tahap ini bertujuan untuk melakukan pelatihan model klasifikasi pada setiap skenario pengujian. Sebelum proses pelatihan dilakukan, fitur terlebih dahulu dinormalisasi menggunakan **StandardScaler**, kemudian digunakan untuk melatih tiga algoritma klasifikasi, yaitu **Support Vector Machine (SVM)**, **K-Nearest Neighbor (KNN)**, dan **Gaussian Naive Bayes (GNB)**.

```
def train_scenario(
    X_train,
    y_train,
    X_test,
    y_test,
    scenario_num,
    encoder
):
```

1. Standardisasi Fitur

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)
```

Tahap pertama adalah melakukan **standardisasi fitur** menggunakan **StandardScaler** dari pustaka *scikit-learn*.

Standardisasi mengubah setiap fitur sehingga memiliki:

- **rata-rata (mean) = 0**
- **simpangan baku (standard deviation) = 1**

Transformasi dilakukan menggunakan persamaan

$$[z = \frac{x - \mu}{\sigma}]$$

dengan:

- (x) = nilai fitur,
- (μ) = rata-rata fitur,
- (σ) = simpangan baku fitur.

Standardisasi diperlukan karena penelitian ini menggabungkan beberapa jenis fitur dengan rentang nilai yang berbeda, misalnya:

- GLCM
- Histogram LBP
- Fourier Descriptor
- Shape Descriptor

Apabila tidak dilakukan standardisasi, fitur dengan rentang nilai yang lebih besar (misalnya **Area** atau **Perimeter**) dapat mendominasi proses pelatihan dan mengurangi kontribusi fitur lain yang memiliki rentang nilai lebih kecil.

Mengapa hanya `fit_transform()` pada data latih?

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

Parameter statistik (mean dan standar deviasi) dihitung **hanya dari data latih** menggunakan `fit_transform()`.

Selanjutnya parameter tersebut digunakan kembali untuk mentransformasi data uji melalui fungsi `transform()`.

Pendekatan ini merupakan praktik standar dalam *machine learning* karena mencegah terjadinya **data leakage**, yaitu kondisi ketika informasi dari data uji ikut digunakan selama proses pelatihan model sehingga dapat menghasilkan estimasi performa yang terlalu optimistis (Kuhn & Johnson, 2013).

2. Inisialisasi Model

```
models = get_models()
```


Fungsi `get_models()` digunakan untuk menginisialisasi tiga algoritma klasifikasi yang digunakan pada penelitian, yaitu:

- Support Vector Machine (SVM)
- K-Nearest Neighbor (KNN)
- Gaussian Naive Bayes (GNB)

Dengan menggunakan fungsi ini, konfigurasi seluruh model dapat dikelola secara terpusat sehingga memudahkan proses eksperimen pada berbagai skenario fitur.

3. Pelatihan Model

```
for model in models.values():
    model.fit(
        X_train_scaled,
        y_train
    )
```

Setiap algoritma dilatih menggunakan data latih yang telah melalui proses standardisasi.

Fungsi `fit()` digunakan untuk membangun model klasifikasi berdasarkan hubungan antara fitur hasil ekstraksi dan label kelas.

Seluruh model menggunakan data latih yang sama sehingga perbedaan performa yang diperoleh benar-benar berasal dari karakteristik algoritma klasifikasi maupun kombinasi fitur yang digunakan.

4. Menyimpan Model

```
for name, model in models.items():
    joblib.dump(
        model,
        f"{SHORT_NAMES[name]}_s{scenario_num}.pkl"
    )
```

Setelah proses pelatihan selesai, setiap model disimpan menggunakan **Joblib**.

Penyimpanan model dilakukan agar model yang telah dilatih dapat digunakan kembali tanpa harus melakukan proses pelatihan ulang.

Nama file disusun berdasarkan:

- nama algoritma
- nomor skenario

Contoh:

```
svm_s1.pkl
```

```
knn_s1.pkl
```

```
nb_s1.pkl
```

Pendekatan ini memudahkan proses evaluasi maupun implementasi sistem pada tahap selanjutnya.

Joblib dipilih karena lebih efisien dalam menyimpan objek NumPy berukuran besar dibandingkan modul `pickle` (Pedregosa et al., 2011).

5. Menyimpan StandardScaler

```
joblib.dump(  
    scaler,  
    f"scaler_s{scenario_num}.pkl"  
)
```

Objek **StandardScaler** juga disimpan.

Hal ini penting karena pada saat proses prediksi data baru, fitur harus mengalami transformasi menggunakan **mean** dan **standard deviation** yang sama seperti saat proses pelatihan.

Apabila scaler tidak disimpan dan dilakukan standardisasi ulang, distribusi data dapat berubah sehingga hasil prediksi menjadi tidak konsisten.

6. Menyimpan Label Encoder

```
joblib.dump(  
    encoder,  
    "label_encoder.pkl"  
)
```

Selain model dan scaler, **LabelEncoder** juga disimpan.

Hal ini bertujuan agar hasil prediksi yang masih berupa indeks numerik dapat dikonversi kembali menjadi nama kelas menggunakan fungsi `inverse_transform()`.

Sebagai contoh,

```
0 → Jambu
```

```
1 → Kunyit
```

```
2 → Paku
```

3 → Singkong

4 → Sirih

Dengan demikian hasil klasifikasi menjadi lebih mudah dipahami oleh pengguna.

Output

Fungsi ini menghasilkan:

- **models** → kumpulan model klasifikasi yang telah dilatih.
- **scaler** → objek StandardScaler.
- **X_train_scaled** → data latih setelah standardisasi.
- **X_test_scaled** → data uji setelah standardisasi.

Seluruh model, scaler, dan label encoder juga disimpan dalam format **.pkl** sehingga dapat digunakan kembali tanpa perlu melakukan proses pelatihan ulang.

```
def train_evaluate_save(X_train, y_train, X_test, y_test,
                        scenario_num, encoder):

    print(f"\n{'#' * 60}")
    print(f"SKENARIO {scenario_num} \u2014 shape fitur train:
{X_train.shape}, test: {X_test.shape}")
    print(f"{'#' * 60}")

    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    models = get_models()
    results = {}

    for name, model in models.items():
        model.fit(X_train_scaled, y_train)

        train_acc = model.score(X_train_scaled, y_train)
        pred = model.predict(X_test_scaled)
        test_acc = accuracy_score(y_test, pred)

        print(f"\n{'=' * 50}")
        print(name)
        print(f"{'=' * 50}")
        print(f"Train Accuracy : {train_acc*100:.2f}%")
        print(f"Test Accuracy : {test_acc*100:.2f}%")
        print(classification_report(y_test, pred,
```

```

target_names=encoder.classes_)

    cm = confusion_matrix(y_test, pred)
    plt.figure(figsize=(7, 6))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=encoder.classes_,
yticklabels=encoder.classes_)
    plt.title(f"Confusion Matrix {name} - Skenario
{scenario_num}")
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.show()

    joblib.dump(model, f"{SHORT_NAMES[name]}_s{scenario_num}.pkl")
    results[name] = {"train_acc": train_acc, "test_acc": test_acc}

    joblib.dump(scaler, f"scaler_s{scenario_num}.pkl")
    joblib.dump(encoder, "label_encoder.pkl")

    print(f"\nModel skenario {scenario_num} berhasil disimpan.")
    return results, scaler

```

Pelatihan, Evaluasi, dan Penyimpanan Model

Tahap ini merupakan inti dari proses klasifikasi, yaitu melakukan pelatihan model, mengevaluasi performa pada data uji, serta menyimpan model yang telah dilatih. Proses ini dilakukan untuk **setiap skenario fitur** sehingga performa masing-masing kombinasi fitur dapat dibandingkan secara objektif.

```

def train_evaluate_save(
    X_train,
    y_train,
    X_test,
    y_test,
    scenario_num,
    encoder
):

```

Tahapan yang dilakukan pada fungsi ini meliputi:

1. Standardisasi fitur.
 2. Pelatihan model.
 3. Evaluasi performa.
 4. Visualisasi Confusion Matrix.
 5. Penyimpanan model.
-

1. Menampilkan Informasi Skenario

```
print(f"SKENARIO {scenario_num}")
```

Pada awal proses ditampilkan nomor skenario beserta ukuran data latih dan data uji.

Informasi ini digunakan untuk memastikan bahwa setiap kombinasi fitur menghasilkan dimensi yang sesuai sebelum dilakukan proses pelatihan.

Sebagai contoh,

Scenario 1

Train : (892, 8)

Test : (56, 8)

menunjukkan bahwa skenario pertama menghasilkan delapan fitur.

2. Standardisasi Fitur

```
scaler = StandardScaler()  
X_train_scaled = scaler.fit_transform(X_train)  
X_test_scaled = scaler.transform(X_test)
```

Sebelum model dilatih, seluruh fitur dinormalisasi menggunakan **StandardScaler**.

Standardisasi dilakukan agar setiap fitur memiliki rata-rata nol dan simpangan baku satu sehingga seluruh fitur berada pada skala yang sebanding.

Tahapan ini sangat penting terutama untuk algoritma **Support Vector Machine (SVM)** dan **K-Nearest Neighbor (KNN)** yang sensitif terhadap skala fitur.

Tanpa proses standardisasi, fitur dengan rentang nilai besar seperti **Area** atau **Perimeter** dapat mendominasi fitur lain sehingga menurunkan performa klasifikasi (Géron, 2022).

Parameter standardisasi dihitung **hanya dari data latih** menggunakan `fit_transform()`, kemudian diterapkan pada data uji menggunakan `transform()` untuk menghindari **data leakage**.

3. Pelatihan Model

```
model.fit(  
    X_train_scaled,
```

```
) y_train
```

Setiap algoritma dilatih menggunakan data latih yang telah melalui proses standardisasi.

Model mempelajari hubungan antara fitur hasil ekstraksi dan label kelas sehingga dapat digunakan untuk mengklasifikasikan citra baru.

Karena seluruh algoritma menggunakan data latih yang sama, maka perbedaan performa yang diperoleh berasal dari karakteristik algoritma maupun kombinasi fitur yang digunakan.

4. Evaluasi Akurasi Data Latih

```
train_acc = model.score(  
    X_train_scaled,  
    y_train  
)
```

Akurasi data latih digunakan untuk mengetahui seberapa baik model mampu mempelajari pola pada data yang digunakan selama proses pelatihan.

Nilai ini dibandingkan dengan akurasi data uji untuk mengidentifikasi kemungkinan:

- **Good Fit** → Train Accuracy $\approx$ Test Accuracy
- **Overfitting** → Train Accuracy jauh lebih tinggi daripada Test Accuracy
- **Underfitting** → Train Accuracy dan Test Accuracy sama-sama rendah

Pendekatan ini umum digunakan pada penelitian *machine learning* untuk mengevaluasi kemampuan generalisasi model (Bishop, 2006).

5. Prediksi Data Uji

```
pred = model.predict(  
    X_test_scaled  
)
```

Model yang telah dilatih kemudian digunakan untuk memprediksi kelas seluruh citra pada data uji.

Data uji dipilih karena belum pernah digunakan selama proses pelatihan sehingga mampu memberikan estimasi performa model terhadap data baru.

6. Menghitung Akurasi

```
test_acc = accuracy_score(
    y_test,
    pred
)
```

Accuracy digunakan sebagai metrik utama penelitian karena permasalahan yang dihadapi merupakan **klasifikasi multikelas** dengan distribusi data yang relatif seimbang.

Accuracy menunjukkan persentase sampel yang berhasil diklasifikasikan dengan benar terhadap seluruh data uji.

Nilai accuracy dihitung menggunakan persamaan:

$$[\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}]$$

Semakin tinggi nilai accuracy, semakin baik performa model dalam mengenali jenis daun herbal.

7. Classification Report

```
classification_report(...)
```

Selain accuracy, penelitian ini juga menggunakan:

- Precision
- Recall
- F1-score

Penggunaan ketiga metrik tersebut bertujuan memberikan evaluasi yang lebih lengkap dibandingkan hanya menggunakan accuracy.

Precision

Precision menunjukkan proporsi prediksi positif yang benar.

Semakin tinggi precision maka semakin sedikit kesalahan prediksi (*False Positive*).

Recall

Recall menunjukkan kemampuan model menemukan seluruh sampel yang benar pada setiap kelas.

Semakin tinggi recall maka semakin sedikit sampel yang gagal dikenali (*False Negative*).

F1-score

F1-score merupakan rata-rata harmonik antara precision dan recall.

Metrik ini digunakan karena mampu memberikan keseimbangan antara kedua ukuran tersebut sehingga lebih representatif dibandingkan hanya melihat accuracy saja (Powers, 2011).

8. Confusion Matrix

```
cm = confusion_matrix(  
    y_test,  
    pred  
)
```

Confusion Matrix digunakan untuk menganalisis performa model pada setiap kelas secara rinci.

Visualisasi dilakukan menggunakan

```
sns.heatmap(...)
```

Heatmap dipilih karena lebih mudah menunjukkan pola kesalahan klasifikasi dibandingkan tabel numerik biasa.

Melalui Confusion Matrix dapat diketahui:

- kelas mana yang paling sering salah dikenali,
- pasangan kelas yang sering tertukar,
- distribusi prediksi benar pada setiap kelas.

Analisis ini sangat penting karena dua model dapat memiliki accuracy yang sama, namun menghasilkan pola kesalahan klasifikasi yang berbeda.

9. Penyimpanan Model

```
joblib.dump(...)
```

Setelah proses evaluasi selesai, setiap model disimpan menggunakan **Joblib**.

Selain model, penelitian ini juga menyimpan:

- StandardScaler
- LabelEncoder

Penyimpanan seluruh objek dilakukan agar proses prediksi pada data baru menggunakan konfigurasi yang sama seperti saat pelatihan sehingga hasil klasifikasi tetap konsisten.

Output

Fungsi ini menghasilkan:

- Train Accuracy
- Test Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Model (*.pkl)
- StandardScaler
- LabelEncoder

Seluruh hasil tersebut digunakan sebagai dasar untuk membandingkan performa setiap skenario fitur dan menentukan kombinasi fitur terbaik.

```
SCENARIO_LABELS = {
    1: "1: GLCM",
    2: "2: LBP",
    3: "3: FD",
    4: "4: GLCM+FD",
    5: "5: LBP+FD",
    6: "6: GLCM+LBP+FD",
    7: "7: Shape",
    8: "8: GLCM+Shape",
    9: "9: LBP+Shape",
    10: "10: GLCM+LBP+Shape",
}

all_results = {}
all_scalers = {}
```

4. IMPLEMENTASI DAN PENGUJIAN

4.1 Lingkungan Implementasi (*Environment*)

- **Bahasa Pemrograman:** Python 3
- **Platform:** Google Colaboratory (Google Colab)
- **Library Utama:** OpenCV (cv2) untuk operasi citra dasar, `scikit-image` untuk ekstraksi fitur GLCM/LBP dan `regionprops`, `rembg` untuk penghapusan latar belakang berbasis U²-Net, `scikit-learn` untuk *preprocessing*, *train-test split*, model SVM/KNN/*Naive Bayes*, dan metrik evaluasi, NumPy/Pandas untuk manipulasi data numerik dan tabular, Matplotlib/Seaborn untuk visualisasi, serta `joblib` untuk penyimpanan model dan *scaler* terlatih. Seluruh *library* tersebut diimpor pada cell pertama di Bagian 2.1.

- **Perangkat Keras:** Tidak memerlukan GPU — salah satu keunggulan pendekatan *hybrid handcrafted feature + machine learning* ringan ini dibandingkan pendekatan *deep learning* (CNN/MobileNetV2) pada Tabel SOTA Bagian 1.1, yang umumnya membutuhkan akselerasi GPU untuk pelatihan dalam waktu yang wajar.

4.2 Skenario Pengujian

Pembagian data latih dan data uji dilakukan dengan rasio **80% : 20%** menggunakan `train_test_split` dengan parameter `stratify=y_encoded` (memastikan proporsi setiap kelas tetap seimbang antara data latih dan data uji) dan `random_state=42` (memastikan hasil pembagian dapat direproduksi). Rasio 80:20 dipilih sebagai rasio standar yang umum digunakan pada studi klasifikasi citra daun dengan ukuran dataset kecil-menengah, termasuk oleh Prayoga dkk. (2023) pada Tabel SOTA yang menggunakan rasio identik.

Untuk menjawab Rumusan Masalah ke-3 (Bagian 1.2) secara sistematis dan langsung mengisi *research gap* yang diidentifikasi, **sepuluh skenario kombinasi fitur** dirancang dan diuji:

Skenario	Kombinasi Fitur	Tujuan Pengujian
1	GLCM	<i>Baseline</i> tekstur dasar (replikasi pendekatan tekstur-saja, mis. Furqan dkk. 2020)
2	LBP	<i>Baseline</i> tekstur kedua, belum diuji secara individual pada studi SOTA
3	<i>Fourier Descriptor</i> (FD)	<i>Baseline</i> bentuk global-frekuensi saja (replikasi pendekatan bentuk-saja, mis. Prayoga dkk. 2023)
4	GLCM + FD	Hybrid tekstur (GLCM) + bentuk (FD)
5	LBP + FD	Hybrid tekstur (LBP) + bentuk (FD)
6	GLCM + LBP + FD	Hybrid penuh: kedua fitur tekstur + FD
7	Deskriptor Bentuk Geometris (<i>Shape</i>)	<i>Baseline</i> bentuk eksplisit-geometris saja (replikasi pendekatan bentuk-saja, mis. Herdiansah dkk. 2022 / Nurhikam dkk. 2024)
8	GLCM + <i>Shape</i>	Hybrid tekstur (GLCM) + bentuk (geometris)
9	LBP + <i>Shape</i>	Hybrid tekstur (LBP) + bentuk (geometris)
10	GLCM + LBP + <i>Shape</i>	Hybrid penuh: kedua fitur tekstur + bentuk geometris

Desain ini secara khusus menguji **dua representasi bentuk yang berbeda** (FD vs deskriptor geometris) masing-masing dikombinasikan dengan fitur tekstur secara bertahap (tekstur tunggal → tekstur ganda), sehingga kontribusi setiap kategori fitur terhadap akurasi akhir dapat dianalisis secara terpisah — desain *ablation study* semacam ini belum ditemukan pada penelitian-penelitian Tabel SOTA, yang umumnya hanya membandingkan satu kombinasi fitur tetap terhadap penggunaan satu fitur saja.

Setiap skenario memanggil fungsi `train_evaluate_save` yang sama (Bagian 3.2) — mengembalikan akurasi *train/test*, *classification report* (presisi, *recall*, *F1-score* per kelas), dan *confusion matrix* — serta menyimpan model dan *scaler* terlatih untuk dipakai kembali pada tahap prediksi citra baru (Lampiran). Kode dan hasil keluaran (akurasi, *classification report*, dan *confusion matrix*) untuk seluruh sepuluh skenario ditampilkan pada cell-cell berikut.

SKENARIO 1

```
scenario = 1

X_train_feat = np.array([
    extract_features_scenario(g, c, scenario)
    for g, c in zip(gray_images, contour_images)
])
X_test_feat = np.array([
    extract_features_scenario(g, c, scenario)
    for g, c in zip(gray_test, contour_test)
])

results_s1, scaler_s1 = train_evaluate_save(
    X_train_feat, y_train_aug, X_test_feat, y_test, scenario, encoder
)

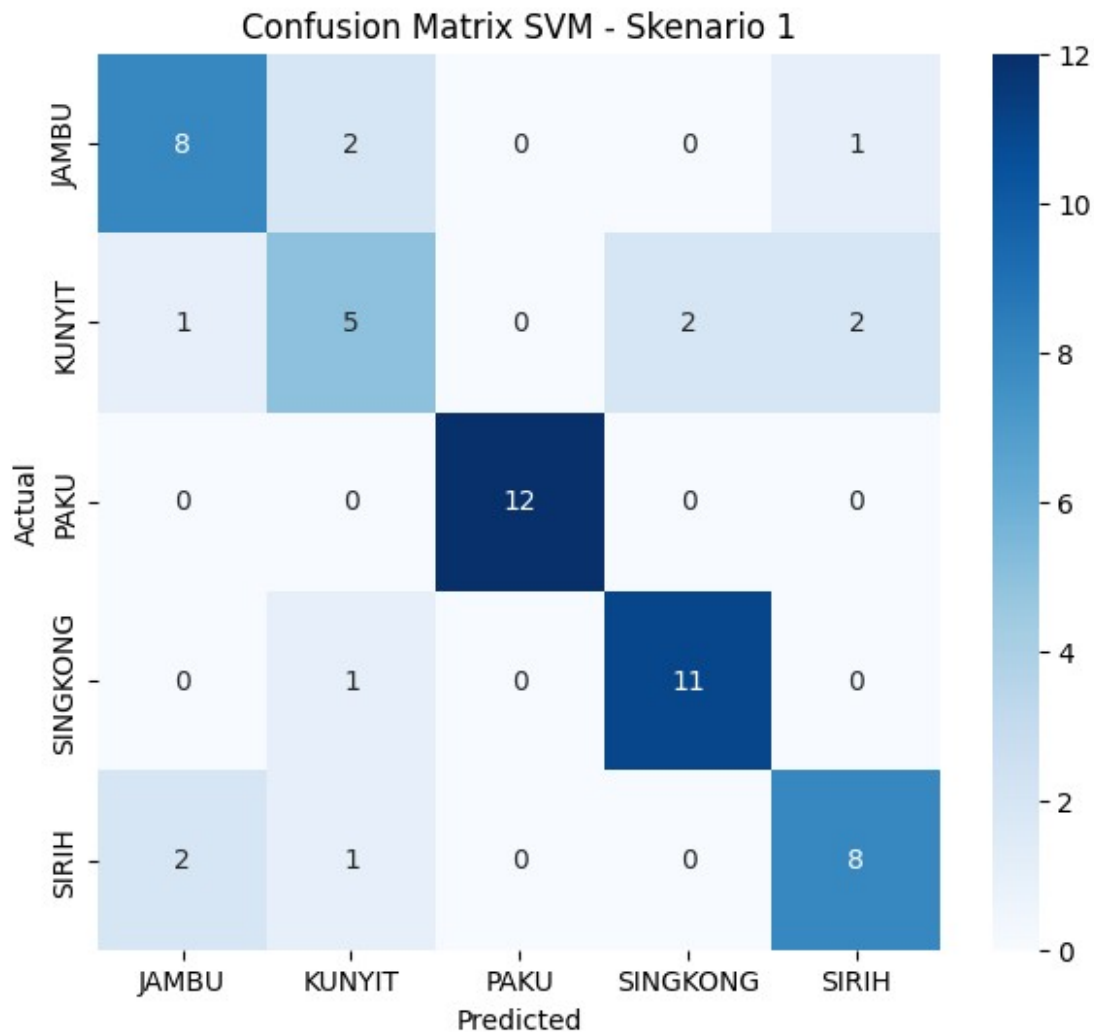
all_results[scenario] = results_s1
all_scalers[scenario] = scaler_s1
```

```
#####
SKENARIO 1 – shape fitur train: (892, 9), test: (56, 9)
#####
```

```
=====
SVM
=====
```

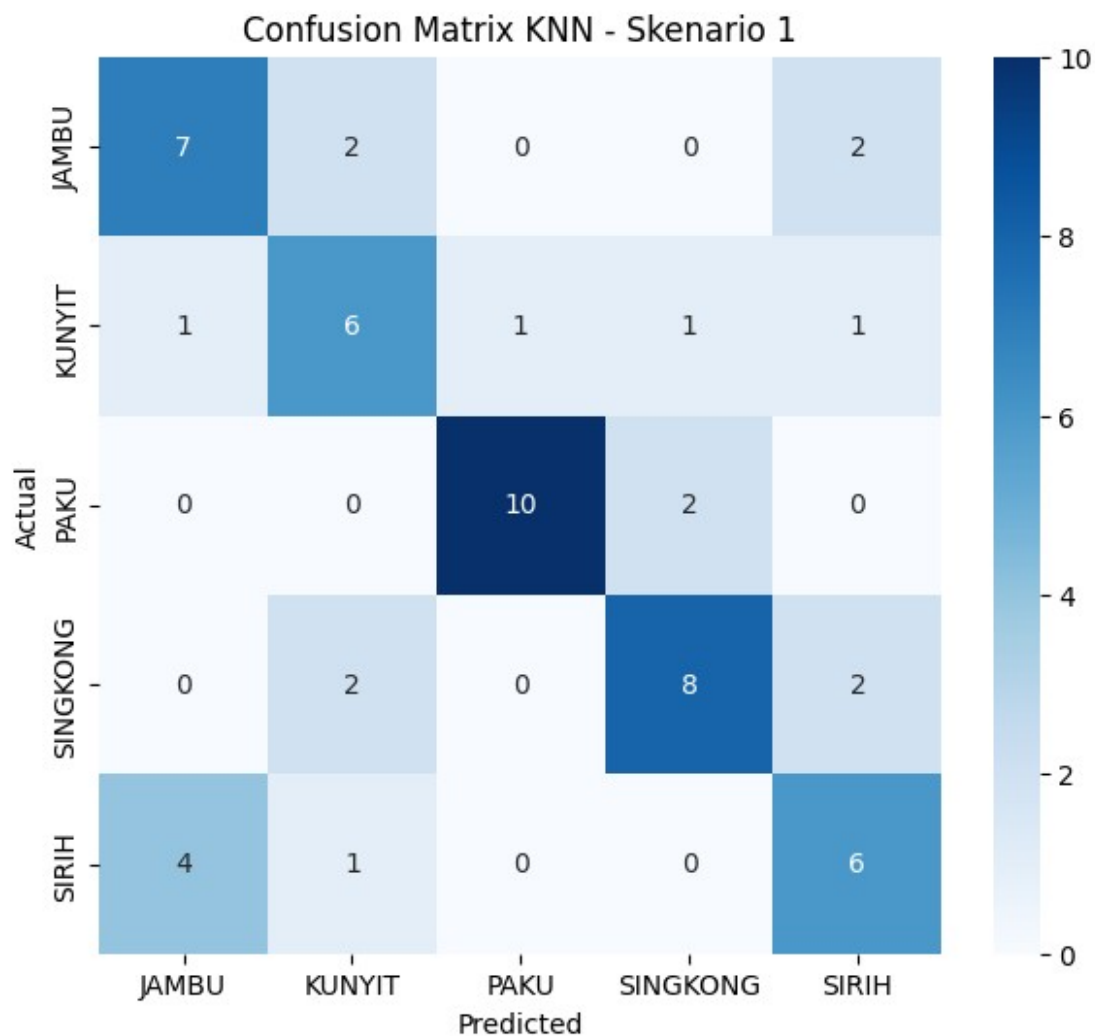
Train Accuracy	: 69.73%			
Test Accuracy	: 78.57%			
	precision	recall	f1-score	support
JAMBU	0.73	0.73	0.73	11
KUNYIT	0.56	0.50	0.53	10
PAKU	1.00	1.00	1.00	12
SINGKONG	0.85	0.92	0.88	12

SIRIH	0.73	0.73	0.73	11
accuracy			0.79	56
macro avg	0.77	0.77	0.77	56
weighted avg	0.78	0.79	0.78	56



=====				
KNN				
=====				
Train Accuracy : 84.75%				
Test Accuracy : 66.07%				
	precision	recall	f1-score	support
JAMBU	0.58	0.64	0.61	11
KUNYIT	0.55	0.60	0.57	10

PAKU	0.91	0.83	0.87	12
SINGKONG	0.73	0.67	0.70	12
SIRIH	0.55	0.55	0.55	11
accuracy			0.66	56
macro avg	0.66	0.66	0.66	56
weighted avg	0.67	0.66	0.66	56



=====

Naive Bayes

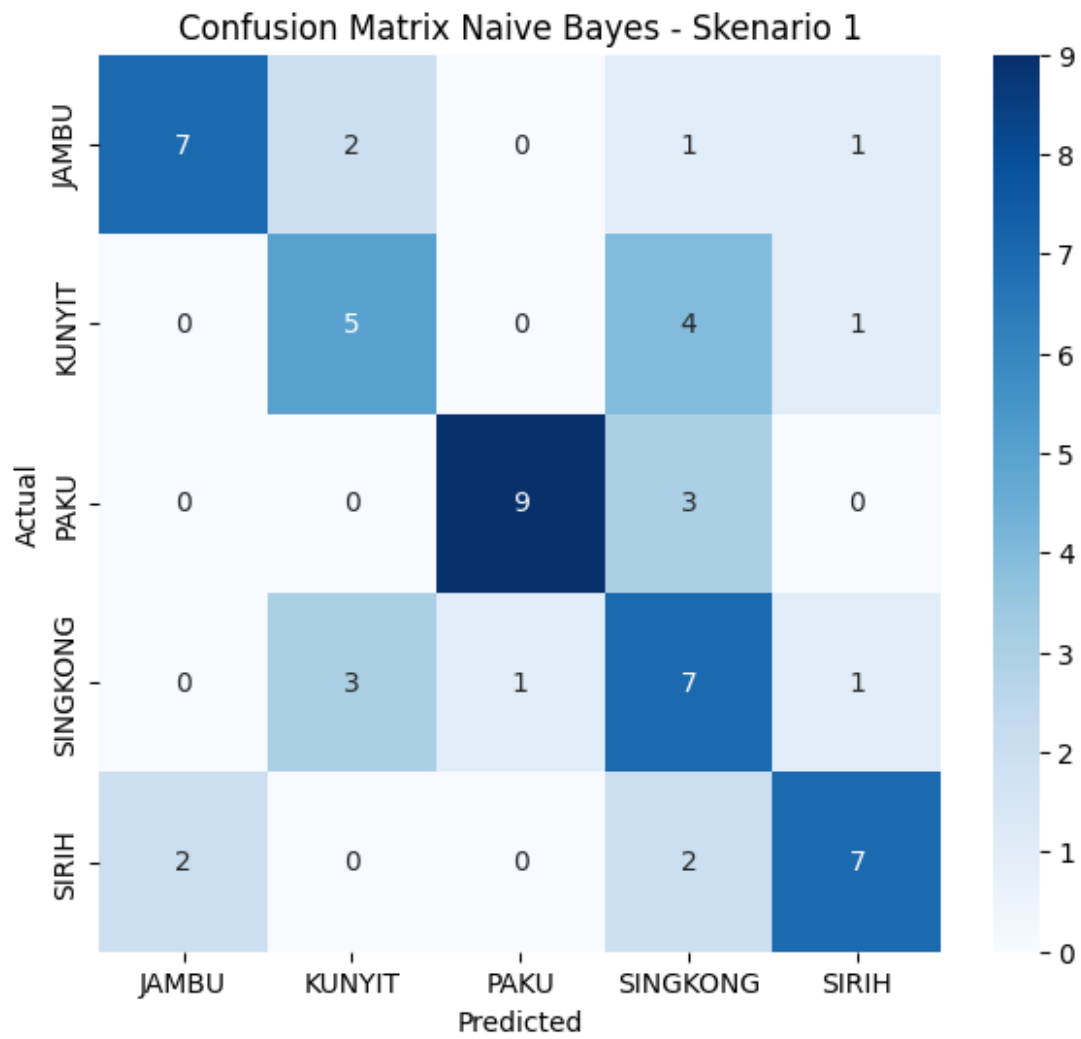
=====

Train Accuracy : 47.76%

Test Accuracy : 62.50%

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

JAMBU	0.78	0.64	0.70	11
KUNYIT	0.50	0.50	0.50	10
PAKU	0.90	0.75	0.82	12
SINGKONG	0.41	0.58	0.48	12
SIRIH	0.70	0.64	0.67	11
accuracy			0.62	56
macro avg	0.66	0.62	0.63	56
weighted avg	0.66	0.62	0.64	56



Model skenario 1 berhasil disimpan.

SKENARIO 2

```
scenario = 2

X_train_feat = np.array([
    extract_features_scenario(g, c, scenario)
    for g, c in zip(gray_images, contour_images)
])

X_test_feat = np.array([
    extract_features_scenario(g, c, scenario)
    for g, c in zip(gray_test, contour_test)
])

results_s2, scaler_s2 = train_evaluate_save(
    X_train_feat, y_train_aug, X_test_feat, y_test, scenario, encoder
)

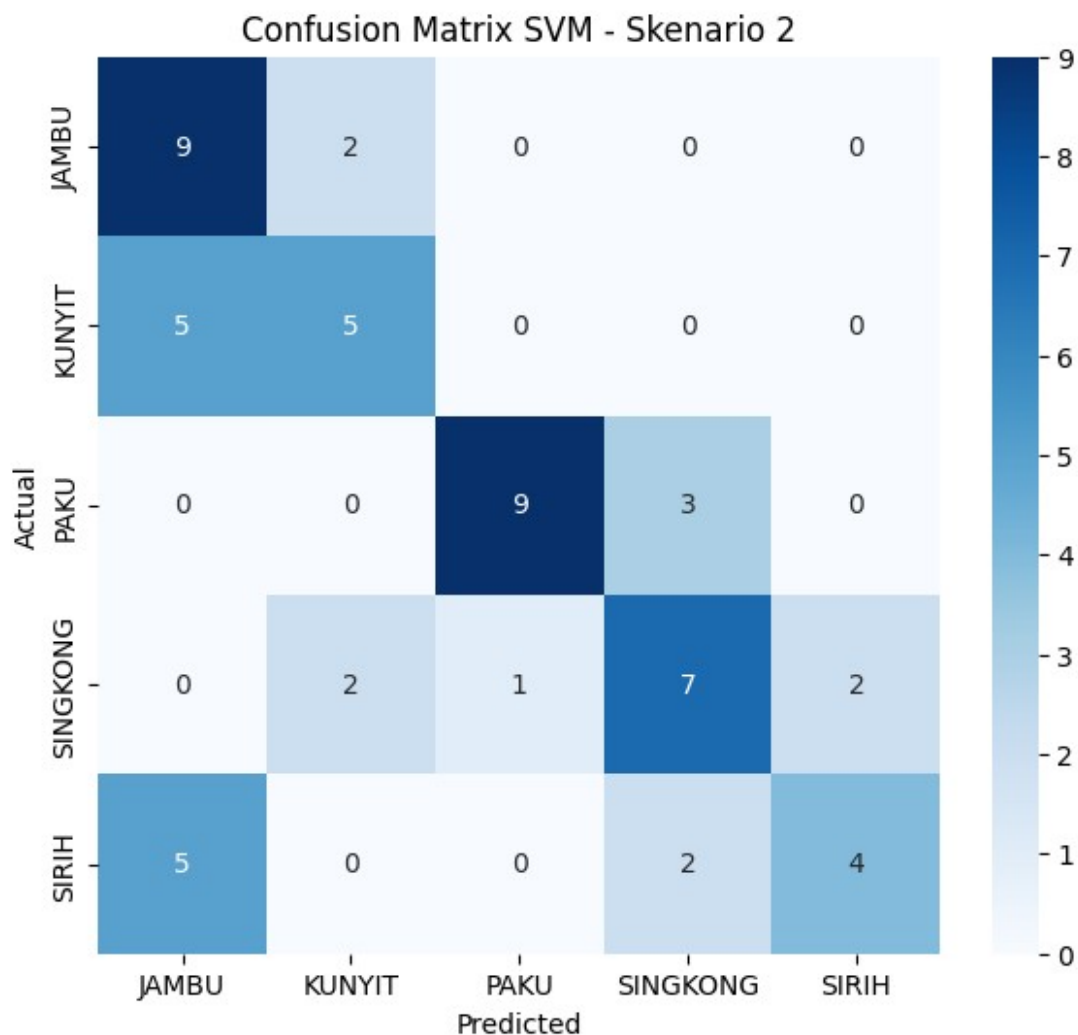
all_results[scenario] = results_s2
all_scalers[scenario] = scaler_s2
```

```
#####
SKENARIO 2 – shape fitur train: (892, 4), test: (56, 4)
#####
```

```
=====
SVM
=====
Train Accuracy : 61.77%
Test Accuracy  : 60.71%

```

	precision	recall	f1-score	support
JAMBU	0.47	0.82	0.60	11
KUNYIT	0.56	0.50	0.53	10
PAKU	0.90	0.75	0.82	12
SINGKONG	0.58	0.58	0.58	12
SIRIH	0.67	0.36	0.47	11
accuracy			0.61	56
macro avg	0.64	0.60	0.60	56
weighted avg	0.64	0.61	0.60	56

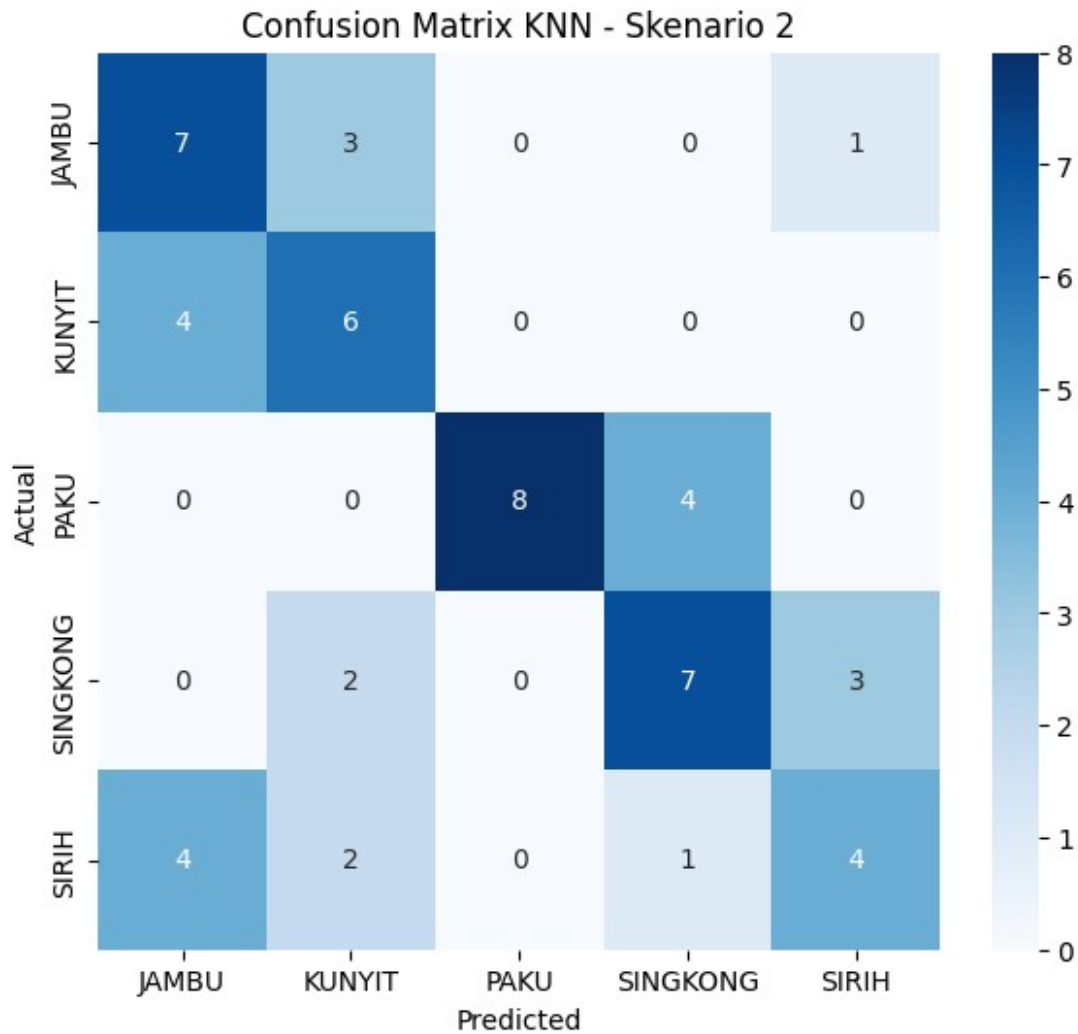


KNN

Train Accuracy : 84.08%

Test Accuracy : 57.14%

	precision	recall	f1-score	support
JAMBU	0.47	0.64	0.54	11
KUNYIT	0.46	0.60	0.52	10
PAKU	1.00	0.67	0.80	12
SINGKONG	0.58	0.58	0.58	12
SIRIH	0.50	0.36	0.42	11
accuracy			0.57	56
macro avg	0.60	0.57	0.57	56
weighted avg	0.61	0.57	0.58	56



=====

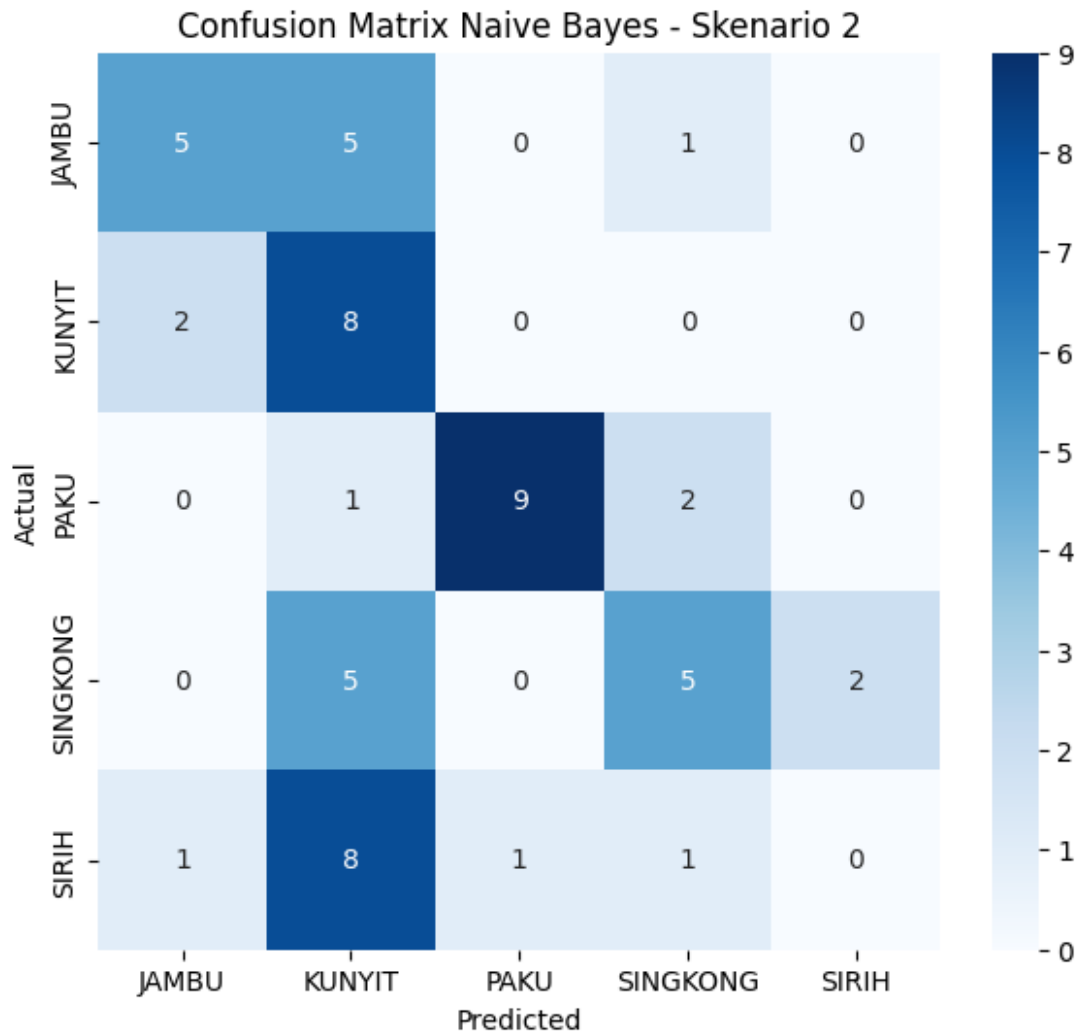
Naive Bayes

=====

Train Accuracy : 45.85%

Test Accuracy : 48.21%

	precision	recall	f1-score	support
JAMBU	0.62	0.45	0.53	11
KUNYIT	0.30	0.80	0.43	10
PAKU	0.90	0.75	0.82	12
SINGKONG	0.56	0.42	0.48	12
SIRIH	0.00	0.00	0.00	11
accuracy			0.48	56
macro avg	0.48	0.48	0.45	56
weighted avg	0.49	0.48	0.46	56



Model skenario 2 berhasil disimpan.

SKENARIO 3

```
scenario = 3

X_train_feat = np.array([
    extract_features_scenario(g, c, scenario)
    for g, c in zip(gray_images, contour_images)
])

X_test_feat = np.array([
    extract_features_scenario(g, c, scenario)
    for g, c in zip(gray_test, contour_test)
])
```

```
results_s3, scaler_s3 = train_evaluate_save(
    X_train_feat, y_train_aug, X_test_feat, y_test, scenario, encoder
)
```

```
all_results[scenario] = results_s3
all_scalers[scenario] = scaler_s3
```

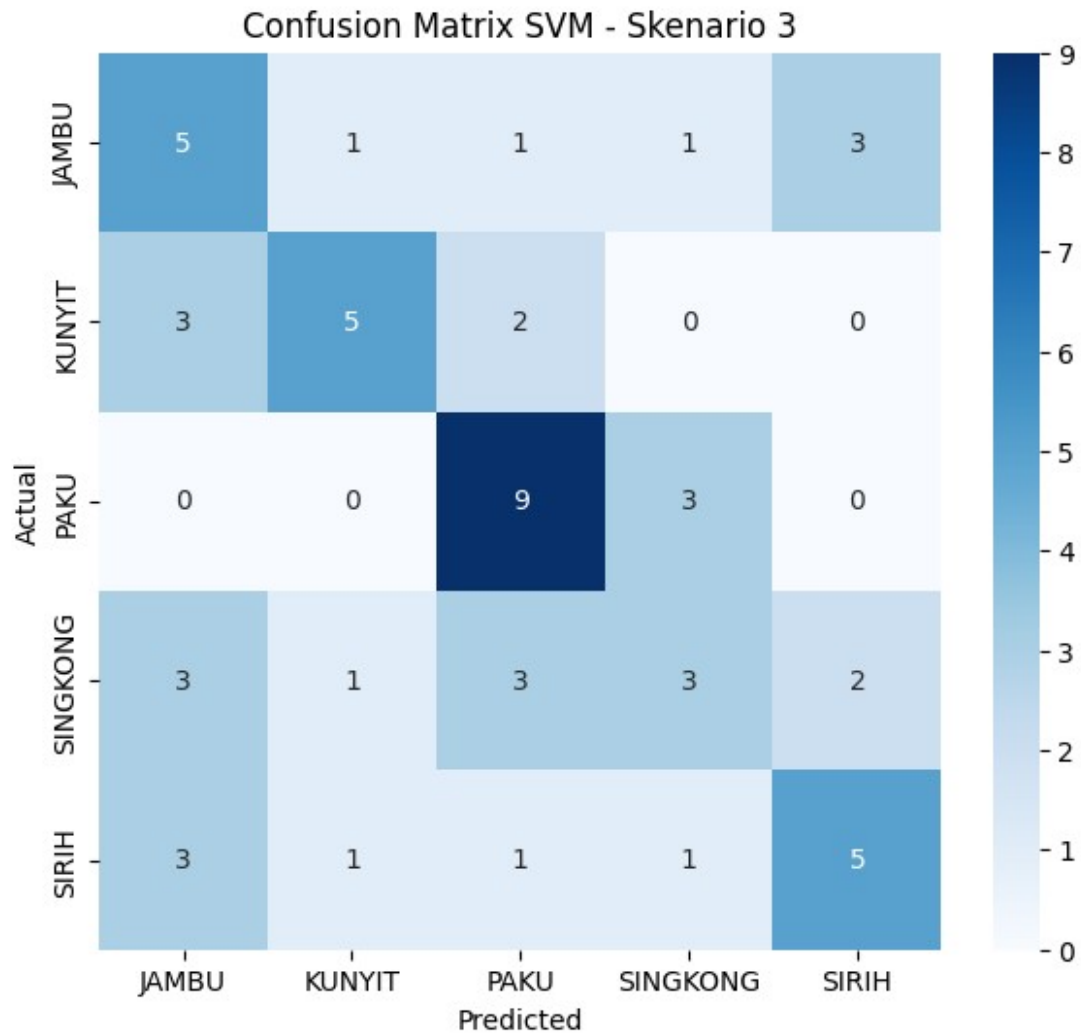
```
#####
SKENARIO 3 – shape fitur train: (892, 40), test: (56, 40)
#####
```

```
=====
SVM
=====
```

```
Train Accuracy : 83.97%
```

```
Test Accuracy  : 48.21%
```

	precision	recall	f1-score	support
JAMBU	0.36	0.45	0.40	11
KUNYIT	0.62	0.50	0.56	10
PAKU	0.56	0.75	0.64	12
SINGKONG	0.38	0.25	0.30	12
SIRIH	0.50	0.45	0.48	11
accuracy			0.48	56
macro avg	0.48	0.48	0.47	56
weighted avg	0.48	0.48	0.47	56

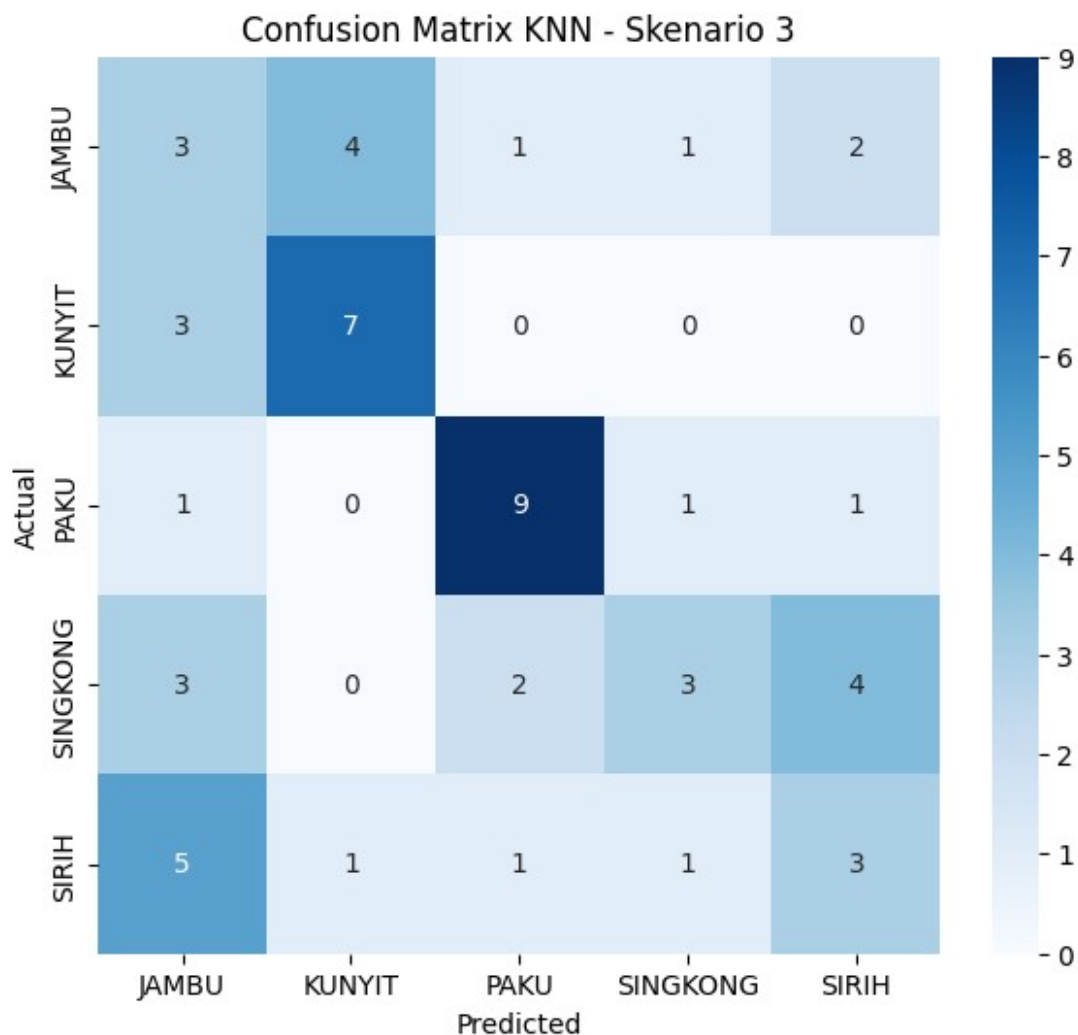


KNN

Train Accuracy : 80.04%

Test Accuracy : 44.64%

	precision	recall	f1-score	support
JAMBU	0.20	0.27	0.23	11
KUNYIT	0.58	0.70	0.64	10
PAKU	0.69	0.75	0.72	12
SINGKONG	0.50	0.25	0.33	12
SIRIH	0.30	0.27	0.29	11
accuracy			0.45	56
macro avg	0.46	0.45	0.44	56
weighted avg	0.46	0.45	0.44	56



=====

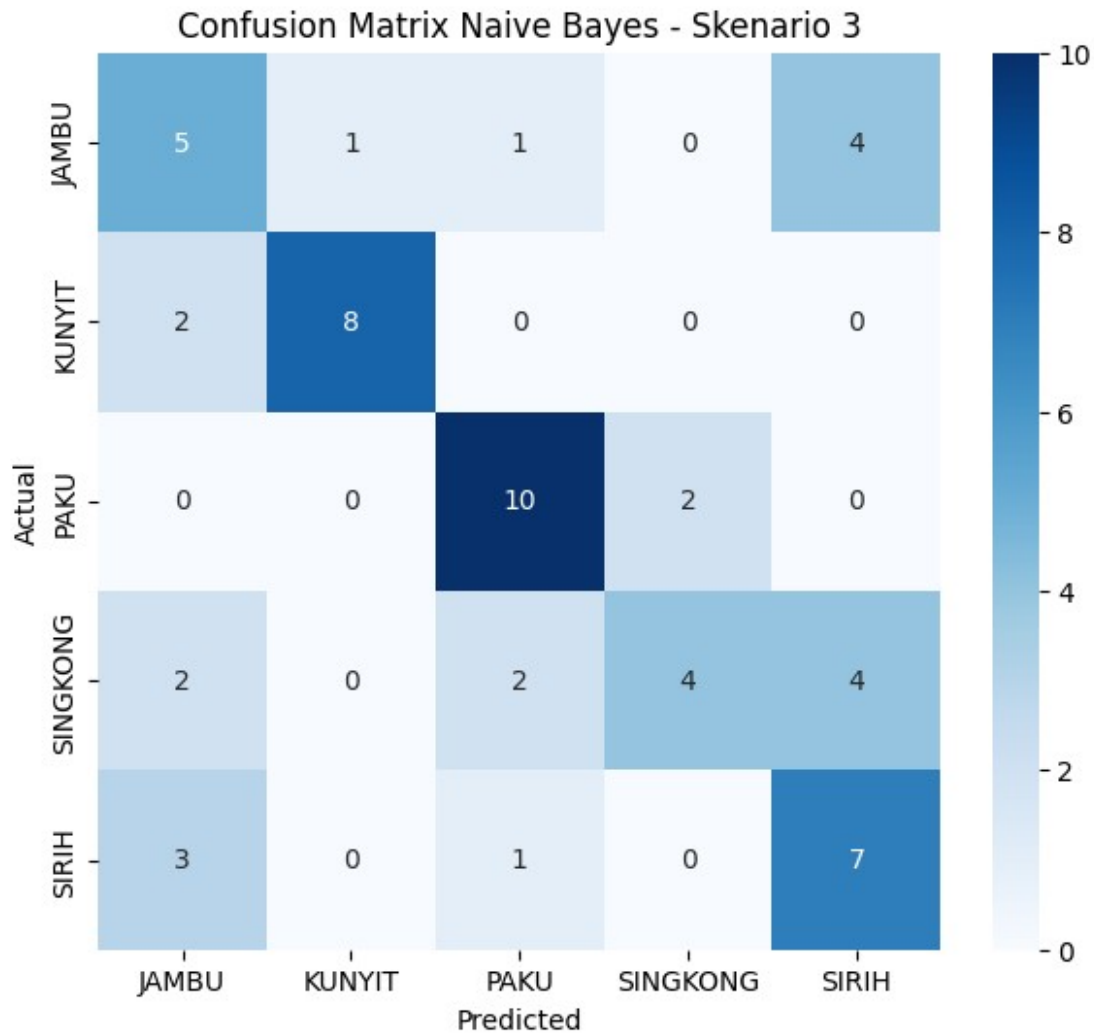
Naive Bayes

=====

Train Accuracy : 63.79%

Test Accuracy : 60.71%

	precision	recall	f1-score	support
JAMBU	0.42	0.45	0.43	11
KUNYIT	0.89	0.80	0.84	10
PAKU	0.71	0.83	0.77	12
SINGKONG	0.67	0.33	0.44	12
SIRIH	0.47	0.64	0.54	11
accuracy			0.61	56
macro avg	0.63	0.61	0.61	56
weighted avg	0.63	0.61	0.60	56



Model skenario 3 berhasil disimpan.

SKENARIO 4

```
scenario = 4

X_train_feat = np.array([
    extract_features_scenario(g, c, scenario)
    for g, c in zip(gray_images, contour_images)
])

X_test_feat = np.array([
    extract_features_scenario(g, c, scenario)
    for g, c in zip(gray_test, contour_test)
])
```

```
results_s4, scaler_s4 = train_evaluate_save(  
    X_train_feat, y_train_aug, X_test_feat, y_test, scenario, encoder  
)
```

```
all_results[scenario] = results_s4  
all_scalers[scenario] = scaler_s4
```

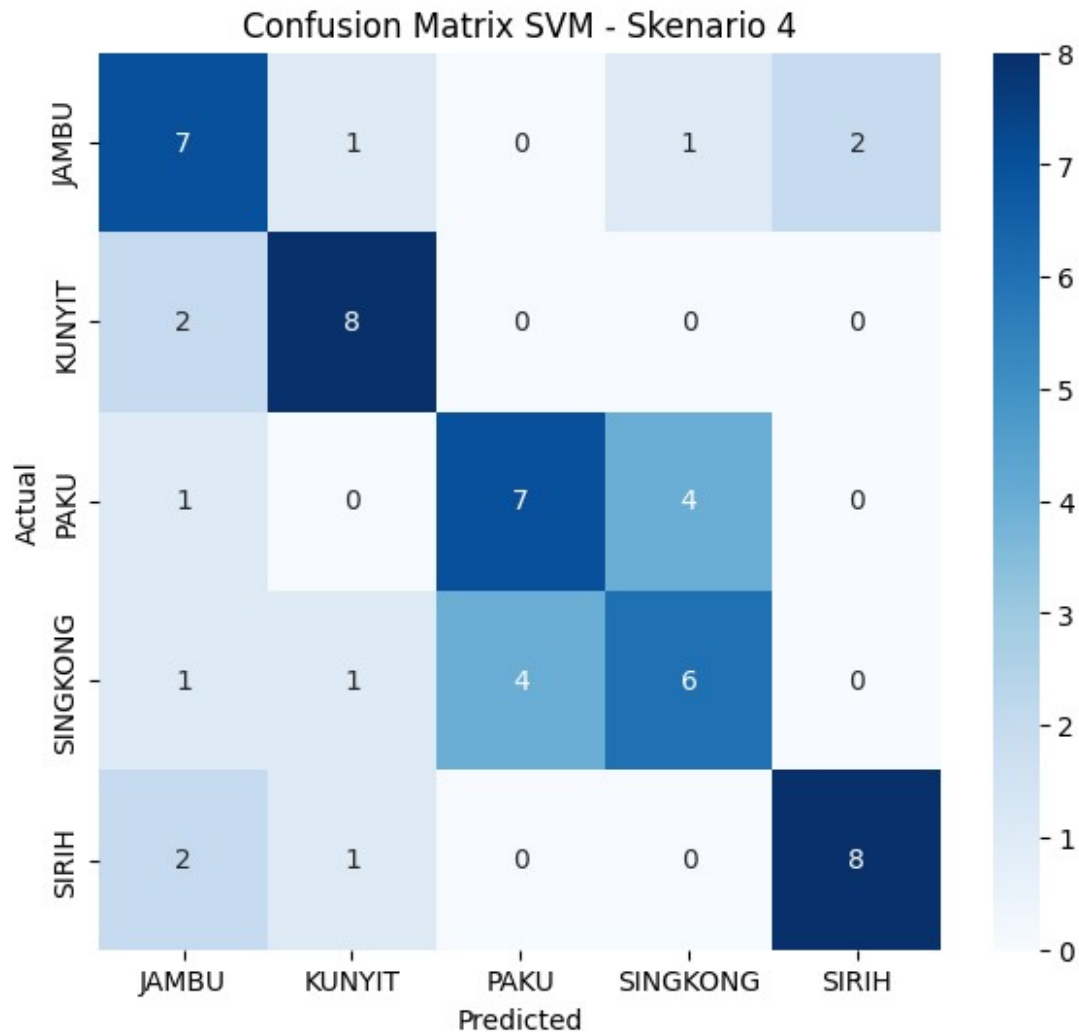
```
#####  
SKENARIO 4 – shape fitur train: (892, 49), test: (56, 49)  
#####
```

```
=====  
SVM  
=====
```

```
Train Accuracy : 93.95%
```

```
Test Accuracy  : 64.29%
```

	precision	recall	f1-score	support
JAMBU	0.54	0.64	0.58	11
KUNYIT	0.73	0.80	0.76	10
PAKU	0.64	0.58	0.61	12
SINGKONG	0.55	0.50	0.52	12
SIRIH	0.80	0.73	0.76	11
accuracy			0.64	56
macro avg	0.65	0.65	0.65	56
weighted avg	0.65	0.64	0.64	56

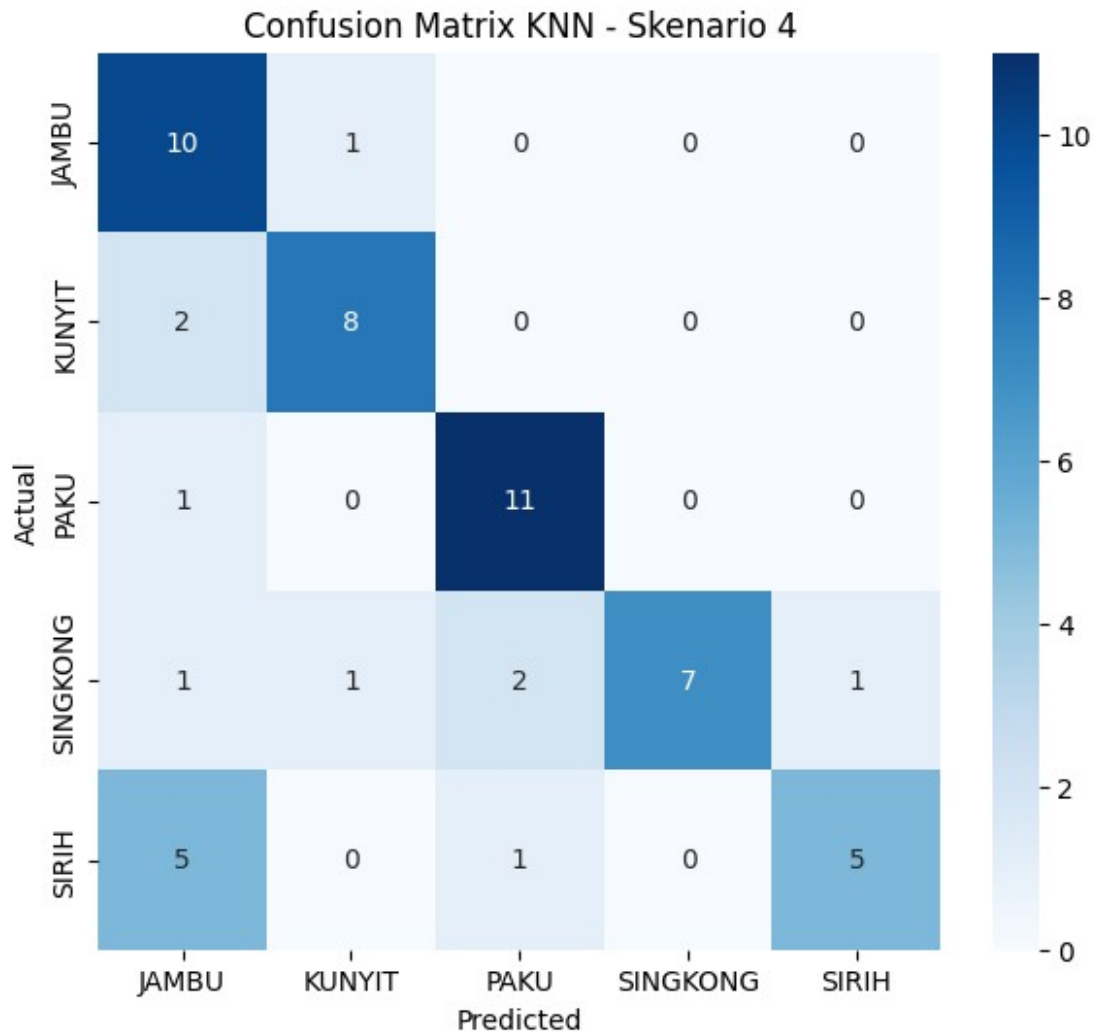


KNN

Train Accuracy : 88.34%

Test Accuracy : 73.21%

	precision	recall	f1-score	support
JAMBU	0.53	0.91	0.67	11
KUNYIT	0.80	0.80	0.80	10
PAKU	0.79	0.92	0.85	12
SINGKONG	1.00	0.58	0.74	12
SIRIH	0.83	0.45	0.59	11
accuracy			0.73	56
macro avg	0.79	0.73	0.73	56
weighted avg	0.79	0.73	0.73	56



=====

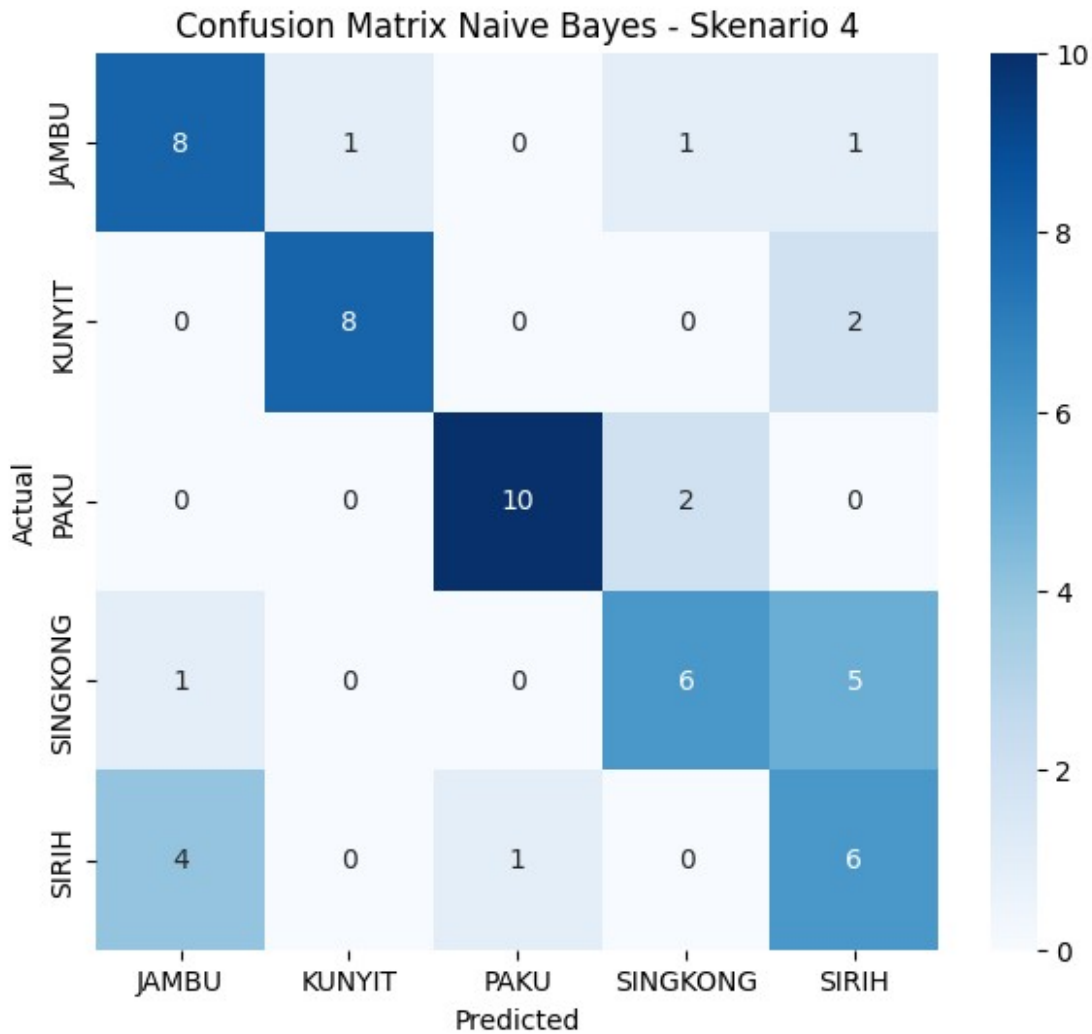
Naive Bayes

=====

Train Accuracy : 67.38%

Test Accuracy : 67.86%

	precision	recall	f1-score	support
JAMBU	0.62	0.73	0.67	11
KUNYIT	0.89	0.80	0.84	10
PAKU	0.91	0.83	0.87	12
SINGKONG	0.67	0.50	0.57	12
SIRIH	0.43	0.55	0.48	11
accuracy			0.68	56
macro avg	0.70	0.68	0.69	56
weighted avg	0.70	0.68	0.68	56



Model skenario 4 berhasil disimpan.

SKENARIO 5

```
scenario = 5

X_train_feat = np.array([
    extract_features_scenario(g, c, scenario)
    for g, c in zip(gray_images, contour_images)
])

X_test_feat = np.array([
    extract_features_scenario(g, c, scenario)
    for g, c in zip(gray_test, contour_test)
])
```

```
results_s5, scaler_s5 = train_evaluate_save(  
    X_train_feat, y_train_aug, X_test_feat, y_test, scenario, encoder  
)
```

```
all_results[scenario] = results_s5  
all_scalers[scenario] = scaler_s5
```

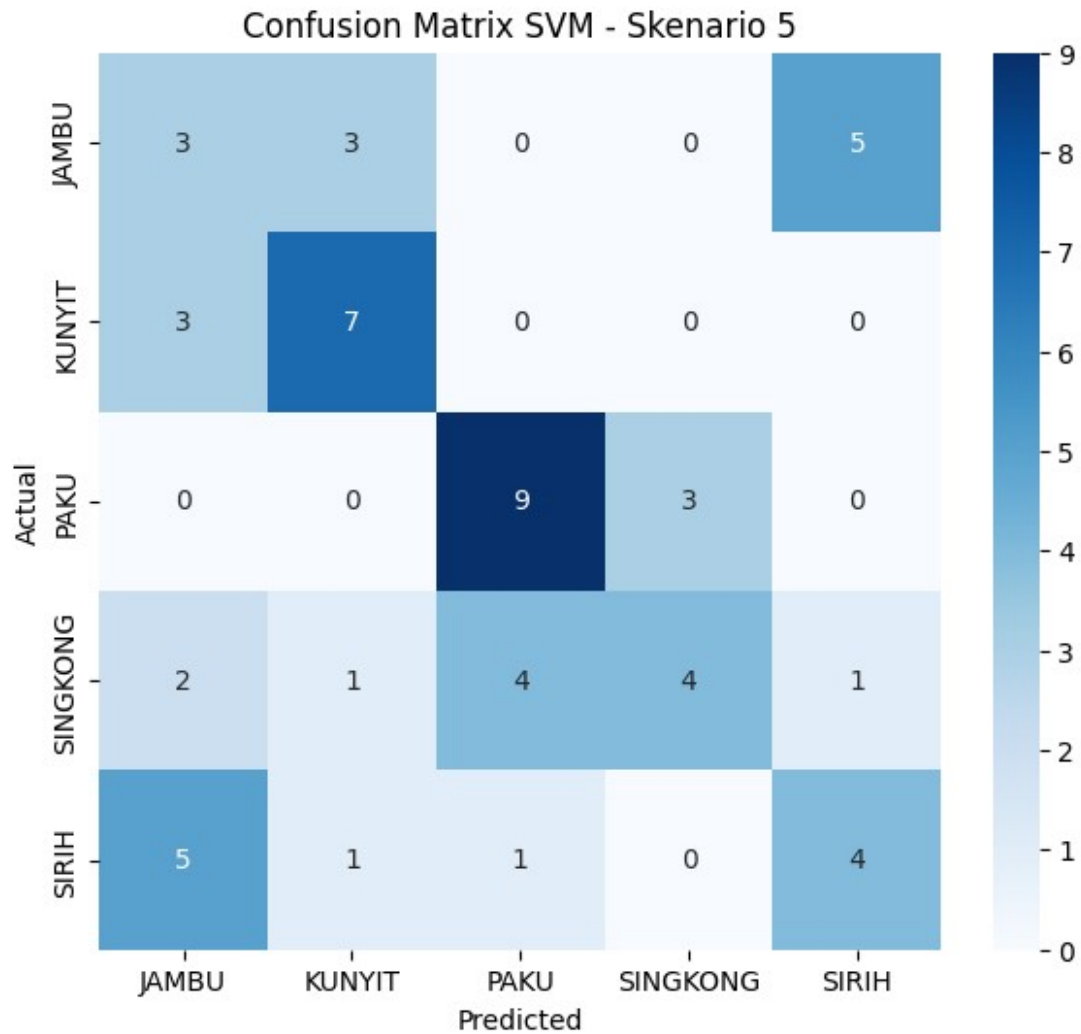
```
#####  
SKENARIO 5 – shape fitur train: (892, 44), test: (56, 44)  
#####
```

```
=====  
SVM  
=====
```

Train Accuracy : 87.22%

Test Accuracy : 48.21%

	precision	recall	f1-score	support
JAMBU	0.23	0.27	0.25	11
KUNYIT	0.58	0.70	0.64	10
PAKU	0.64	0.75	0.69	12
SINGKONG	0.57	0.33	0.42	12
SIRIH	0.40	0.36	0.38	11
accuracy			0.48	56
macro avg	0.49	0.48	0.48	56
weighted avg	0.49	0.48	0.48	56



=====

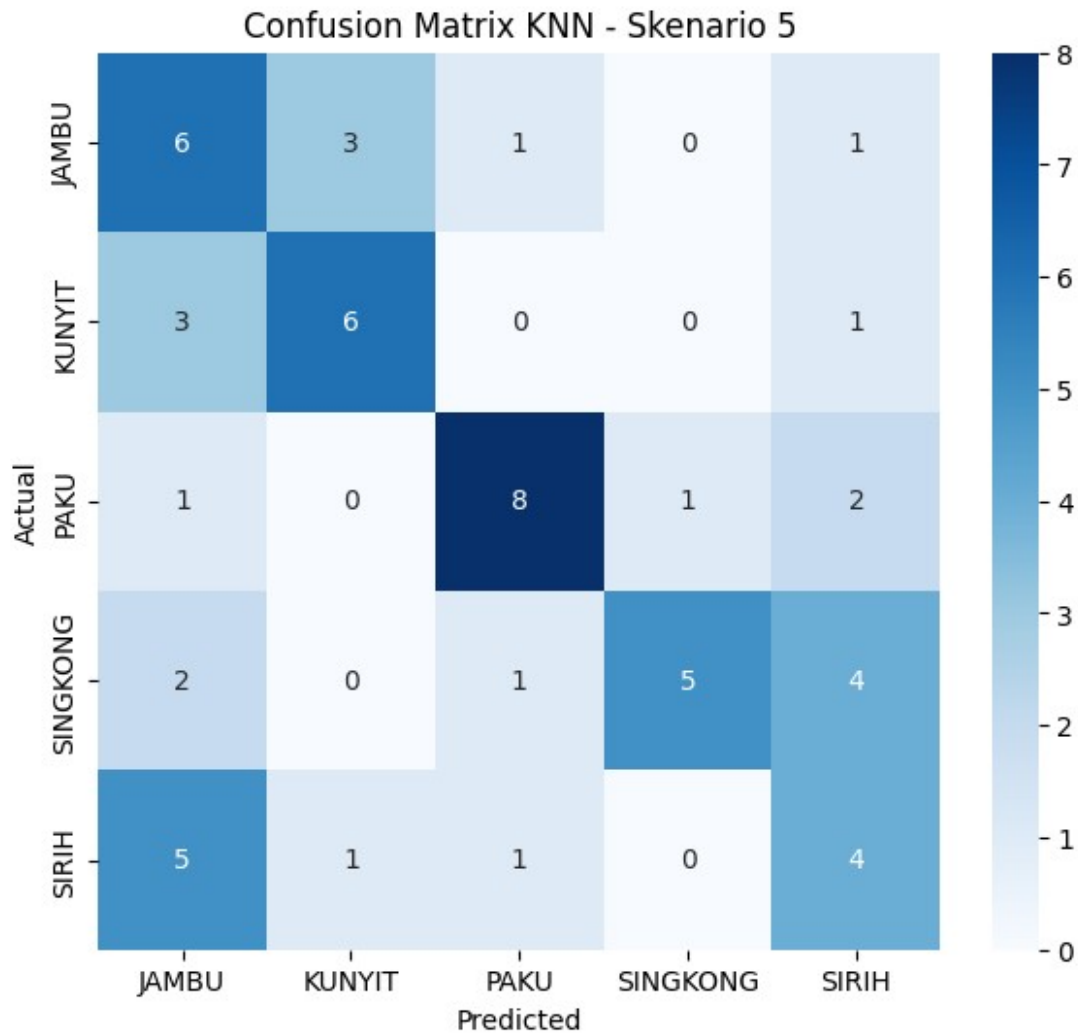
KNN

=====

Train Accuracy : 87.33%

Test Accuracy : 51.79%

	precision	recall	f1-score	support
JAMBU	0.35	0.55	0.43	11
KUNYIT	0.60	0.60	0.60	10
PAKU	0.73	0.67	0.70	12
SINGKONG	0.83	0.42	0.56	12
SIRIH	0.33	0.36	0.35	11
accuracy			0.52	56
macro avg	0.57	0.52	0.53	56
weighted avg	0.58	0.52	0.53	56



=====

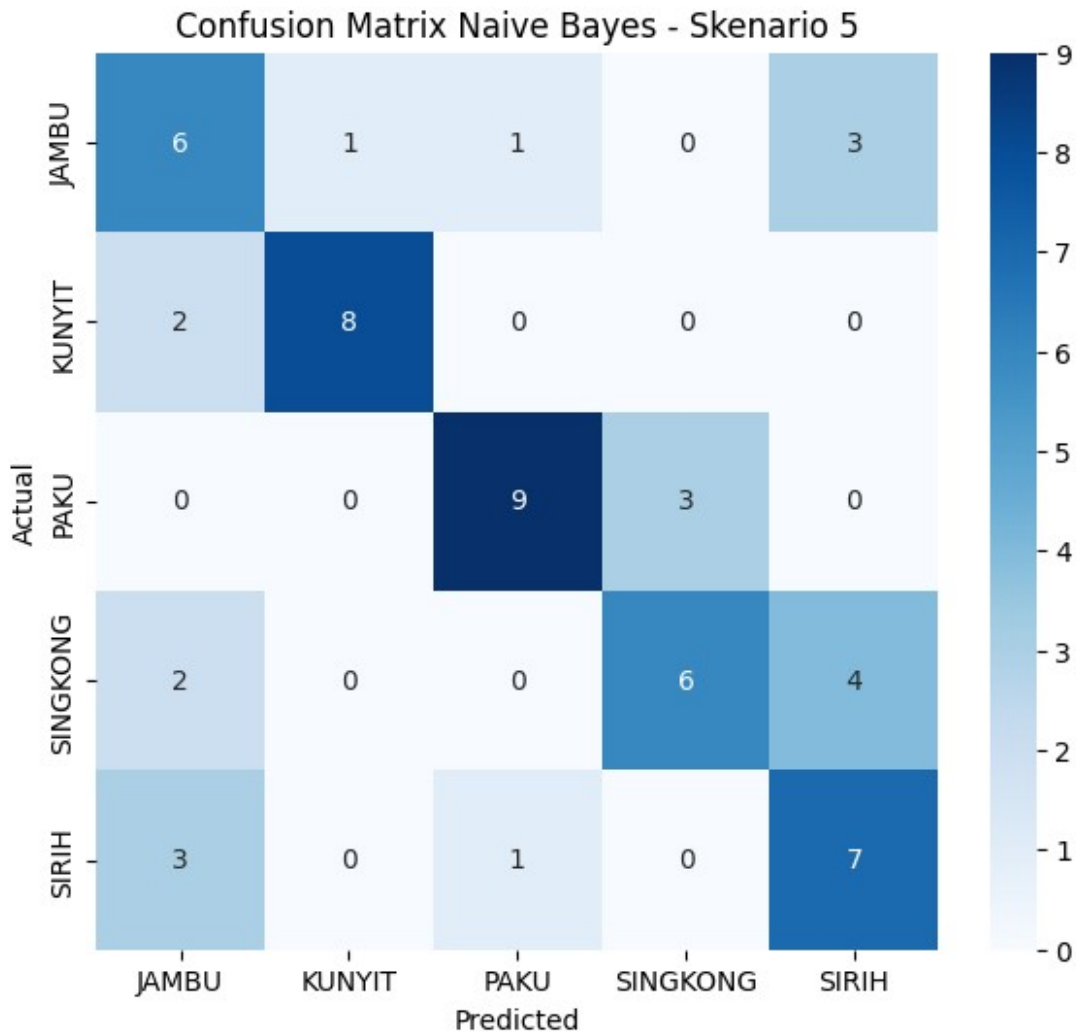
Naive Bayes

=====

Train Accuracy : 64.46%

Test Accuracy : 64.29%

	precision	recall	f1-score	support
JAMBU	0.46	0.55	0.50	11
KUNYIT	0.89	0.80	0.84	10
PAKU	0.82	0.75	0.78	12
SINGKONG	0.67	0.50	0.57	12
SIRIH	0.50	0.64	0.56	11
accuracy			0.64	56
macro avg	0.67	0.65	0.65	56
weighted avg	0.67	0.64	0.65	56



Model skenario 5 berhasil disimpan.

SKENARIO 6

```
scenario = 6

X_train_feat = np.array([
    extract_features_scenario(g, c, scenario)
    for g, c in zip(gray_images, contour_images)
])

X_test_feat = np.array([
    extract_features_scenario(g, c, scenario)
    for g, c in zip(gray_test, contour_test)
])
```

```
results_s6, scaler_s6 = train_evaluate_save(  
    X_train_feat, y_train_aug, X_test_feat, y_test, scenario, encoder  
)
```

```
all_results[scenario] = results_s6  
all_scalers[scenario] = scaler_s6
```

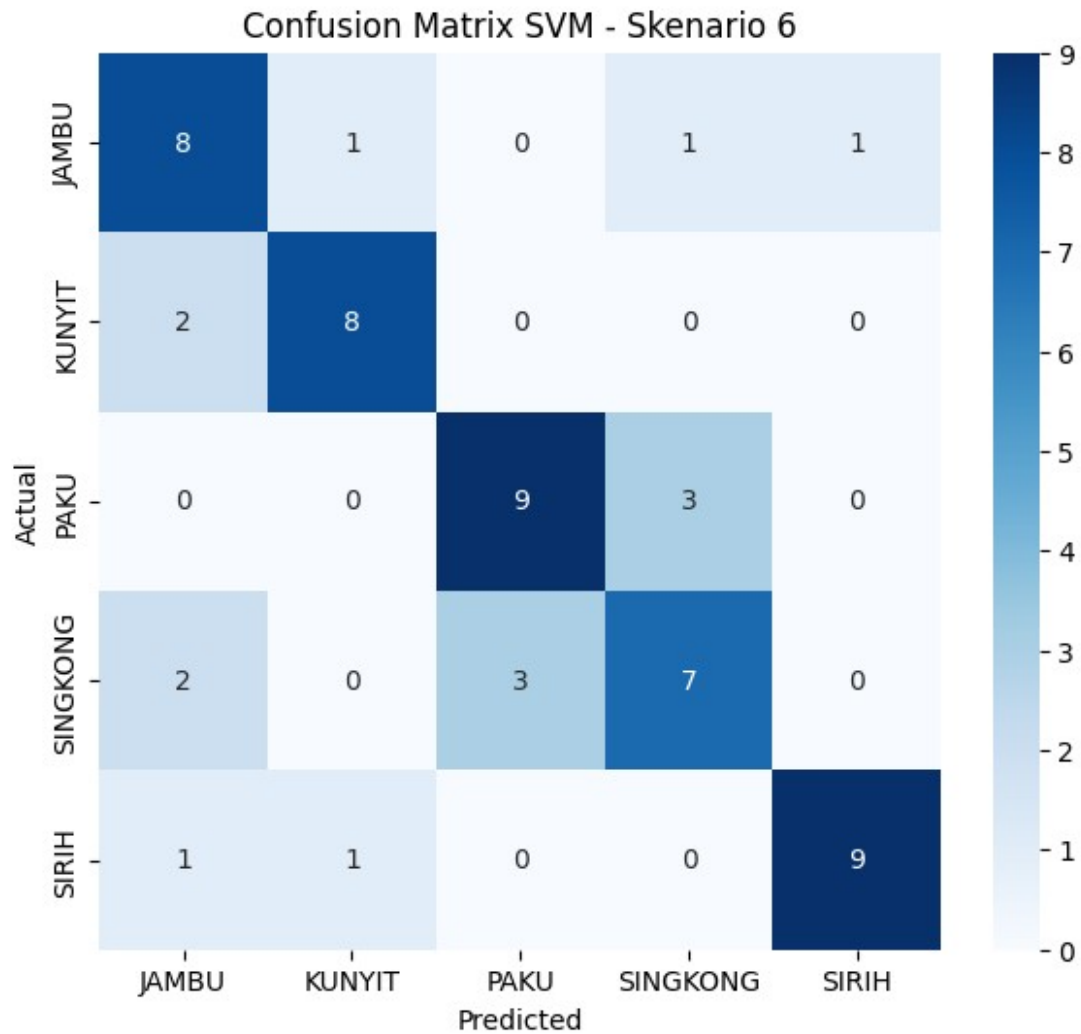
```
#####  
SKENARIO 6 – shape fitur train: (892, 53), test: (56, 53)  
#####
```

```
=====  
SVM  
=====
```

```
Train Accuracy : 94.62%
```

```
Test Accuracy  : 73.21%
```

	precision	recall	f1-score	support
JAMBU	0.62	0.73	0.67	11
KUNYIT	0.80	0.80	0.80	10
PAKU	0.75	0.75	0.75	12
SINGKONG	0.64	0.58	0.61	12
SIRIH	0.90	0.82	0.86	11
accuracy			0.73	56
macro avg	0.74	0.74	0.74	56
weighted avg	0.74	0.73	0.73	56



=====

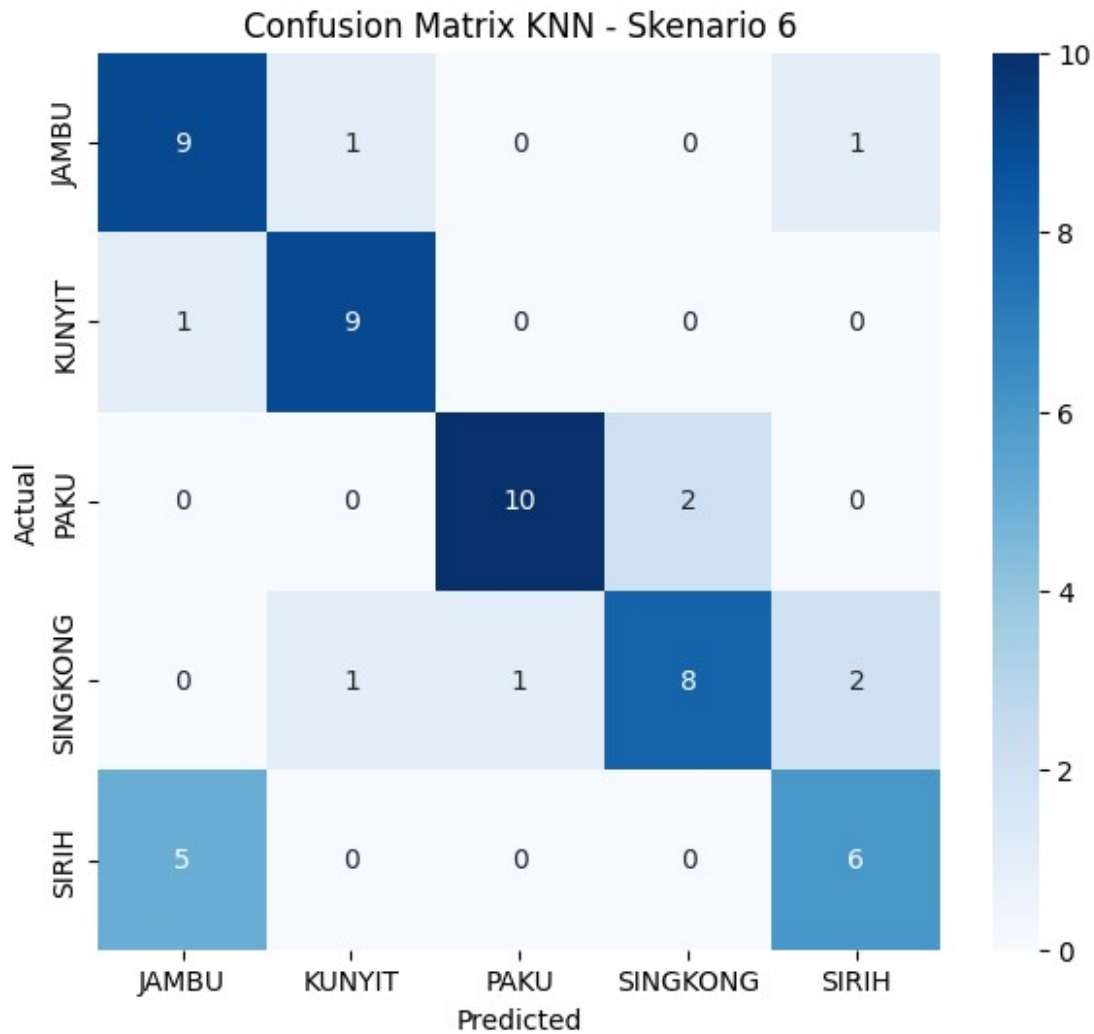
KNN

=====

Train Accuracy : 90.58%

Test Accuracy : 75.00%

	precision	recall	f1-score	support
JAMBU	0.60	0.82	0.69	11
KUNYIT	0.82	0.90	0.86	10
PAKU	0.91	0.83	0.87	12
SINGKONG	0.80	0.67	0.73	12
SIRIH	0.67	0.55	0.60	11
accuracy			0.75	56
macro avg	0.76	0.75	0.75	56
weighted avg	0.76	0.75	0.75	56



=====

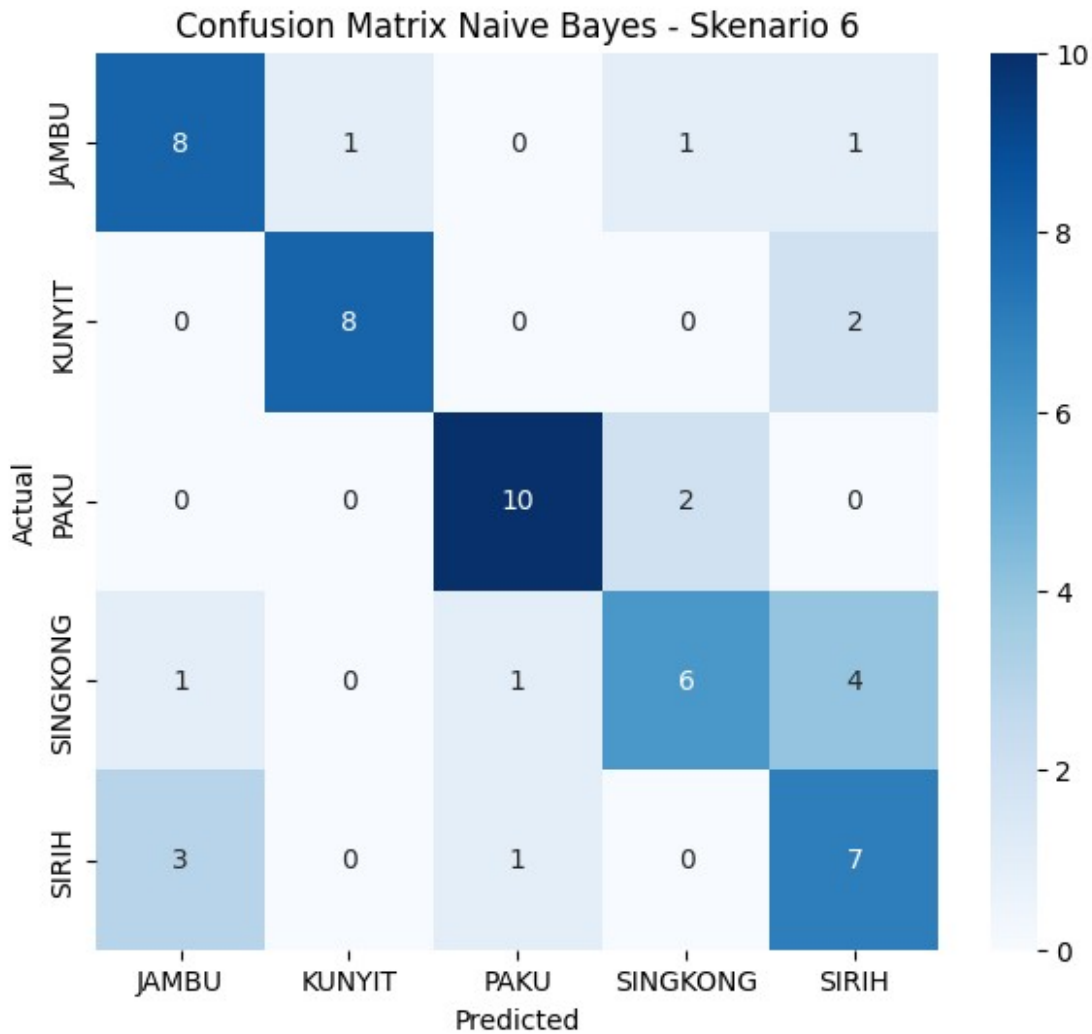
Naive Bayes

=====

Train Accuracy : 66.93%

Test Accuracy : 69.64%

	precision	recall	f1-score	support
JAMBU	0.67	0.73	0.70	11
KUNYIT	0.89	0.80	0.84	10
PAKU	0.83	0.83	0.83	12
SINGKONG	0.67	0.50	0.57	12
SIRIH	0.50	0.64	0.56	11
accuracy			0.70	56
macro avg	0.71	0.70	0.70	56
weighted avg	0.71	0.70	0.70	56



Model skenario 6 berhasil disimpan.

SKENARIO 7

```
scenario = 7

X_train_feat = np.array([
    extract_features_scenario(g, c, scenario)
    for g, c in zip(gray_images, contour_images)
])

X_test_feat = np.array([
    extract_features_scenario(g, c, scenario)
    for g, c in zip(gray_test, contour_test)
])
```

```
results_s7, scaler_s7 = train_evaluate_save(  
    X_train_feat, y_train_aug, X_test_feat, y_test, scenario, encoder  
)
```

```
all_results[scenario] = results_s7  
all_scalers[scenario] = scaler_s7
```

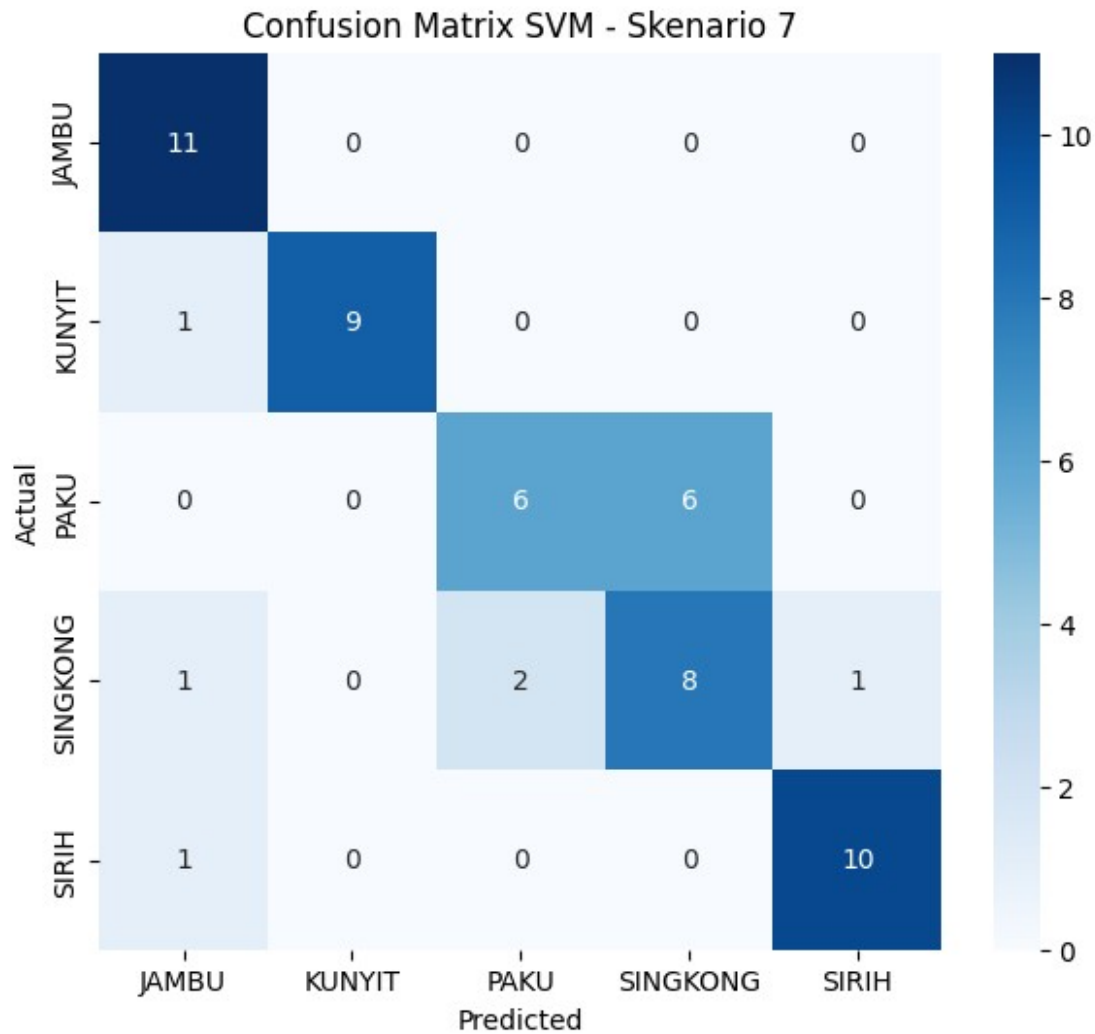
```
#####  
SKENARIO 7 – shape fitur train: (892, 9), test: (56, 9)  
#####
```

```
=====  
SVM  
=====
```

Train Accuracy : 86.77%

Test Accuracy : 78.57%

	precision	recall	f1-score	support
JAMBU	0.79	1.00	0.88	11
KUNYIT	1.00	0.90	0.95	10
PAKU	0.75	0.50	0.60	12
SINGKONG	0.57	0.67	0.62	12
SIRIH	0.91	0.91	0.91	11
accuracy			0.79	56
macro avg	0.80	0.80	0.79	56
weighted avg	0.79	0.79	0.78	56



=====

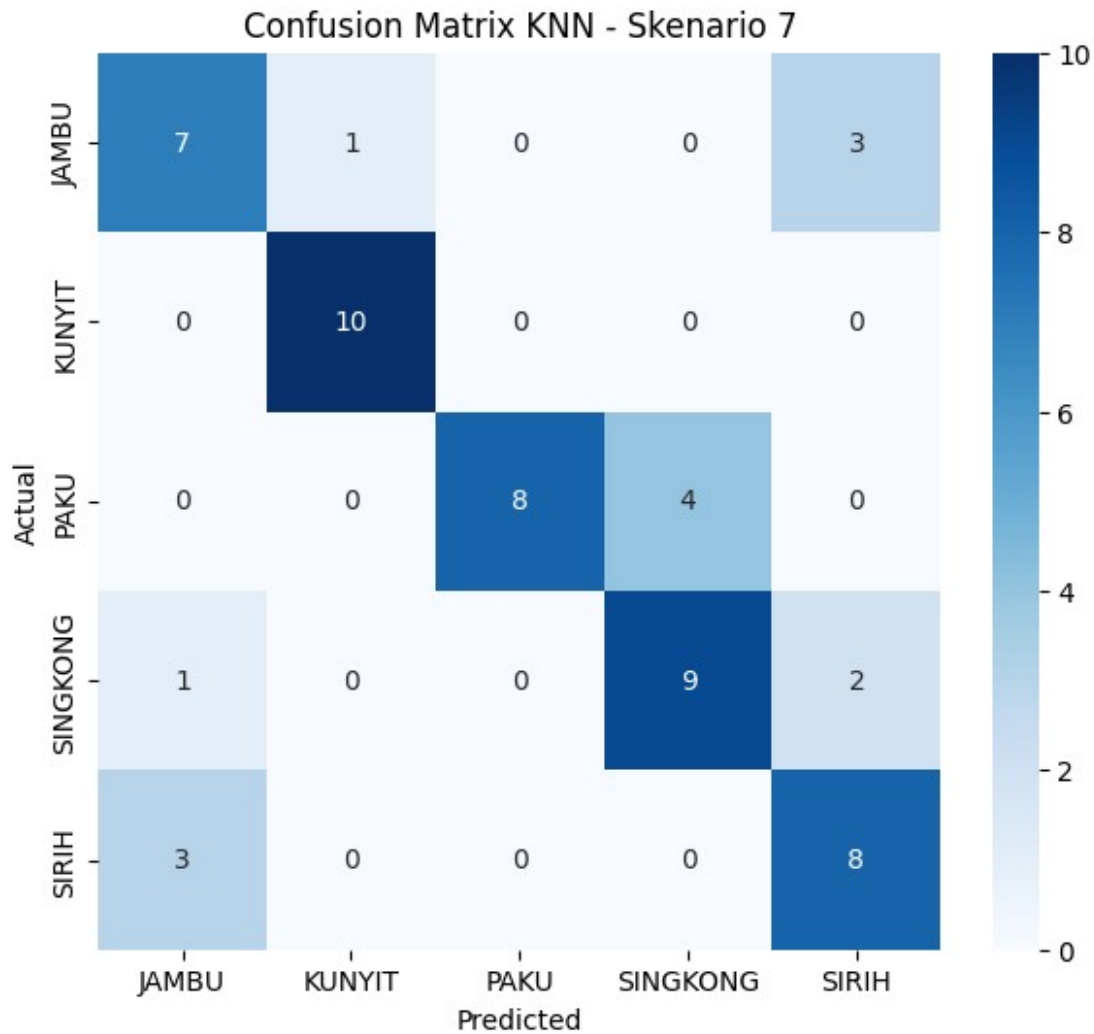
KNN

=====

Train Accuracy : 92.83%

Test Accuracy : 75.00%

	precision	recall	f1-score	support
JAMBU	0.64	0.64	0.64	11
KUNYIT	0.91	1.00	0.95	10
PAKU	1.00	0.67	0.80	12
SINGKONG	0.69	0.75	0.72	12
SIRIH	0.62	0.73	0.67	11
accuracy			0.75	56
macro avg	0.77	0.76	0.76	56
weighted avg	0.77	0.75	0.75	56



=====

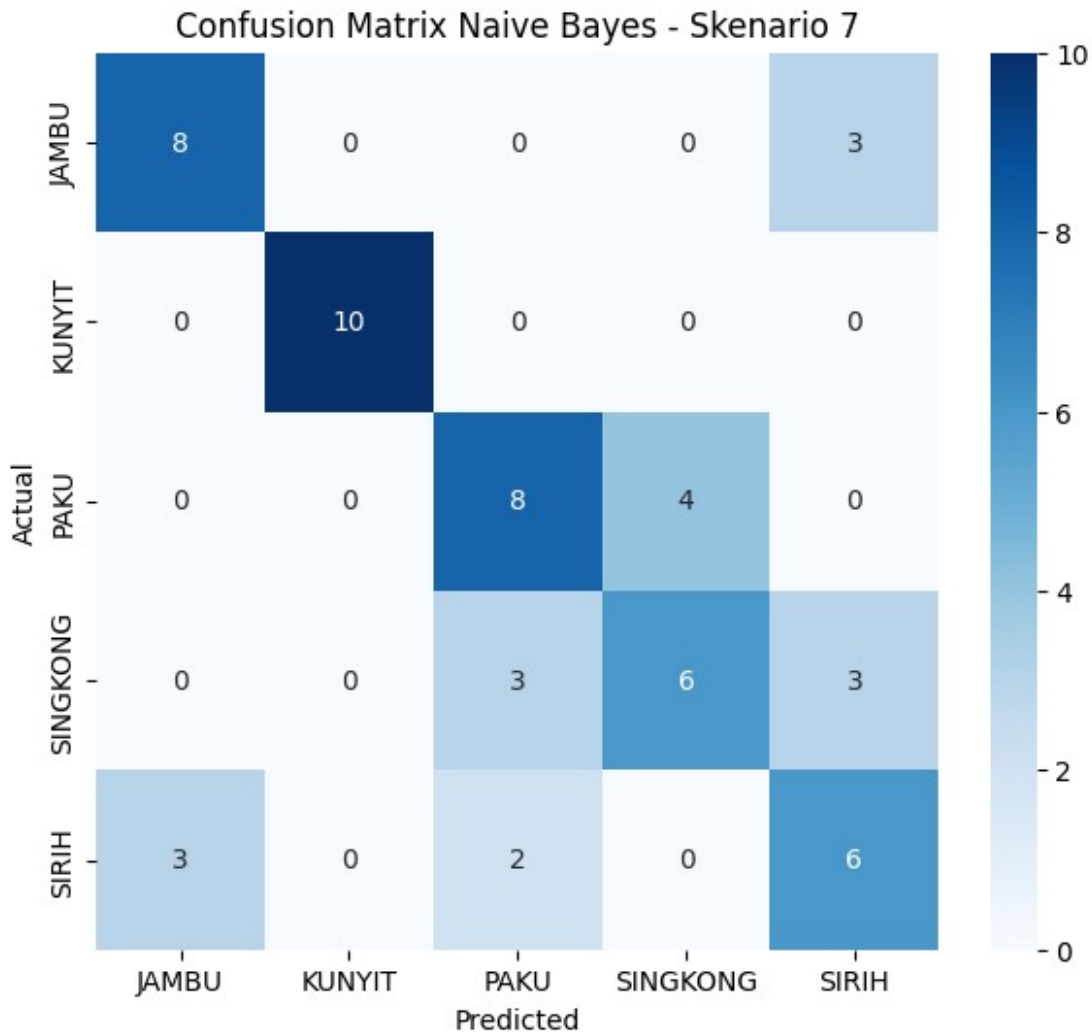
Naive Bayes

=====

Train Accuracy : 75.11%

Test Accuracy : 67.86%

	precision	recall	f1-score	support
JAMBU	0.73	0.73	0.73	11
KUNYIT	1.00	1.00	1.00	10
PAKU	0.62	0.67	0.64	12
SINGKONG	0.60	0.50	0.55	12
SIRIH	0.50	0.55	0.52	11
accuracy			0.68	56
macro avg	0.69	0.69	0.69	56
weighted avg	0.68	0.68	0.68	56



Model skenario 7 berhasil disimpan.

SKENARIO 8

```
scenario = 8

X_train_feat = np.array([
    extract_features_scenario(g, c, scenario)
    for g, c in zip(gray_images, contour_images)
])

X_test_feat = np.array([
    extract_features_scenario(g, c, scenario)
    for g, c in zip(gray_test, contour_test)
])
```

```
results_s8, scaler_s8 = train_evaluate_save(  
    X_train_feat, y_train_aug, X_test_feat, y_test, scenario, encoder  
)
```

```
all_results[scenario] = results_s8  
all_scalers[scenario] = scaler_s8
```

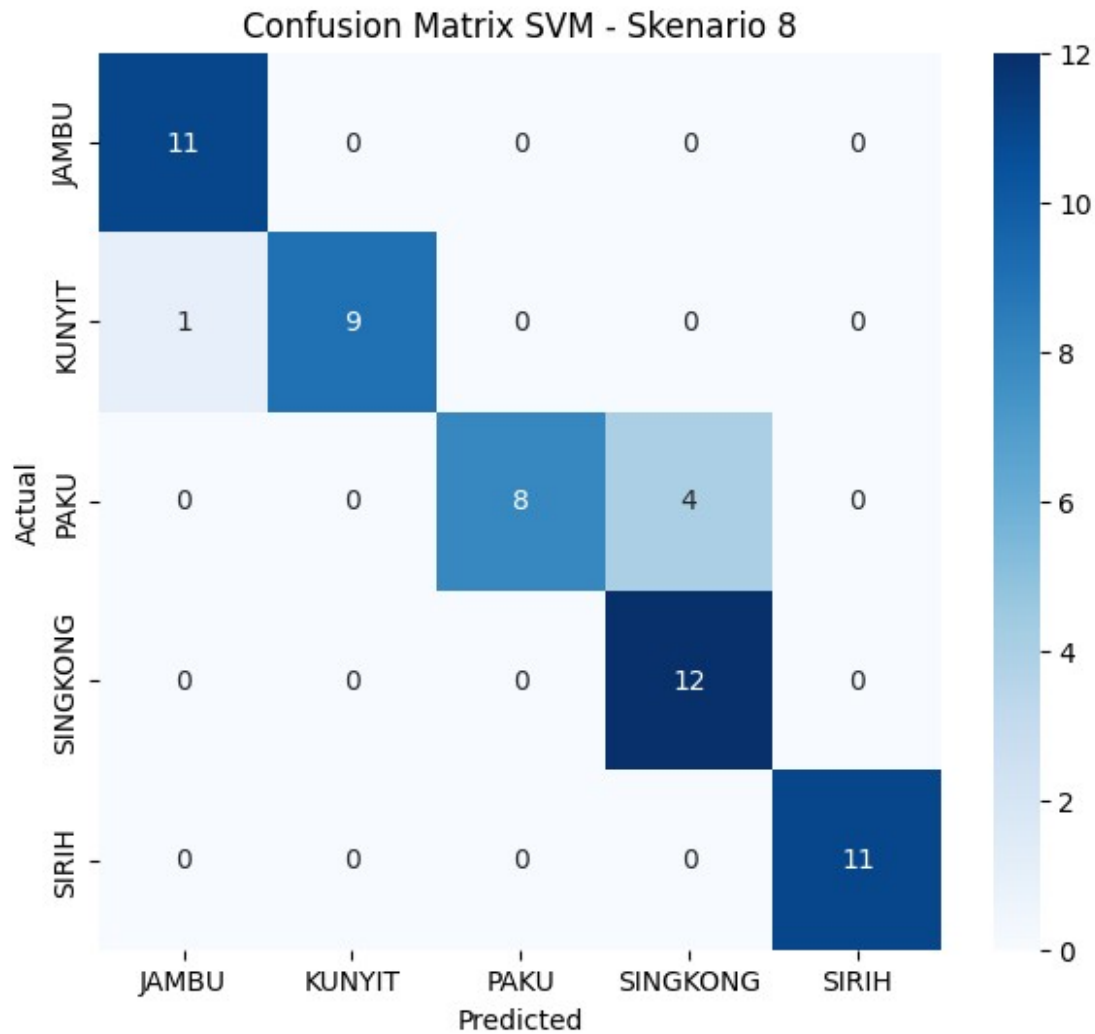
```
#####  
SKENARIO 8 – shape fitur train: (892, 18), test: (56, 18)  
#####
```

```
=====  
SVM  
=====
```

```
Train Accuracy : 91.48%
```

```
Test Accuracy  : 91.07%
```

	precision	recall	f1-score	support
JAMBU	0.92	1.00	0.96	11
KUNYIT	1.00	0.90	0.95	10
PAKU	1.00	0.67	0.80	12
SINGKONG	0.75	1.00	0.86	12
SIRIH	1.00	1.00	1.00	11
accuracy			0.91	56
macro avg	0.93	0.91	0.91	56
weighted avg	0.93	0.91	0.91	56



=====

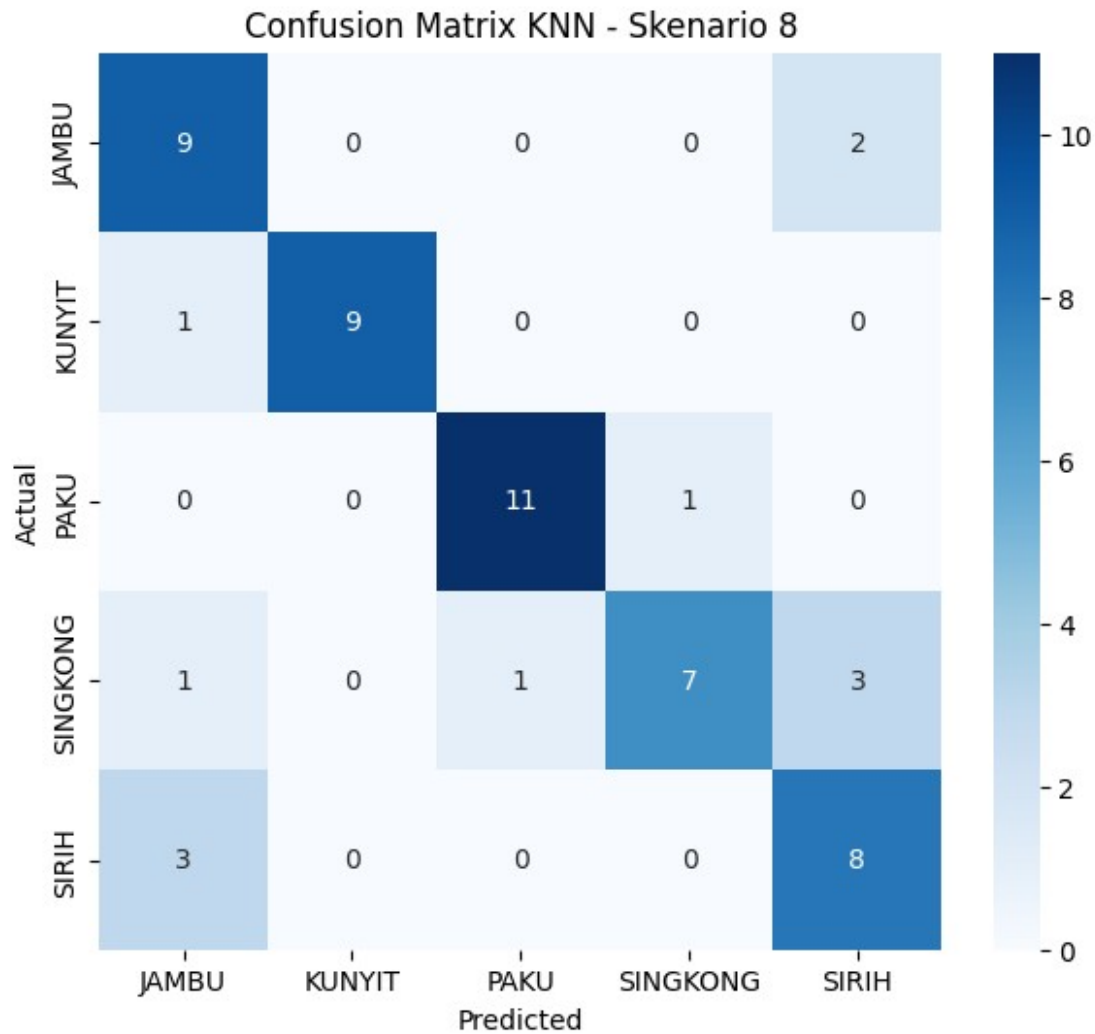
KNN

=====

Train Accuracy : 93.61%

Test Accuracy : 78.57%

	precision	recall	f1-score	support
JAMBU	0.64	0.82	0.72	11
KUNYIT	1.00	0.90	0.95	10
PAKU	0.92	0.92	0.92	12
SINGKONG	0.88	0.58	0.70	12
SIRIH	0.62	0.73	0.67	11
accuracy			0.79	56
macro avg	0.81	0.79	0.79	56
weighted avg	0.81	0.79	0.79	56



=====

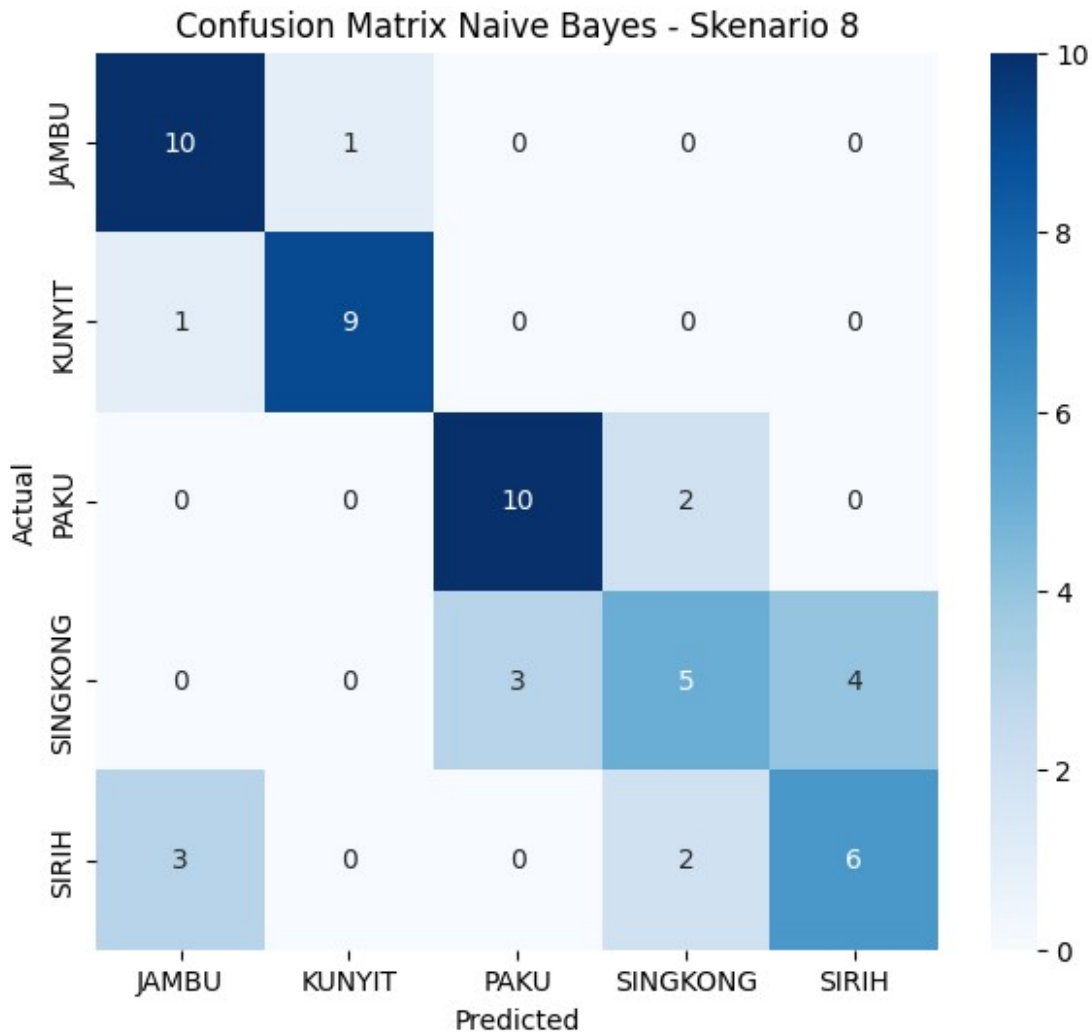
Naive Bayes

=====

Train Accuracy : 75.56%

Test Accuracy : 71.43%

	precision	recall	f1-score	support
JAMBU	0.71	0.91	0.80	11
KUNYIT	0.90	0.90	0.90	10
PAKU	0.77	0.83	0.80	12
SINGKONG	0.56	0.42	0.48	12
SIRIH	0.60	0.55	0.57	11
accuracy			0.71	56
macro avg	0.71	0.72	0.71	56
weighted avg	0.70	0.71	0.70	56



Model skenario 8 berhasil disimpan.

SKENARIO 9

```
scenario = 9

X_train_feat = np.array([
    extract_features_scenario(g, c, scenario)
    for g, c in zip(gray_images, contour_images)
])

X_test_feat = np.array([
    extract_features_scenario(g, c, scenario)
    for g, c in zip(gray_test, contour_test)
])
```

```
results_s9, scaler_s9 = train_evaluate_save(  
    X_train_feat, y_train_aug, X_test_feat, y_test, scenario, encoder  
)
```

```
all_results[scenario] = results_s9  
all_scalers[scenario] = scaler_s9
```

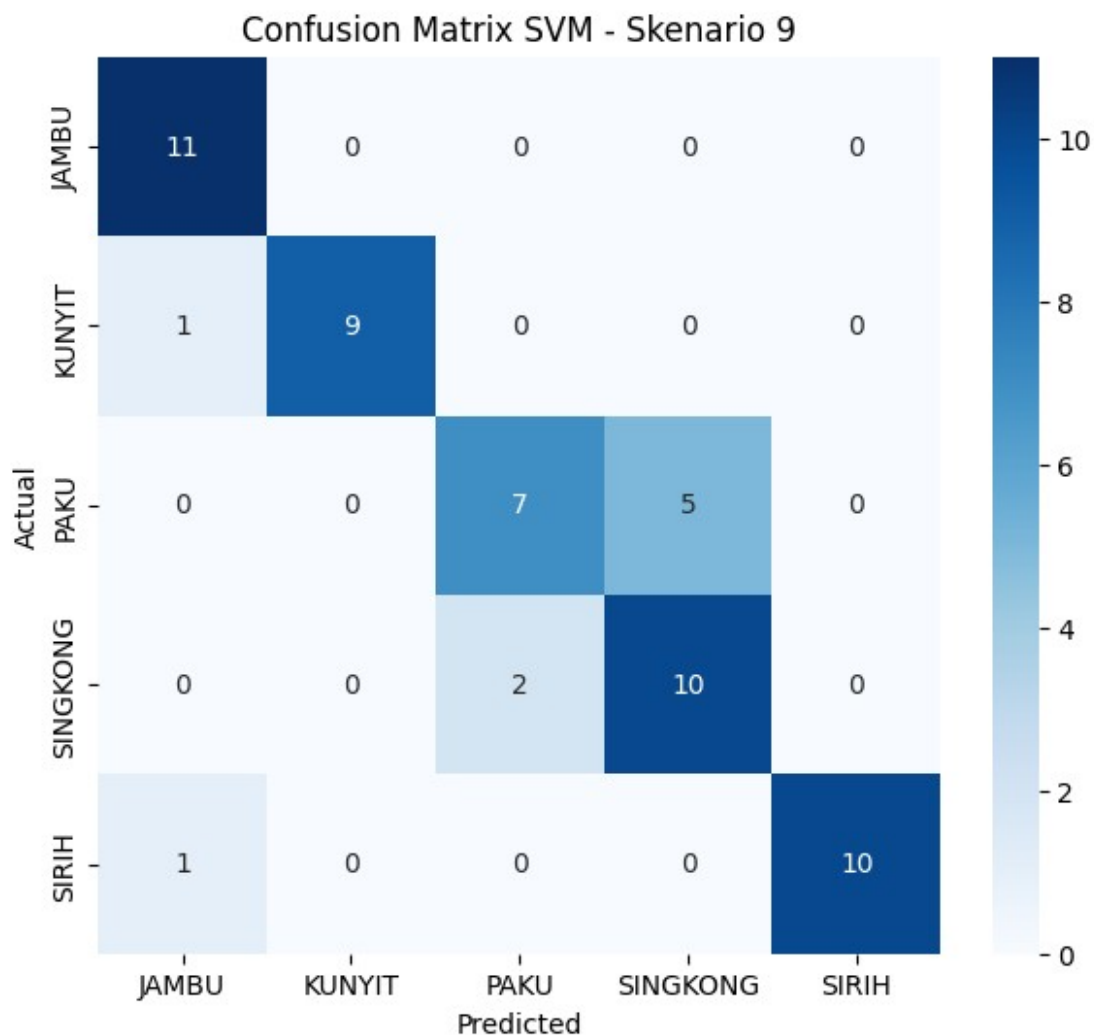
```
#####  
SKENARIO 9 – shape fitur train: (892, 13), test: (56, 13)  
#####
```

```
=====  
SVM  
=====
```

Train Accuracy : 89.57%

Test Accuracy : 83.93%

	precision	recall	f1-score	support
JAMBU	0.85	1.00	0.92	11
KUNYIT	1.00	0.90	0.95	10
PAKU	0.78	0.58	0.67	12
SINGKONG	0.67	0.83	0.74	12
SIRIH	1.00	0.91	0.95	11
accuracy			0.84	56
macro avg	0.86	0.85	0.84	56
weighted avg	0.85	0.84	0.84	56



=====

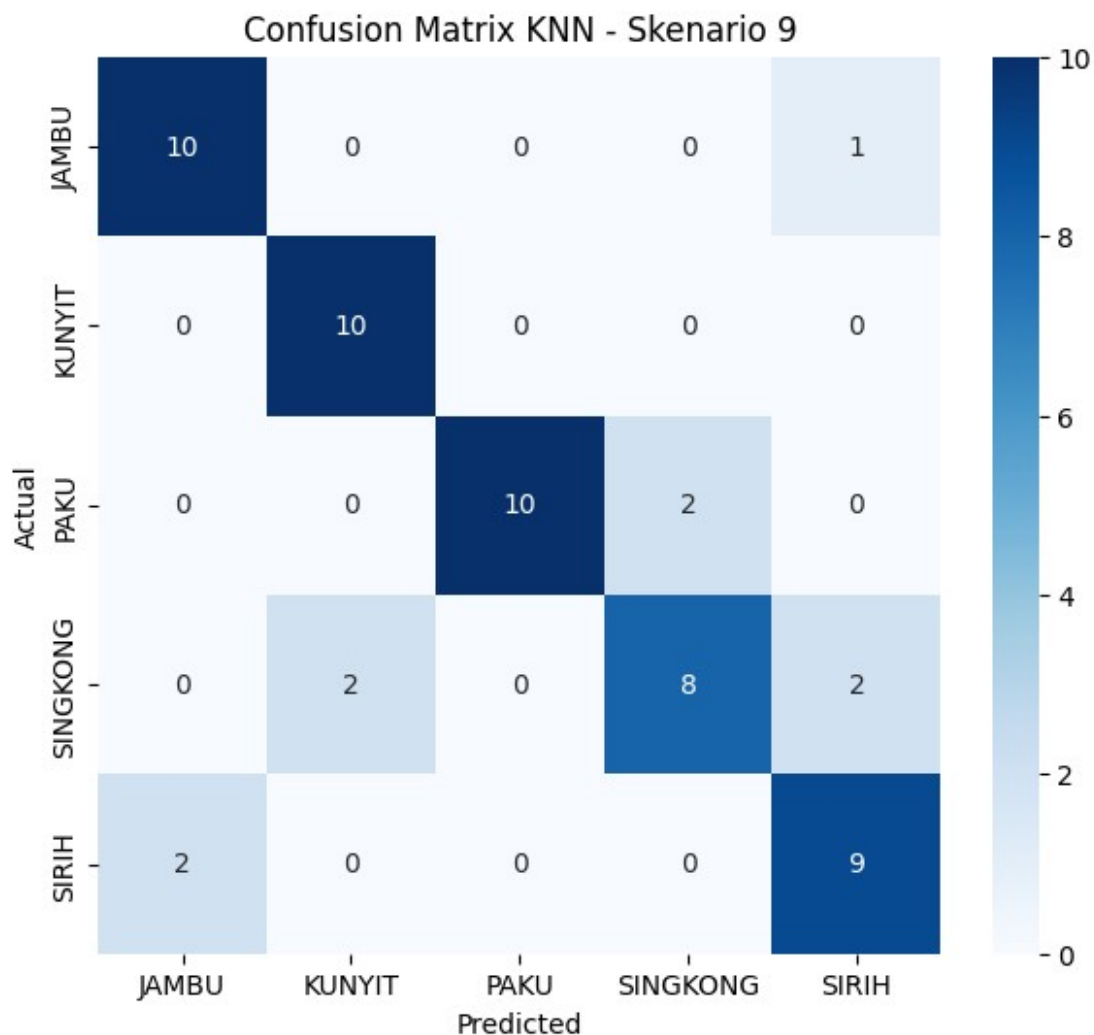
KNN

=====

Train Accuracy : 93.05%

Test Accuracy : 83.93%

	precision	recall	f1-score	support
JAMBU	0.83	0.91	0.87	11
KUNYIT	0.83	1.00	0.91	10
PAKU	1.00	0.83	0.91	12
SINGKONG	0.80	0.67	0.73	12
SIRIH	0.75	0.82	0.78	11
accuracy			0.84	56
macro avg	0.84	0.85	0.84	56
weighted avg	0.85	0.84	0.84	56



=====

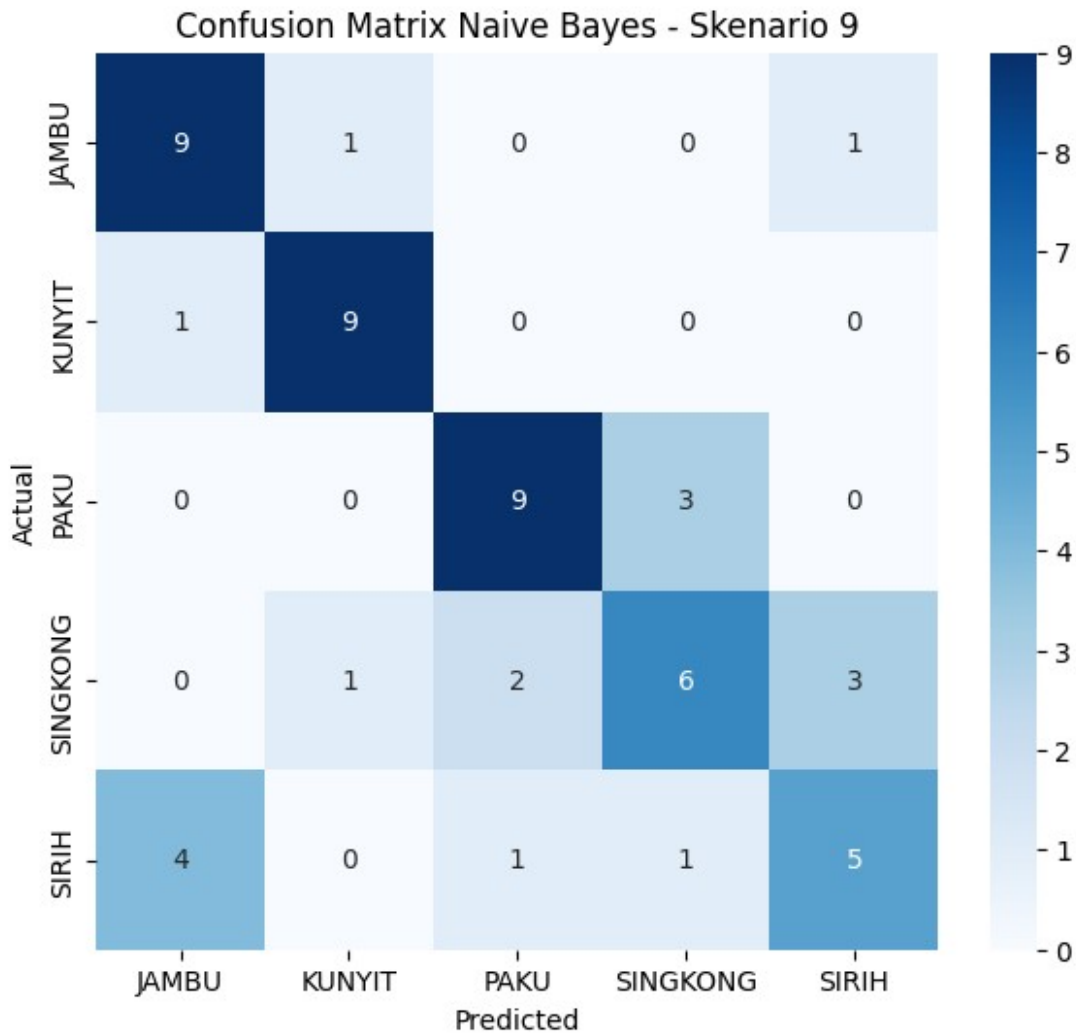
Naive Bayes

=====

Train Accuracy : 73.43%

Test Accuracy : 67.86%

	precision	recall	f1-score	support
JAMBU	0.64	0.82	0.72	11
KUNYIT	0.82	0.90	0.86	10
PAKU	0.75	0.75	0.75	12
SINGKONG	0.60	0.50	0.55	12
SIRIH	0.56	0.45	0.50	11
accuracy			0.68	56
macro avg	0.67	0.68	0.67	56
weighted avg	0.67	0.68	0.67	56



Model skenario 9 berhasil disimpan.

SKENARIO 10

```
scenario = 10

X_train_feat = np.array([
    extract_features_scenario(g, c, scenario)
    for g, c in zip(gray_images, contour_images)
])

X_test_feat = np.array([
    extract_features_scenario(g, c, scenario)
    for g, c in zip(gray_test, contour_test)
])
```

```
results_s10, scaler_s10 = train_evaluate_save(  
    X_train_feat, y_train_aug, X_test_feat, y_test, scenario, encoder  
)
```

```
all_results[scenario] = results_s10  
all_scalers[scenario] = scaler_s10
```

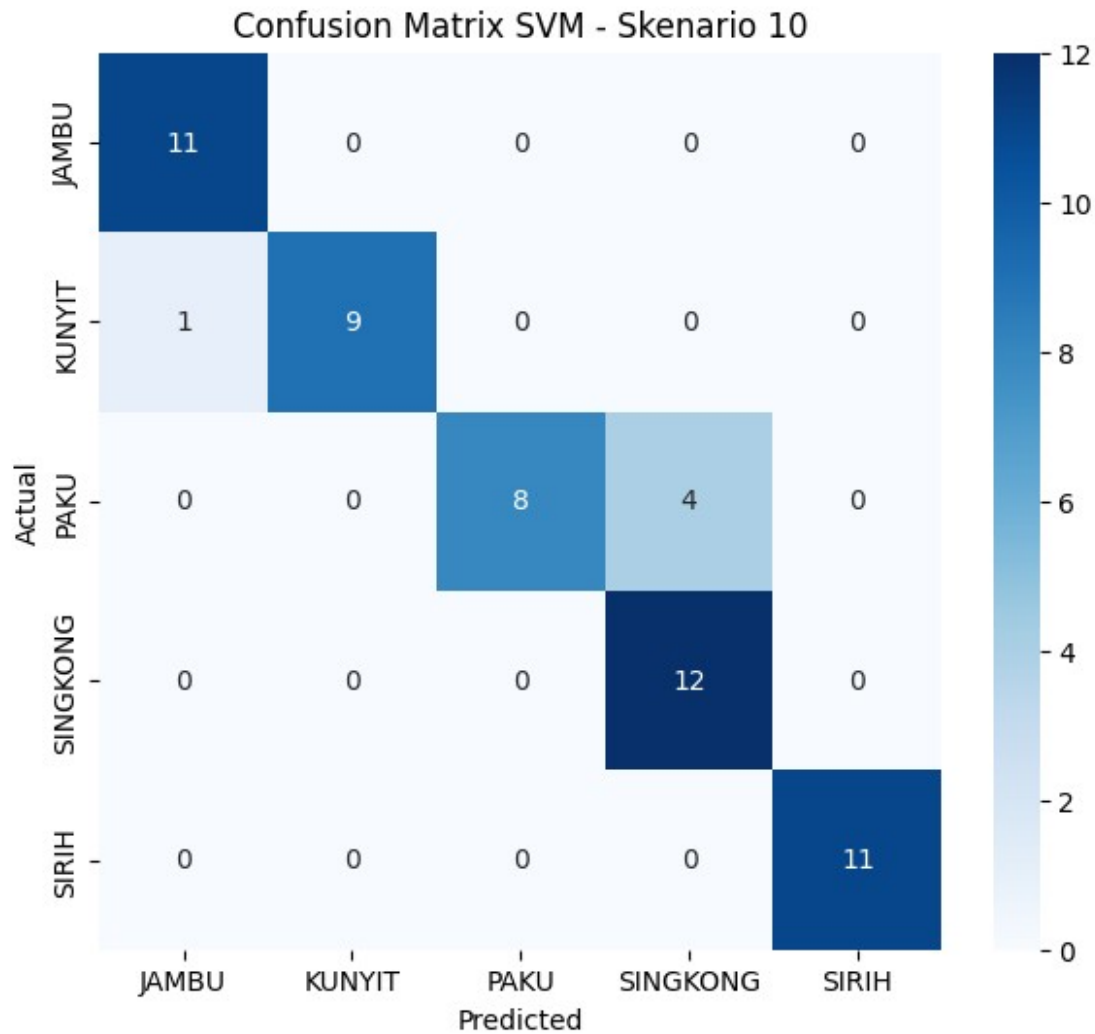
```
#####  
SKENARIO 10 – shape fitur train: (892, 22), test: (56, 22)  
#####
```

```
=====  
SVM  
=====
```

```
Train Accuracy : 91.82%
```

```
Test Accuracy  : 91.07%
```

	precision	recall	f1-score	support
JAMBU	0.92	1.00	0.96	11
KUNYIT	1.00	0.90	0.95	10
PAKU	1.00	0.67	0.80	12
SINGKONG	0.75	1.00	0.86	12
SIRIH	1.00	1.00	1.00	11
accuracy			0.91	56
macro avg	0.93	0.91	0.91	56
weighted avg	0.93	0.91	0.91	56



=====

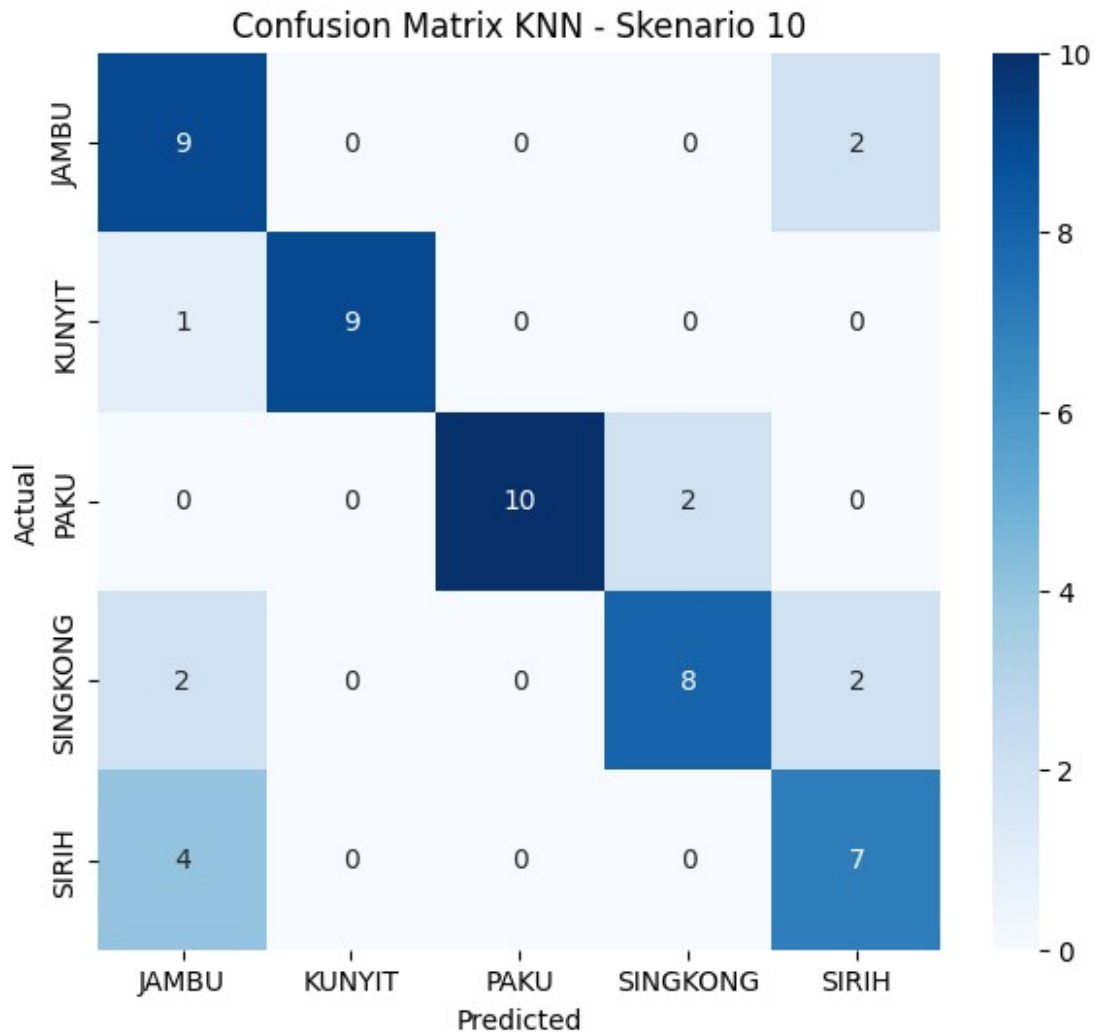
KNN

=====

Train Accuracy : 93.61%

Test Accuracy : 76.79%

	precision	recall	f1-score	support
JAMBU	0.56	0.82	0.67	11
KUNYIT	1.00	0.90	0.95	10
PAKU	1.00	0.83	0.91	12
SINGKONG	0.80	0.67	0.73	12
SIRIH	0.64	0.64	0.64	11
accuracy			0.77	56
macro avg	0.80	0.77	0.78	56
weighted avg	0.80	0.77	0.78	56



=====

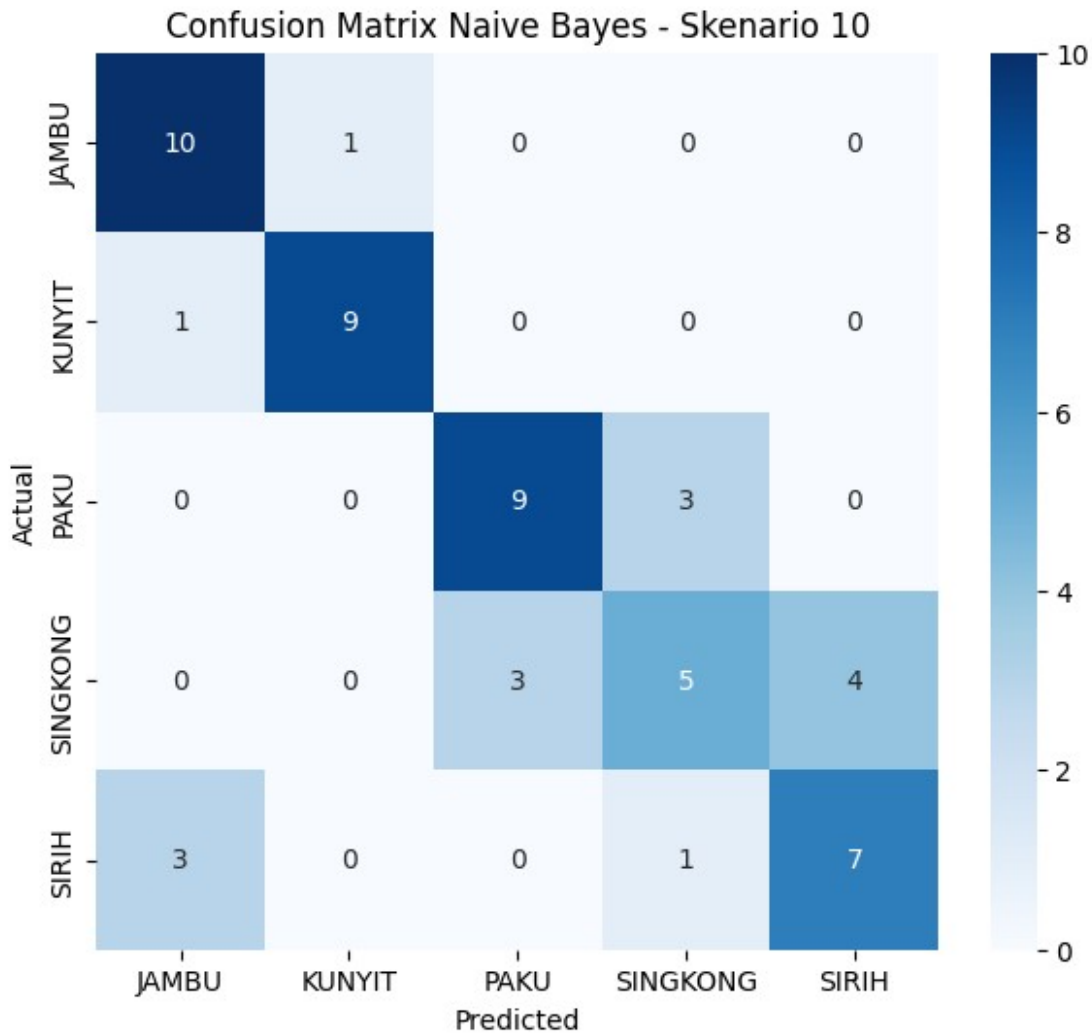
Naive Bayes

=====

Train Accuracy : 73.88%

Test Accuracy : 71.43%

	precision	recall	f1-score	support
JAMBU	0.71	0.91	0.80	11
KUNYIT	0.90	0.90	0.90	10
PAKU	0.75	0.75	0.75	12
SINGKONG	0.56	0.42	0.48	12
SIRIH	0.64	0.64	0.64	11
accuracy			0.71	56
macro avg	0.71	0.72	0.71	56
weighted avg	0.71	0.71	0.71	56



Model skenario 10 berhasil disimpan.

5. HASIL DAN PEMBAHASAN

5.1 Grafik Pelatihan (*Training Progress*)

Ketiga algoritma yang digunakan (SVM, KNN, *Naive Bayes*) merupakan metode *machine learning* klasik yang dilatih secara langsung (*closed-form/non-iterative* untuk KNN dan *Naive Bayes*, optimisasi *konveks* untuk SVM), **bukan** dilatih secara iteratif per *epoch* seperti pada jaringan saraf tiruan/*deep learning*. Oleh karena itu, grafik *Loss vs Epoch* dan *Accuracy vs Epoch* yang lazim disajikan pada proyek berbasis CNN **tidak relevan** untuk proyek ini. Sebagai gantinya, indikator *overfitting/underfitting/good fit* yang setara diperoleh dengan membandingkan **akurasi data latih vs akurasi data uji** pada setiap skenario dan model (ditampilkan pada *output train_evaluate_save* di Bagian 4.2), serta ringkasan perbandingan menyeluruh pada *bar chart* di bawah ini.

Panduan interpretasi (isi dengan hasil aktual setelah seluruh skenario dijalankan):

- Jika akurasi *train* jauh lebih tinggi daripada akurasi *test* (selisih besar, >15–20%) pada suatu kombinasi skenario+model, ini mengindikasikan **overfitting** — model terlalu menyesuaikan diri pada data latih (termasuk efek augmentasi) dan kurang mampu generalisasi ke data uji yang belum pernah dilihat.
- Jika akurasi *train* dan *test* keduanya rendah dan berdekatan, ini mengindikasikan **underfitting** — kombinasi fitur pada skenario tersebut tidak cukup informatif untuk memisahkan kelima kelas daun (kemungkinan besar terjadi pada Skenario 3 dan 7 yang hanya menggunakan satu jenis fitur bentuk tanpa tekstur sama sekali).
- Kombinasi skenario+model dengan akurasi *test* tertinggi **dan** selisih *train-test* yang wajar (good fit) menjadi kandidat model terbaik yang dibahas lebih lanjut pada Bagian 5.2 dan Bagian 6.

[Setelah menjalankan seluruh skenario, tuliskan di sini: skenario dan model mana yang menunjukkan *overfitting*/*underfitting*/*good fit*, beserta nilai akurasi *train* dan *test* spesifiknya, mengacu pada *output* di atas dan tabel ringkasan di bawah.]

5.2 Evaluasi Performa Model

Tabel ringkasan akurasi *train* dan *test* untuk seluruh kombinasi sepuluh skenario × tiga model, beserta visualisasi *bar chart* perbandingan, ditampilkan pada cell di bawah ini. *Classification report* (presisi, *recall*, F1-score per kelas) dan *confusion matrix* untuk masing-masing skenario telah ditampilkan secara rinci pada *output* setiap skenario di Bagian 4.2.

Ringkasan Hasil Semua Skenario

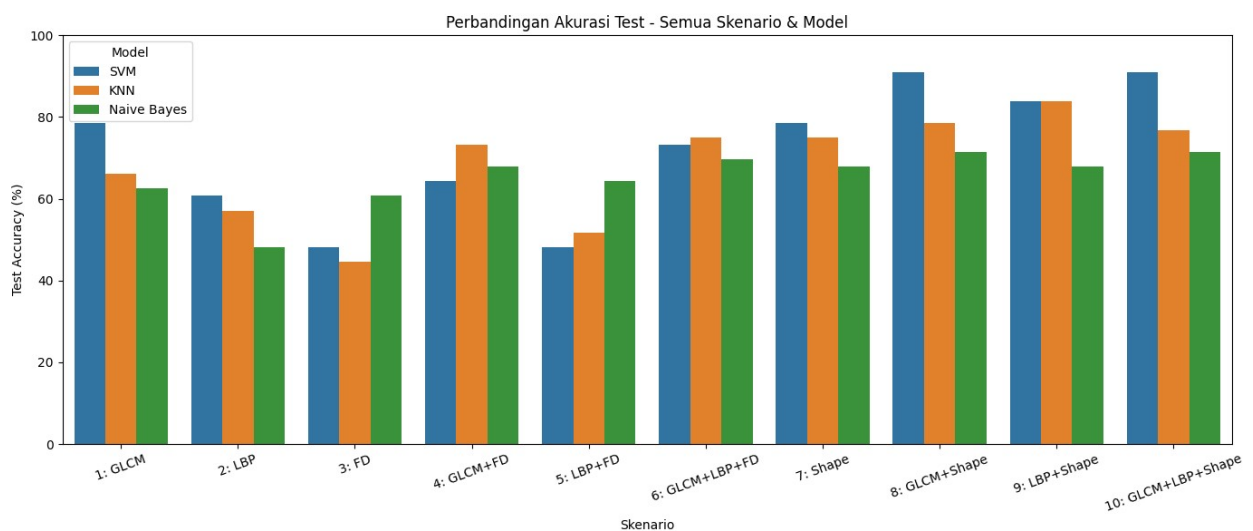
```
summary_rows = []

for scenario_num, results in all_results.items():
    for model_name, metrics in results.items():
        summary_rows.append({
            "Skenario": SCENARIO_LABELS[scenario_num],
            "Model": model_name,
            "Train Accuracy (%)": round(metrics["train_acc"] * 100,
2),
            "Test Accuracy (%)": round(metrics["test_acc"] * 100, 2)
        })

summary_df = pd.DataFrame(summary_rows)
print(summary_df.to_string(index=False))

plt.figure(figsize=(14, 6))
sns.barplot(data=summary_df, x="Skenario", y="Test Accuracy (%)",
hue="Model")
plt.title("Perbandingan Akurasi Test - Semua Skenario & Model")
plt.ylim(0, 100)
plt.xticks(rotation=20)
plt.tight_layout()
plt.show()
```

Skenario	Model	Train Accuracy (%)	Test Accuracy (%)
1: GLCM	SVM	69.73	78.57
1: GLCM	KNN	84.75	66.07
1: GLCM	Naive Bayes	47.76	62.50
2: LBP	SVM	61.77	60.71
2: LBP	KNN	84.08	57.14
2: LBP	Naive Bayes	45.85	48.21
3: FD	SVM	83.97	48.21
3: FD	KNN	80.04	44.64
3: FD	Naive Bayes	63.79	60.71
4: GLCM+FD	SVM	93.95	64.29
4: GLCM+FD	KNN	88.34	73.21
4: GLCM+FD	Naive Bayes	67.38	67.86
5: LBP+FD	SVM	87.22	48.21
5: LBP+FD	KNN	87.33	51.79
5: LBP+FD	Naive Bayes	64.46	64.29
6: GLCM+LBP+FD	SVM	94.62	73.21
6: GLCM+LBP+FD	KNN	90.58	75.00
6: GLCM+LBP+FD	Naive Bayes	66.93	69.64
7: Shape	SVM	86.77	78.57
7: Shape	KNN	92.83	75.00
7: Shape	Naive Bayes	75.11	67.86
8: GLCM+Shape	SVM	91.48	91.07
8: GLCM+Shape	KNN	93.61	78.57
8: GLCM+Shape	Naive Bayes	75.56	71.43
9: LBP+Shape	SVM	89.57	83.93
9: LBP+Shape	KNN	93.05	83.93
9: LBP+Shape	Naive Bayes	73.43	67.86
10: GLCM+LBP+Shape	SVM	91.82	91.07
10: GLCM+LBP+Shape	KNN	93.61	76.79
10: GLCM+LBP+Shape	Naive Bayes	73.88	71.43



Akurasi Keseluruhan (Skenario & Model Terbaik): [Isi dengan skenario dan model yang mencapai akurasi *test* tertinggi pada tabel ringkasan di atas, beserta nilai akurasinya, mis. "Skenario X dengan model Y mencapai akurasi test tertinggi sebesar Z%".]

Analisis *Confusion Matrix*: Berdasarkan *confusion matrix* skenario dan model terbaik (lihat *output* pada Bagian 4.2), identifikasi pasangan kelas yang paling sering tertukar (nilai *off-diagonal* terbesar pada matriks). Beberapa kemungkinan penyebab kesalahan klasifikasi antar kelas daun herbal yang perlu diperiksa:

- **Kemiripan tekstur permukaan** — daun-daun dengan permukaan licin dan mengkilap (mis. Jambu dan Sirih) dapat menghasilkan profil GLCM/LBP yang serupa meskipun bentuk keseluruhannya berbeda.
- **Kemiripan bentuk garis besar** — daun berbentuk oval/lonjong (mis. Jambu dan Kunyit) dapat menghasilkan nilai *Fourier Descriptor* frekuensi rendah yang berdekatan, karena FD menangkap garis besar bentuk dan kurang sensitif terhadap detail lokal seperti lekukan pangkal daun.
- **Variasi kualitas segmentasi** — citra dengan kontur yang kurang sempurna (akibat variasi pencahayaan/latar belakang pada akuisisi data, lihat Bagian 2.1) dapat menghasilkan fitur bentuk yang kurang representatif untuk sampel tersebut.

[Lengkapi analisis di atas dengan pasangan kelas spesifik yang paling sering tertukar berdasarkan *confusion matrix* aktual pada skenario terbaik, dan kaitkan dengan salah satu atau beberapa penyebab di atas, atau penyebab lain yang teramati.]

6. KESIMPULAN DAN SARAN

6.1 Kesimpulan

1. Metode *hybrid feature extraction* yang mengombinasikan fitur tekstur (GLCM, LBP) dan fitur bentuk (*Fourier Descriptor* dan/atau deskriptor geometris) berhasil diimplementasikan untuk klasifikasi lima jenis daun herbal (Jambu, Kunyit, Paku, Singkong, Sirih) menggunakan algoritma *machine learning* ringan (SVM, KNN, *Naive Bayes*), dengan akurasi akhir terbaik sebesar **[isi % akurasi terbaik]** pada Skenario **[isi nomor skenario]** menggunakan model **[isi nama model]**.
2. Berdasarkan perbandingan sepuluh skenario pada Bagian 5, kombinasi fitur yang paling berkontribusi terhadap performa klasifikasi adalah **[isi: GLCM/LBP/FD/Shape atau kombinasinya]**, yang mengonfirmasi/tidak mengonfirmasi (*pilih salah satu setelah melihat hasil*) hipotesis *hybrid feature extraction* bahwa kombinasi tekstur dan bentuk memberikan performa lebih baik dibandingkan penggunaan satu jenis fitur saja sebagaimana pada penelitian-penelitian Tabel SOTA (Bagian 1.1).
3. Pendekatan *hybrid handcrafted feature + machine learning* ringan pada proyek ini terbukti dapat dijalankan tanpa kebutuhan GPU atau dataset berskala ribuan citra, menjadikannya alternatif yang lebih efisien dibandingkan pendekatan *deep learning* (CNN/MobileNetV2) pada Tabel SOTA, meski [isi: dengan/tanpa] trade-off akurasi yang signifikan — bandingkan akurasi terbaik proyek ini dengan akurasi CNN (98%) dan MobileNetV2 (92%) pada Tabel SOTA.

6.2 Saran / Pengembangan ke Depan

1. **Kombinasi kedua representasi bentuk sekaligus:** Sepuluh skenario pada proyek ini menguji FD dan deskriptor geometris secara terpisah, namun belum menguji kombinasi keduanya secara bersamaan (mis. GLCM + LBP + FD + *Shape*). Penelitian selanjutnya dapat menguji kombinasi penuh ini untuk melihat apakah kedua representasi bentuk saling melengkapi atau justru redundan.
2. **Validasi yang lebih robust:** Evaluasi saat ini menggunakan satu kali pembagian *train-test split* (80:20). *K-fold cross-validation* dapat memberikan estimasi performa yang lebih stabil dan mengurangi ketergantungan hasil pada satu pembagian data tertentu secara acak.
3. **Perluasan dan penyeimbangan dataset:** Menambah jumlah sampel per kelas serta memperbanyak variasi kondisi akuisisi (musim, usia daun, kondisi kesehatan daun) dapat meningkatkan kemampuan generalisasi model terhadap kondisi dunia nyata yang lebih beragam.
4. **Eksplorasi *model ensemble* atau *tuning* hiperparameter lebih lanjut:** Mis. GridSearchCV untuk pencarian kombinasi *C/kernel* SVM yang optimal, atau *k* optimal untuk KNN, daripada menggunakan nilai *default* tunggal.
5. **Deployment:** Skenario dan model terbaik dapat dikembangkan lebih lanjut menjadi aplikasi *mobile* atau *web* sederhana untuk identifikasi daun herbal secara *real-time*, memanfaatkan fakta bahwa pipeline ini tidak memerlukan GPU sehingga dapat dijalankan pada perangkat dengan spesifikasi terbatas — fungsi `predict_best_model` pada Lampiran dapat menjadi titik awal untuk *deployment* tersebut.

DAFTAR PUSTAKA

Abdiel dkk. (2026). *Klasifikasi 10 Jenis Daun Tanaman Herbal Indonesia Menggunakan CNN dengan Arsitektur MobileNetV2 Berbasis Web*. [Lengkapi dengan nama jurnal/konferensi, volume, dan halaman.]

Ahonen, T., Hadid, A., & Pietikäinen, M. (2006). Face Description with Local Binary Patterns: Application to Face Recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 28(12), 2037–2041. <https://doi.org/10.1109/TPAMI.2006.244>

Azzahra dkk. (2023). *Klasifikasi Daun Herbal Menggunakan Metode CNN dan Naïve Bayes dengan Fitur GLCM*. [Lengkapi dengan nama jurnal/konferensi, volume, dan halaman.]

Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.

Bradski, G., & Kaehler, A. (2008). *Learning OpenCV: Computer Vision with the OpenCV Library*. O'Reilly Media.

Burger, W., & Burge, M. J. (2022). *Digital Image Processing: An Algorithmic Introduction Using Java* (3rd ed.). Springer. <https://doi.org/10.1007/978-3-662-65283-2>

Cooley, J. W., & Tukey, J. W. (1965). An Algorithm for the Machine Calculation of Complex Fourier Series. *Mathematics of Computation*, 19(90), 297–301. <https://doi.org/10.2307/2003354>

- Cortes, C., & Vapnik, V. (1995). Support-Vector Networks. *Machine Learning*, 20(3), 273–297. <https://doi.org/10.1007/BF00994018>
- Cover, T. M., & Hart, P. E. (1967). Nearest Neighbor Pattern Classification. *IEEE Transactions on Information Theory*, 13(1), 21–27. <https://doi.org/10.1109/TIT.1967.1053964>
- Furqan dkk. (2020). *Klasifikasi Daun Bugenvil Menggunakan Gray Level Co-Occurrence Matrix dan K-Nearest Neighbor*. [Lengkapi dengan nama jurnal/konferensi, volume, dan halaman.]
- Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow* (3rd ed.). O'Reilly Media.
- Gonzalez, R. C., & Woods, R. E. (2018). *Digital Image Processing* (4th ed.). Pearson.
- Hall-Beyer, M. (2017). *GLCM Texture: A Tutorial*. University of Calgary.
- Han, J., Kamber, M., & Pei, J. (2012). *Data Mining: Concepts and Techniques* (3rd ed.). Morgan Kaufmann.
- Haralick, R. M., Shanmugam, K., & Dinstein, I. (1973). Textural Features for Image Classification. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-3(6), 610–621. <https://doi.org/10.1109/TSMC.1973.4309314>
- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., et al. (2020). Array Programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- Herdiansah dkk. (2022). *Klasifikasi Citra Daun Herbal Dengan Menggunakan Backpropagation Neural Networks Berdasarkan Ekstraksi Ciri Bentuk*. [Lengkapi dengan nama jurnal/konferensi, volume, dan halaman.]
- Hsu, C.-W., Chang, C.-C., & Lin, C.-J. (2003). *A Practical Guide to Support Vector Classification*. Department of Computer Science, National Taiwan University.
- Jain, A. K., Duin, R. P. W., & Mao, J. (2000). Statistical Pattern Recognition: A Review. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(1), 4–37.
- John, G. H., & Langley, P. (1995). Estimating Continuous Distributions in Bayesian Classifiers. In *Proceedings of the Eleventh Conference on Uncertainty in Artificial Intelligence* (pp. 338–345). Morgan Kaufmann.
- Kuhn, M., & Johnson, K. (2013). *Applied Predictive Modeling*. Springer.
- Linsangan, N., Martin, M., Yumang, A., Cruz, F., Costa, A., Teng, K., & Villa, J. (2023). Leaf Shape Recognition Using Fourier Descriptors and Hough Transform and Classification Using Probabilistic Neural Network. *Proceedings of the 2023 6th International Conference on Digital Medicine and Image Processing*.
- McKinney, W. (2010). Data Structures for Statistical Computing in Python. In *Proceedings of the 9th Python in Science Conference* (pp. 56–61).

Nurhikam dkk. (2024). *Klasifikasi Tanaman Obat Berdasarkan Citra Daun Menggunakan Backpropagation Neural Networks*. [Lengkapi dengan nama jurnal/konferensi, volume, dan halaman.]

Ojala, T., Pietikäinen, M., & Mäenpää, T. (2002). Multiresolution Gray-Scale and Rotation Invariant Texture Classification with Local Binary Patterns. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 24(7), 971–987. <https://doi.org/10.1109/TPAMI.2002.1017623>

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Persoon, E., & Fu, K. S. (1977). Shape Discrimination Using Fourier Descriptors. *IEEE Transactions on Systems, Man, and Cybernetics*, 7(3), 170–179. <https://doi.org/10.1109/TSMC.1977.4309700>

Powers, D. M. W. (2011). Evaluation: From Precision, Recall and F-measure to ROC, Informedness, Markedness and Correlation. *Journal of Machine Learning Technologies*, 2(1), 37–63.

Prayoga dkk. (2023). *Klasifikasi Daun Herbal Menggunakan K-Nearest Neighbor dan Support Vector Machine dengan Fitur Fourier Descriptor*. [Lengkapi dengan nama jurnal/konferensi, volume, dan halaman.]

Qin, X., Zhang, Z., Huang, C., Dehghan, M., Zaiane, O. R., & Jagersand, M. (2020). U²-Net: Going Deeper with Nested U-Structure for Salient Object Detection. *Pattern Recognition*, 106, 107404. <https://doi.org/10.1016/j.patcog.2020.107404>

Russ, J. C. (2016). *The Image Processing Handbook* (7th ed.). CRC Press.

Shorten, C., & Khoshgoftaar, T. M. (2019). A Survey on Image Data Augmentation for Deep Learning. *Journal of Big Data*, 6(1), 60. <https://doi.org/10.1186/s40537-019-0197-0>

van der Walt, S., Schönberger, J. L., Nunez-Iglesias, J., Boulogne, F., Warner, J. D., Yager, N., Gouillart, E., & Yu, T. (2014). scikit-image: Image Processing in Python. *PeerJ*, 2, e453. <https://doi.org/10.7717/peerj.453>

Wardhani dkk. (2026). *Klasifikasi Daun Herbal Bertulang Menyirip Menggunakan K-Nearest Neighbor*. [Lengkapi dengan nama jurnal/konferensi, volume, dan halaman.]

Waskom, M. L. (2021). seaborn: Statistical Data Visualization. *Journal of Open Source Software*, 6(60), 3021. <https://doi.org/10.21105/joss.03021>

Wu, S. G., Bao, F. S., Xu, E. Y., Wang, Y.-X., Chang, Y.-F., & Xiang, Q.-L. (2007). A Leaf Recognition Algorithm for Plant Classification Using Probabilistic Neural Network. *2007 IEEE International Symposium on Signal Processing and Information Technology*, 11–16. <https://doi.org/10.1109/ISSPIT.2007.4458016>

Zahn, C. T., & Roskies, R. Z. (1972). Fourier Descriptors for Plane Closed Curves. *IEEE Transactions on Computers*, C-21(3), 269–281. <https://doi.org/10.1109/TC.1972.5008949>

LAMPIRAN

- **Link Repository Kode (GitHub / Google Colab):** <https://github.com/unproduktif/herbal-leaf-classification.git>
- **Sampel Visualisasi Hasil Prediksi Model (Prediksi Benar vs Prediksi Salah):** Fungsi-fungsi pendukung untuk memuat model tersimpan dan melakukan prediksi pada citra baru (di luar dataset latih/uji), beserta visualisasi setiap tahap pra-proses dan skor probabilitas tiap kelas, ditampilkan pada cell-cell di bawah ini. **BEST_SCENARIO** dan **BEST_MODEL** pada setiap contoh prediksi sebaiknya diisi sesuai skenario dan model dengan akurasi *test* terbaik dari Bagian 5.2, bukan nilai sementara yang digunakan saat eksperimen.

```
def load_image_rgb(image_path):  
    img = cv2.imread(image_path)  
    return cv2.cvtColor(img, cv2.COLOR_BGR2RGB)  
  
def load_all_models():  
    models = {}  
    scalers = {}  
    for scenario in range(1, 11):  
        models[scenario] = {  
            "SVM": joblib.load(f"svm_s{scenario}.pkl"),  
            "KNN": joblib.load(f"knn_s{scenario}.pkl"),  
            "Naive Bayes": joblib.load(f"nb_s{scenario}.pkl"),  
        }  
        scalers[scenario] = joblib.load(f"scaler_s{scenario}.pkl")  
    enc = joblib.load("label_encoder.pkl")  
    return models, scalers, enc  
  
def predict_best_model(image_path, target_scenario, target_model,  
                        models, scalers, encoder):  
  
    model = models[target_scenario][target_model]  
    scaler = scalers[target_scenario]  
  
    img_rgb = load_image_rgb(image_path)  
    img_rgb = cv2.resize(img_rgb, IMG_SIZE)  
    img_no_bg = remove_bg(img_rgb)  
    gray = grayscale(img_no_bg)  
    thresh = otsu_threshold(gray)  
    contour = contour_detection(thresh)  
  
    contour_overlay = img_rgb.copy()  
    if contour is not None:  
        cv2.drawContours(contour_overlay, [contour], -1, (255, 0, 0),  
2)  
  
    fig, axes = plt.subplots(1, 4, figsize=(16, 4))  
    axes[0].imshow(img_rgb); axes[0].set_title("Original");  
    axes[0].axis("off")
```

```

    axes[1].imshow(img_no_bg); axes[1].set_title("Remove BG");
    axes[1].axis("off")
    axes[2].imshow(thresh, cmap="gray");
    axes[2].set_title("Threshold"); axes[2].axis("off")
    axes[3].imshow(contour_overlay); axes[3].set_title("Contour");
    axes[3].axis("off")
    plt.tight_layout()
    plt.show()

    feature = extract_features_scenario(gray, contour,
target_scenario)
    feature = scaler.transform(feature.reshape(1, -1))

    pred_idx = model.predict(feature)[0]
    label = encoder.inverse_transform([pred_idx])[0]

    proba = model.predict_proba(feature)[0]
    class_labels = encoder.inverse_transform(model.classes_)
    proba_dict = {lab: round(p * 100, 2) for lab, p in
zip(class_labels, proba)}

    return label, proba_dict

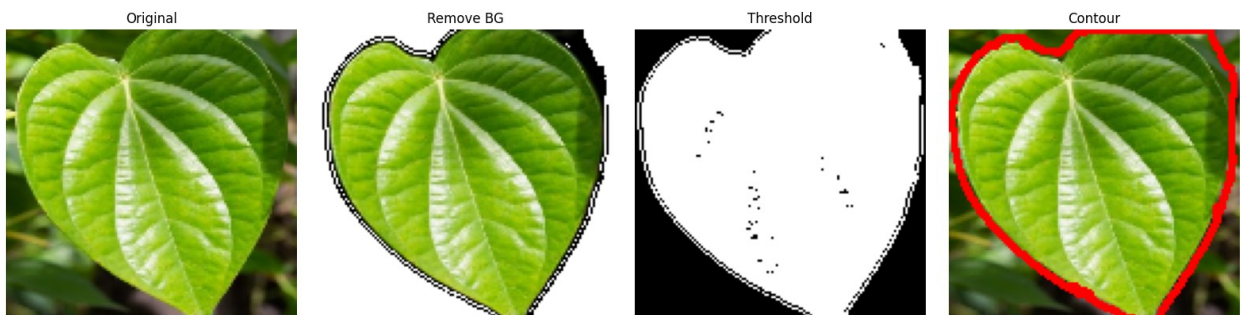
models, scalers, encoder = load_all_models()

TEST_IMAGE = "CITRA BARU/sirih.png"
BEST_SCENARIO = 8
BEST_MODEL = "SVM"

label, probabilitas = predict_best_model(TEST_IMAGE, BEST_SCENARIO,
BEST_MODEL, models, scalers, encoder)

print(f"Hasil Prediksi Utama: {label}")
print("\nSkor Kepercayaan (Probabilitas):")
for kelas, nilai in probabilitas.items():
    print(f"- {kelas}: {nilai}%")

```



Hasil Prediksi Utama: SIRIH

Skor Kepercayaan (Probabilitas):

- JAMBU: 0.29%
- KUNYIT: 0.13%
- PAKU: 1.68%
- SINGKONG: 0.14%
- SIRIH: 97.76%

```
TEST_IMAGE = "CITRA BARU/jambu1.png"
```

```
BEST_SCENARIO = 8
```

```
BEST_MODEL = "SVM"
```

```
label, probabilitas = predict_best_model(TEST_IMAGE, BEST_SCENARIO,  
BEST_MODEL, models, scalers, encoder)
```

```
print(f"Hasil Prediksi Utama: {label}")  
print("\nSkor Kepercayaan (Probabilitas):")  
for kelas, nilai in probabilitas.items():  
    print(f"- {kelas}: {nilai}%")
```

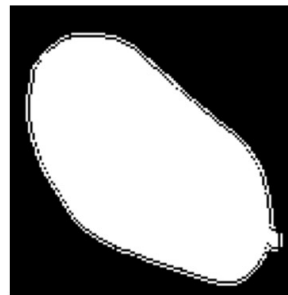
Original



Remove BG



Threshold



Contour



Hasil Prediksi Utama: JAMBU

Skor Kepercayaan (Probabilitas):

- JAMBU: 72.98%
- KUNYIT: 0.22%
- PAKU: 0.89%
- SINGKONG: 1.17%
- SIRIH: 24.75%

```
TEST_IMAGE = "CITRA BARU/singkong1.png"
```

```
BEST_SCENARIO = 8
```

```
BEST_MODEL = "SVM"
```

```
label, probabilitas = predict_best_model(TEST_IMAGE, BEST_SCENARIO,  
BEST_MODEL, models, scalers, encoder)
```

```
print(f"Hasil Prediksi Utama: {label}")  
print("\nSkor Kepercayaan (Probabilitas):")  
for kelas, nilai in probabilitas.items():  
    print(f"- {kelas}: {nilai}%")
```



Hasil Prediksi Utama: SINGKONG

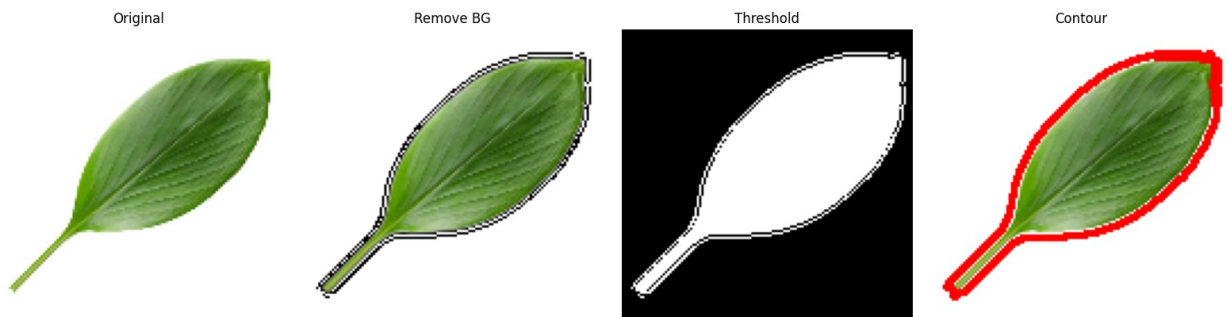
Skor Kepercayaan (Probabilitas):

- JAMBU: 0.28%
- KUNYIT: 0.04%
- PAKU: 21.96%
- SINGKONG: 75.24%
- SIRIH: 2.49%

```
TEST_IMAGE = "CITRA BARU/kunyit.png"
BEST_SCENARIO = 8
BEST_MODEL = "SVM"
```

```
label, probabilitas = predict_best_model(TEST_IMAGE, BEST_SCENARIO,
BEST_MODEL, models, scalers, encoder)
```

```
print(f"Hasil Prediksi Utama: {label}")
print("\nSkor Kepercayaan (Probabilitas):")
for kelas, nilai in probabilitas.items():
    print(f"- {kelas}: {nilai}%")
```



Hasil Prediksi Utama: KUNYIT

Skor Kepercayaan (Probabilitas):

- JAMBU: 1.17%
- KUNYIT: 80.51%
- PAKU: 2.79%
- SINGKONG: 8.73%
- SIRIH: 6.79%

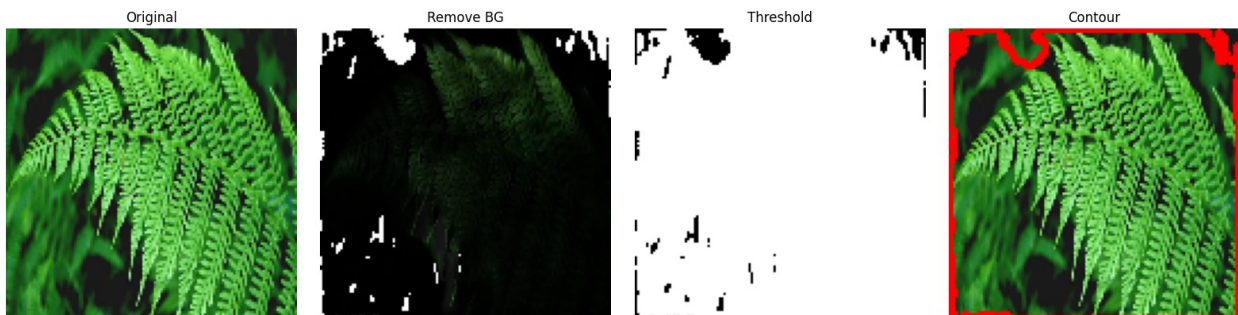
```

TEST_IMAGE = "CITRA BARU/paku.png"
BEST_SCENARIO = 8
BEST_MODEL = "SVM"

label, probabilitas = predict_best_model(TEST_IMAGE, BEST_SCENARIO,
BEST_MODEL, models, scalers, encoder)

print(f"Hasil Prediksi Utama: {label}")
print("\nSkor Kepercayaan (Probabilitas):")
for kelas, nilai in probabilitas.items():
    print(f"- {kelas}: {nilai}%")

```



Hasil Prediksi Utama: PAKU

Skor Kepercayaan (Probabilitas):

- JAMBU: 18.95%
- KUNYIT: 4.22%
- PAKU: 45.61%
- SINGKONG: 12.77%
- SIRIH: 18.45%

```

TEST_IMAGE = "CITRA BARU/paku1.jpg"
BEST_SCENARIO = 8
BEST_MODEL = "SVM"

```

```

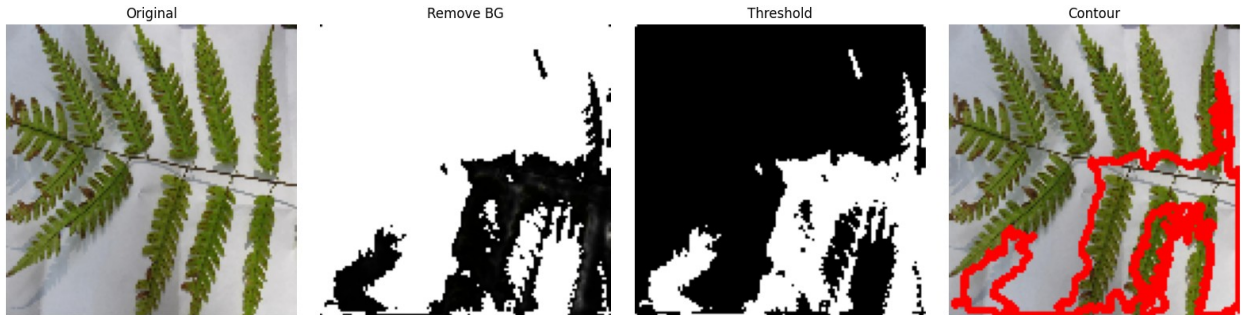
label, probabilitas = predict_best_model(TEST_IMAGE, BEST_SCENARIO,
BEST_MODEL, models, scalers, encoder)

```

```

print(f"Hasil Prediksi Utama: {label}")
print("\nSkor Kepercayaan (Probabilitas):")
for kelas, nilai in probabilitas.items():
    print(f"- {kelas}: {nilai}%")

```



Hasil Prediksi Utama: SINGKONG

Skor Kepercayaan (Probabilitas):

- JAMBU: 0.0%
- KUNYIT: 0.01%
- PAKU: 5.48%
- SINGKONG: 94.52%
- SIRIH: 0.0%

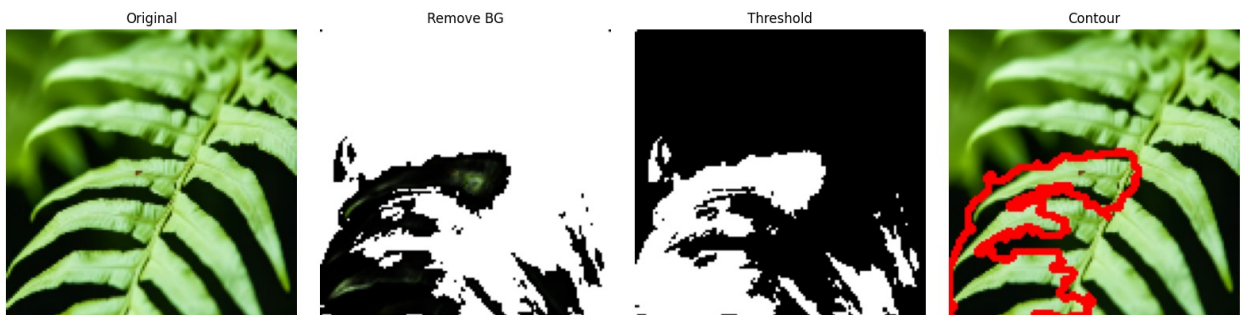
TEST_IMAGE = "CITRA BARU/paku2.png"

BEST_SCENARIO = 8

BEST_MODEL = "SVM"

```
label, probabilitas = predict_best_model(TEST_IMAGE, BEST_SCENARIO,
BEST_MODEL, models, scalers, encoder)
```

```
print(f"Hasil Prediksi Utama: {label}")
print("\nSkor Kepercayaan (Probabilitas):")
for kelas, nilai in probabilitas.items():
    print(f"- {kelas}: {nilai}%")
```



Hasil Prediksi Utama: PAKU

Skor Kepercayaan (Probabilitas):

- JAMBU: 0.0%
- KUNYIT: 0.0%
- PAKU: 78.45%
- SINGKONG: 21.55%
- SIRIH: 0.0%

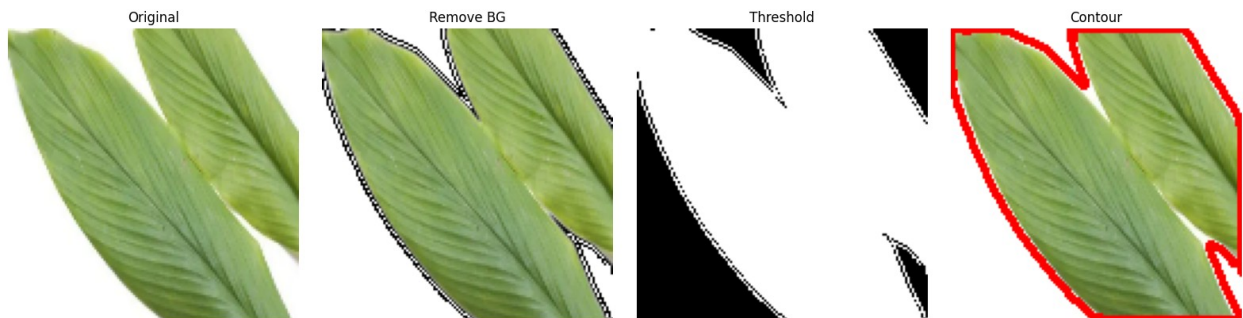

```

TEST_IMAGE = "CITRA BARU/kunyit1.png"
BEST_SCENARIO = 8
BEST_MODEL = "SVM"

label, probabilitas = predict_best_model(TEST_IMAGE, BEST_SCENARIO,
BEST_MODEL, models, scalers, encoder)

print(f"Hasil Prediksi Utama: {label}")
print("\nSkor Kepercayaan (Probabilitas):")
for kelas, nilai in probabilitas.items():
    print(f"- {kelas}: {nilai}%")

```



Hasil Prediksi Utama: JAMBU

Skor Kepercayaan (Probabilitas):

- JAMBU: 91.33%
- KUNYIT: 0.14%
- PAKU: 0.25%
- SINGKONG: 0.03%
- SIRIH: 8.26%